Subgraph Algorithmus Wiederverwendung

Von der Python-Referenz über `csubgraph` zu Gen-DB, Subgraph-SAT-Solver und Experimenten

Stephan Epp

`M.Sc.`

10. Oktober 2026

Zusammenfassung

Der Subgraph Algorithmus wurde ursprünglich in Python implementiert. Seither wurde er auf achtzehn verschiedene Weisen wiederverwendet: als C++-Neuimplementierung, als Verallgemeinerung auf Schichtstapel, als Bibliothek hinter einer C-Schnittstelle, als Kommandozeilenprogramm, als Orakel in einer SAT-Reduktion, als Vorbild für einen Paarindex und als Gegenstand einer Experimentierkette. Diese Arbeit führt alle Wiederverwendungen in einem Katalog (**W1** bis **W18**) auf, ordnet sie nach der Art der Wiederverwendung, zeigt, welche Teile des Algorithmus jeweils tatsächlich wiederverwendet werden, und belegt die Kernaussagen durch Beweise, ein durchgerechnetes Beispiel und eigene Prüfläufe. Ein zentrales Ergebnis ist die *Paarcharakterisierung* (Satz 2.2): Sie erklärt, warum fünf scheinbar verschiedene Wiederverwendungen im Kern dieselbe Relation berechnen. Ein UML-Sequenzdiagramm zeigt am Use Case einer Suche in Gen-DB, wie Browser, Webserver, Prozess-Pool, Python- und C++-Code zusammenspielen. Die Arbeit vergleicht, wie Gen-DB und `subgraph-sat-solver-econ` ihre C/C++-Bibliothek laden, und entwickelt daraus ausführlich das Konzept einer einheitlichen, betriebssystemunabhängigen gemeinsamen Bibliothek (`.sl`) mit standardisiertem Ladeverfahren. Sie wertet außerdem die Experimente zum Subgraph-SAT-Solver aus: Eine erste Messreihe mit 288 Läufen erweist sich als vom Prozessstart und von der Zeitauflösung bestimmt; eine neue Version mit korrigierter Kodierung, Basislinie, unerfüllbaren Instanzen und Referenzsolver zeigt für die gelösten ISCAS'85-Circuits ein etwa quadratisches Wachstum in der Literalzahl (Exponent $2{,}08 \pm 0{,}07$) und einen Ausfall ab 10 487 Literalen. Vierzehn Befunde mit Verbesserungsvorschlägen und ein umfangreicher Ausblick schließen die Arbeit ab.

Inhaltsverzeichnis

I Ausgangslage **1**

1 Einführung **2**
1.1 Fragestellung . . . 2
1.2 Arten von Aussagen in dieser Arbeit . . . 3
1.3 Aufbau der Arbeit . . . 3

2 Grundlagen **5**
2.1 Graphen, Adjazenzmatrizen und Schichtstapel . . . 5
2.2 Teilgraphen, Cliquen und Erfüllbarkeit . . . 5
2.3 Signaturen und die Relation des Algorithmus . . . 6
2.4 Die Paarcharakterisierung . . . 7
2.5 Das Subgraph-Orakel als abstrakte Schnittstelle . . . 8

3 Projektlandschaft und Wiederverwendungsformen **9**
3.1 Die Repositories . . . 9
3.2 Abhängigkeiten . . . 10
3.3 Eine Systematik der Wiederverwendung . . . 10

II Der Kern **13**

4 Die Python-Referenz **14**
4.1 Aufbau . . . 14
4.2 Der Rechenweg im Einzelnen . . . 15
4.3 Ein durchgerechnetes Beispiel . . . 16
4.4 Eigenschaften der berechneten Relation . . . 17
4.5 Laufzeit und Speicher der Referenz . . . 18
4.6 Die Rolle der Referenz in der Kette . . . 18
4.7 Grenzen der Referenz . . . 19

5 Die C++-Portierung: `csubgraph` **20**
5.1 Bauziele . . . 20
5.2 Das Modul `SubgraphAlgorithm` . . . 20
5.3 Abweichungen zwischen Python und C++ . . . 21

6 Erweiterungen: MultiOmics und ExactSubgraph **23**
6.1 Das Modul `MultiOmics` . . . 23
6.2 Das Modul `ExactSubgraph` . . . 24
6.2.1 Die Cliquensuche . . . 24

6.2.2 Die allgemeine Suche . . . 25
6.3 Die mengenbasierte Verallgemeinerung . . . 25

7 Außenflächen des Kerns **26**
7.1 Das JSON-Programm `subgraph-cli` (**W4**) . . . 26
7.2 Der Datei-Adapter `SolverCli` (**W5**) . . . 26
7.3 Die C-ABI für Gen-DB (**W7**, **W8**) . . . 27
7.4 Testsuite . . . 27

III Wiederverwendungen **28**

8 Der Katalog der Wiederverwendungen **29**
8.1 Übersicht . . . 29
8.2 Der Wiederverwendungsbaum . . . 30
8.3 Was wird tatsächlich wiederverwendet? . . . 31
8.4 Die Wiederverwendungen im Einzelnen . . . 31

9 Gen-DB **35**
9.1 Das Python-Paket als Abhängigkeit (**W6**) . . . 35
9.2 C-Wrapper und `ctypes` (**W7**, **W8**) . . . 35
9.3 Fallbacks: **W6** und **W9** . . . 36
9.4 Die Relation auf Mengenfolgen (**W10**) . . . 36
9.5 Der invertierte Paarindex (**W11**) . . . 36
9.6 Der Prozess-Pool (**W12**) . . . 37
9.7 Experimente und Differenzialtests (**W13**) . . . 37
9.8 Ablauf einer Suche: Sequenzdiagramm . . . 37
9.9 Messungen aus Gen-DBs eigenen Dateien . . . 40

10 Der Subgraph-SAT-Solver **43**
10.1 Die Reduktionskette . . . 43
10.2 Dateiprotokoll und Adapter . . . 44
10.3 Belegungsextraktion und Korrektheit (**W16**) . . . 44
10.4 Ein durchgerechnetes Beispiel . . . 45
10.5 Der Signaturmodus als Diagnosepfad (**W15**) und Grenzen . . . 45

11 Das Experimente-Framework **47**
11.1 Gegenstand und die beiden Messreihen . . . 47
11.1.1 Fragestellungen . . . 48
11.2 Kopplung an den Solver (**W17**) . . . 48
11.3 Die Instanzen . . . 49
11.3.1 Target-AND . . . 50
11.3.2 Target-AND-Random . . . 51
11.3.3 ISCAS'85 . . . 51
11.3.4 Miter-Instanzen . . . 51
11.3.5 Skalierungsinstanzen . . . 51
11.4 CNF-Kodierung und ihre Korrektur . . . 52
11.5 Messmethodik . . . 52
11.5.1 Erste Messreihe . . . 52

11.5.2 Neue Version 2.0.0 . . . 53
11.5.3 Vergleich der beiden Aufbauten . . . 54
11.5.4 Die beiden Solver . . . 54
11.5.5 Experiment-Suites der ersten Reihe . . . 54
11.5.6 Statistische Auswertung . . . 55
11.6 Die Umgebung der neuen Messreihe . . . 55
11.7 Ergebnisse der ersten Messreihe . . . 56
11.7.1 Laufzeit über der Instanzgröße (F1 bis F3) . . . 56
11.7.2 Vergleich der Typen und Suites (F4) . . . 57
11.7.3 Zeitliche Auflösung . . . 57
11.7.4 Solver-Ausgabe und Erfüllbarkeit . . . 57
11.7.5 Was die erste Messreihe nicht stützt . . . 58
11.8 Ergebnisse der neuen Version . . . 58
11.8.1 Datenbasis . . . 58
11.8.2 Basislinie . . . 58
11.8.3 Gesamtüberblick . . . 58
11.8.4 Target-AND und Skalierung . . . 59
11.8.5 Target-AND-Random . . . 60
11.8.6 ISCAS'85 . . . 60
11.8.7 Miter-Instanzen . . . 62
11.8.8 Vergleich der beiden Solver . . . 62
11.9 Vergleich der beiden Reihen . . . 62
11.10 Was die Messungen für diese Arbeit bedeuten . . . 63
11.11 Reproduktion und offene Punkte . . . 64

12 Vergleich der Kopplungsarten **66**
12.1 Vier Gestaltungsprinzipien . . . 66
12.2 Die Kopplungsarten nebeneinander . . . 66
12.3 Bewertung . . . 67

13 Laden von C/C++-Bibliotheken: zwei Verfahren und ein einheitliches Konzept **68**
13.1 Ausgangslage: ein Konzept, drei Namen . . . 68
13.2 Verfahren 1: Gen-DB baut den Wrapper beim ersten Gebrauch . . . 68
13.3 Verfahren 2: subgraph-sat-solver-econ lädt eine fertig gebaute Bibliothek . . . 71
13.4 Gegenüberstellung der beiden Verfahren . . . 72
13.5 Das Konzept: eine einheitliche gemeinsame Bibliothek mit standardisiertem Ladeverfahren . . . 73
13.5.1 Die Kernforderung . . . 73
13.5.2 Problemanalyse . . . 73
13.5.3 Ziele und Nichtziele . . . 75
13.5.4 Die Kernanforderung im Einzelnen: ein betriebssystemunabhängiges Ladeverfahren . . . 75
13.5.5 Das Format `.sl` . . . 81
13.5.6 Nutzen . . . 83
13.5.7 Einwände und ehrliche Einschätzung . . . 83
13.5.8 Umsetzungspfade . . . 84
13.5.9 Sicherheit . . . 85

13.5.10 Risiken . . . 85
13.5.11 Anwendung auf die beiden Beispielprojekte . . . 86
13.5.12 Empfohlener Fahrplan . . . 86
13.5.13 Offene Fragen . . . 87
13.5.14 Stand . . . 87

IV Prüfung **88**

14 Kompatibilitätsprüfung **89**
14.1 Aufbau . . . 89
14.2 Ergebnisse . . . 89
14.3 Interpretation . . . 89

15 Befunde und Verbesserungen **91**
15.1 Mängel der früheren Fassung dieser Arbeit . . . 92

16 Bau- und Aufrufanweisungen **93**
16.1 Bibliothek und Programme mit CMake . . . 93
16.2 Ohne CMake . . . 93
16.3 Adapter und Solver aufrufen . . . 93
16.4 Gen-DB mit der Bibliothek . . . 93

V Schluss **95**

17 Zusammenfassung **96**
17.1 Die Wiederverwendungen . . . 96
17.2 Die vier inhaltlichen Ergebnisse . . . 96
17.3 Das Zusammenspiel der Sprachen: das Sequenzdiagramm . . . 97
17.4 Das Laden der Bibliothek und das einheitliche Konzept . . . 97
17.5 Die Experimente . . . 98
17.6 Prüfung und Befunde . . . 98
17.7 Grenzen der Arbeit . . . 98

18 Ausblick **100**
18.1 Sofort umsetzbar: die Befunde beheben . . . 100
18.1.1 Die drei hohen Befunde . . . 100
18.1.2 Die mittleren und niedrigen Befunde . . . 100
18.1.3 Der Modus `heuristic` . . . 101
18.2 Der Subgraph Algorithmus und seine Verwandten . . . 101
18.2.1 Der Paarindex für weitere Relationen . . . 101
18.2.2 Laufzeit und Schranken . . . 101
18.2.3 Worst-Case-Instanzen . . . 102
18.2.4 Die 63-Knoten-Grenze . . . 102
18.3 Gen-DB und die Suche . . . 102
18.3.1 Neumessung unter Linux . . . 102
18.3.2 Der serielle Anteil der Suche . . . 103
18.3.3 Weitere Eingabearten . . . 103

18.3.4 Ein zweites Sequenzdiagramm . . . 103
18.4 Der Subgraph-SAT-Solver . . . 103
18.4.1 Ursache der Ausfälle ab 10 487 Literalen . . . 103
18.4.2 Speicherärmere Reduktion . . . 103
18.4.3 Unerfüllbare Instanzen . . . 104
18.4.4 Vergleich mit etablierten Solvern . . . 104
18.4.5 Alternative Orakel . . . 104
18.5 Das Experimente-Framework . . . 104
18.5.1 Metadaten in jeder Messreihe . . . 104
18.5.2 Vollständige Wiederholung des Full-Presets . . . 105
18.5.3 Mehr Messpunkte und bessere Fits . . . 105
18.5.4 Weitere Instanzfamilien . . . 105
18.5.5 Die Messreihen reproduzierbar halten . . . 105
18.6 Das einheitliche Laden von Bibliotheken . . . 105
18.6.1 Der Fahrplan in Schritten . . . 106
18.6.2 Den Nutzen messen . . . 106
18.6.3 Die offenen Fragen . . . 106
18.6.4 Zwischenlösung: ein gemeinsames Hilfspaket . . . 106
18.6.5 Ausweitung auf weitere Sprachen . . . 107
18.7 Verbesserungen an dieser Arbeit . . . 107
18.8 Eine Reihenfolge . . . 107

Teil I

Ausgangslage

Kapitel 1

Einführung

Ein Algorithmus, der an mehreren Stellen gebraucht wird, kann auf zwei Arten in Software verankert werden: Jedes Projekt bringt seine eigene Implementierung mit, oder es gibt eine einzige Implementierung, die alle anderen Projekte über eine schmale Schnittstelle benutzen. Die erste Variante ist zunächst einfacher, führt aber dazu, dass Korrekturen, Erweiterungen und Messergebnisse an jeder Stelle einzeln nachgezogen werden müssen. Die zweite Variante verlangt Disziplin bei der Schnittstelle, hat aber den Vorteil, dass jedes Projekt genau das ist, was sein Name sagt: ein Projekt über seine eigene Aufgabe.

Der *Subgraph Algorithmus* [1] ist ein Beispiel für beides. Er wurde in Python implementiert (Repository `github.com/naphez86/subgraph`). Die C++-Bibliothek `github.com/naphez86/csubgraph` ist bereits die erste Wiederverwendung: eine effizientere Neuimplementierung, die dann selbst wieder zum gemeinsamen Kern wurde. Auf ihr bauen die Biologie-Datenbank `github.com/naphez86/gen-db`, der `github.com/naphez86/subgraph-sat-solver` und das Experimente-Framework auf. Hinzu kommen Wiederverwendungen *innerhalb* dieser Projekte, etwa die Python-Neuimplementierung der Mehrschicht-Relation in Gen-DB als Fallback und der invertierte Paarindex, der Vergleiche ganz vermeidet.

1.1 Fragestellung

Die Arbeit beantwortet acht Fragen.

1. Welche Wiederverwendungen des Subgraph Algorithmus gibt es in den vier hochgeladenen Projekten (`subgraph`, `csubgraph`, `gen-db`, `subgraph-sat-solver`) und im Experimente-Framework, dessen Quelltext nicht geprüft wurde, dessen eigene wissenschaftliche Arbeit aber ausgewertet ist? (Katalog in Kapitel 8)

2. Von welcher *Art* ist jede Wiederverwendung, und was genau wird wiederverwendet: der Algorithmus, seine Relation, nur die Bibliothek oder nur die Infrastruktur? (Kapitel 3 und 8)

3. Welche formalen Aussagen tragen die Wiederverwendungen? Insbesondere: Warum berechnen die Mehrschicht-Relation, ihre Python-Neuimplementierung, die mengenbasierte Verallgemeinerung und der Paarindex dasselbe? (Kapitel 2 und 9)

4. Welche Kopplungsart passt zu welcher Anwendung? (Kapitel 12)

5. Wie laden die Projekte ihre C/C++-Bibliothek, worin unterscheiden sich die Verfahren von Gen-DB und `subgraph-sat-solver-econ`, und wie ließe sich das Ladeverhalten betriebssystemunabhängig vereinheitlichen? (Kapitel 13)

6. Wie spielen Browser, Webserver, Prozess-Pool sowie Python- und C++-Code bei einer Suche in Gen-DB zusammen? (Sequenzdiagramm in Abschnitt 9.8)

7. Was zeigen die Experimente zum Subgraph-SAT-Solver, und was zeigen sie nicht? (Kapitel 11)

8. Verträgt sich die Kette noch, wenn der Kern wächst, und welche Schwachstellen hat sie? (Kapitel 14 und 15)

1.2 Arten von Aussagen in dieser Arbeit

Die Arbeit unterscheidet vier Arten von Aussagen und kennzeichnet sie im Text.

- *Bewiesen:* Sätze und Lemmata mit Beweis, etwa die Korrektheit der Reduktion, die Paarcharakterisierung oder die Korrektheit der Belegungsextraktion.
- *Aus dem Quelltext gelesen:* Eigenschaften, die sich aus den hochgeladenen Quellen ergeben, etwa Funktionssignaturen, Rückgabecodes oder Standardwerte. Sie gelten für den untersuchten Stand der Repositories.
- *Geprüft:* Ergebnisse eigener Läufe unter Linux (GCC 13), mit aus den Quellen neu gebauten Programmen (Kapitel 14). Sie belegen Verträglichkeit und Korrektheit der geprüften Instanzen, keine allgemeinen Laufzeitaussagen.
- *Offen:* Aussagen, die einen weiteren Lauf, einen nicht vorliegenden Quelltext oder eine Entscheidung erfordern. Sie sind im Text rot gekennzeichnet, etwa **[offen]**.

Die Trennung ist wichtig, weil der Gegenstand zwei leicht zu verwechselnde Dinge verbindet: den Signaturalgorithmus mit Laufzeit $O(n^3)$ und einen exakten Cliquen- und Teilgraphtest, dessen Laufzeit im ungünstigsten Fall exponentiell ist (Kapitel 5 und 6).

1.3 Aufbau der Arbeit

Die Arbeit hat fünf Teile (Abbildung 1.1).

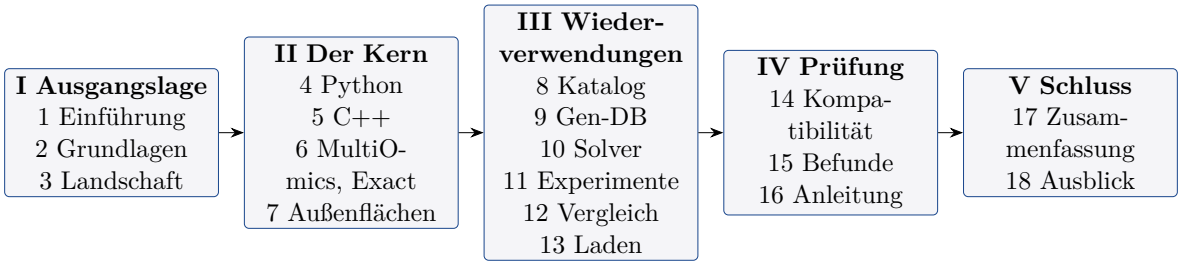


Abbildung 1.1: Gliederung der Arbeit. Teil II beschreibt, was wiederverwendet wird, Teil III, *wie* und *wo*, Teil IV, ob es hält, Teil V, was daraus folgt.

Teil I legt Begriffe, Projektlandschaft und eine Systematik der Wiederverwendungsformen fest. Teil II beschreibt den Kern in seinen drei Entwicklungsstufen (Python-Referenz, C++-Portierung, Erweiterungen) und seine Außenflächen. Teil III ist der eigentliche Gegenstand. Er beginnt mit dem Katalog aller Wiederverwendungen. Es folgt das Kapitel zu Gen-DB mit dem Paarindex, den Messungen und dem UML-Sequenzdiagramm einer Suche, das das Zusammenspiel von Python und C++ zeigt (Abschnitt 9.8). Das Kapitel zum Subgraph-SAT-Solver behandelt Reduktionskette, Protokoll und Belegungsextraktion. Das Kapitel zu den Experimenten beschreibt Instanzen, Kodierung, Messmethodik und die Ergebnisse beider Messreihen und ordnet sie ein (Kapitel 11). Danach kommt der Vergleich der Kopplungsarten. Den Abschluss des Teils bildet das Laden der C/C++-Bibliothek: die zwei Verfahren von Gen-DB und `subgraph-sat-solver-econ` und das ausführlich beschriebene Konzept eines einheitlichen, betriebssystemunabhängigen Ladeverhaltens (Kapitel 13). Teil IV prüft die Kette, berichtet vierzehn Befunde mit Verbesserungen und gibt Bau- und Aufrufanweisungen. Teil V fasst die Ergebnisse zusammen (Kapitel 17) und gibt einen ausführlichen Ausblick auf Befunde, Algorithmus, Gen-DB, Solver, Experimente und das einheitliche Laden (Kapitel 18).

Kapitel 2

Grundlagen

2.1 Graphen, Adjazenzmatrizen und Schichtstapel

Definition 2.1 (Graph und Adjazenzmatrix)**.** Ein *ungerichteter einfacher Graph* ist ein Paar $G = (V, E)$ mit endlicher Knotenmenge $V = \{v_0, \ldots, v_{n-1}\}$ und einer Kantenmenge E aus zweielementigen Teilmengen von V. Seine Adjazenzmatrix $A \in \{0,1\}^{n \times n}$ hat $A_{ij} = A_{ji} = 1$ genau dann, wenn $\{v_i, v_j\} \in E$, und $A_{ii} = 0$. Ein *gerichteter* Graph hat dagegen $E \subseteq V \times V$ und eine im Allgemeinen nicht symmetrische Matrix.

Der Algorithmus und `csubgraph` arbeiten auf Adjazenzmatrizen (Python: `numpy`-Arrays, C++: `std::vector<std::vector<int>>`). Gen-DB speichert biologische Netzwerke als gerichtete Graphen; der SAT-Solver erzeugt ungerichtete Graphen und der Adapter trägt Kanten symmetrisch ein (Abschnitt 7.2).

Definition 2.2 (Schichtstapel)**.** Ein *Schichtstapel* mit $L \geq 1$ Schichten über derselben geordneten Knotenmenge der Größe n ist ein Tupel $\mathbf{A} = (A^{(1)}, \ldots, A^{(L)})$ von Adjazenzmatrizen $A^{(\ell)} \in \{0,1\}^{n \times n}$. Für $L = 1$ ist ein Schichtstapel eine gewöhnliche Adjazenzmatrix.

2.2 Teilgraphen, Cliquen und Erfüllbarkeit

Definition 2.3 (Teilgraph-Monomorphismus)**.** Seien $H = (V_H, E_H)$ und $G = (V_G, E_G)$ Graphen. H ist *in G enthalten* (als nicht notwendig induzierter Teilgraph), geschrieben $H \sqsubseteq_{\text{mono}} G$, wenn es eine injektive Abbildung $\varphi : V_H \to V_G$ gibt mit $\{u, v\} \in E_H \Rightarrow \{\varphi(u), \varphi(v)\} \in E_G$. Die Abbildung φ heißt *Einbettung*.

Das ist der Begriff, den das Modul `ExactSubgraph` entscheidet. Er ist vom Begriff des *induzierten* Teilgraphen zu unterscheiden; für vollständige Muster fallen beide zusammen, weil jede Einbettung eines vollständigen Graphen automatisch induziert ist.

Definition 2.4 (Clique)**.** Eine *k-Clique* in G ist eine Knotenmenge $C \subseteq V_G$ mit $|C| = k$, in der je zwei verschiedene Knoten benachbart sind. K_k bezeichnet den vollständigen Graphen auf k Knoten.

Lemma 2.1 (Clique als Teilgraph)**.** G enthält genau dann eine k-Clique, wenn $K_k \sqsubseteq_{\text{mono}} G$.

Beweis. Ist C eine k-Clique, so ist jede Bijektion von $V(K_k)$ auf C eine Einbettung. Ist umgekehrt φ eine Einbettung von K_k, so sind die k Bildknoten wegen der Injektivität

verschieden, und je zwei von ihnen sind benachbart, weil in K_k je zwei Knoten benachbart sind. Das Bild ist also eine k-Clique. □

Definition 2.5 (CNF, 3-SAT)**.** Eine Formel in konjunktiver Normalform über den Variablen $x_1, \dots, x_N$ ist eine Konjunktion $\varphi = C_1 \wedge \cdots \wedge C_m$ von *Klauseln*, von denen jede eine Disjunktion von *Literalen* x_i oder $\neg x_i$ ist. Eine *Belegung* $\alpha : \{x_1, \dots, x_N\} \to \{\text{wahr}, \text{falsch}\}$ *erfüllt* φ, wenn jede Klausel mindestens ein unter α wahres Literal enthält. Bei *3-SAT* hat jede Klausel höchstens drei Literale.

Die Eingabe des Solvers ist das DIMACS-Format: eine Kopfzeile `p cnf N m` und je Klausel eine Zeile ganzer Zahlen, mit `0` abgeschlossen; ein negatives Vorzeichen bedeutet Negation.

Satz 2.1 (3-SAT auf Clique)**.** Sei $\varphi = C_1 \wedge \cdots \wedge C_m$ eine 3-SAT-Formel mit $m \geq 1$. Der Graph G_φ habe als Knoten die Paare (i, p) mit $i \in \{1, \dots, m\}$ und p einer Position in C_i; $\lambda(i, p)$ sei das Literal dort. Zwei Knoten (i, p), (j, q) seien genau dann benachbart, wenn $i \neq j$ und $\lambda(i, p) \neq \neg\lambda(j, q)$. Dann ist φ genau dann erfüllbar, wenn G_φ eine m-Clique enthält.

Beweis. ($\Rightarrow$) Sei α eine erfüllende Belegung. Wähle je Klausel C_i eine Position p_i, deren Literal unter α wahr ist. Die Knoten (i, p_i) liegen in verschiedenen Klauseln. Zwei von ihnen können keine komplementären Literale tragen, denn dann wäre eines unter α falsch. Also sind je zwei der m Knoten benachbart.

($\Leftarrow$) Sei Q eine m-Clique. Je zwei Knoten von Q liegen in verschiedenen Klauseln; bei m Knoten und m Klauseln enthält Q also aus jeder Klausel genau einen Knoten (i, p_i). Die Literale $\lambda(i, p_i)$ enthalten kein komplementäres Paar. Setze jedes auf wahr (das ist konsistent) und die übrigen Variablen beliebig. Dann ist jede Klausel erfüllt. □

Korollar 2.1 (Kette bis zum Subgraph-Problem)**.** φ ist genau dann erfüllbar, wenn $K_m \sqsubseteq_{\text{mono}} G_\varphi$.

Beweis. Satz 2.1 und Lemma 2.1. □

Das Korollar sagt nichts über die *Laufzeit* des Teilgraphtests. Das Clique-Problem ist NP-vollständig [2, 3]; ein Verfahren, das den Test für alle Eingaben in polynomieller Zeit entscheidet, würde P = NP bedeuten. Ein solches Verfahren enthält keines der Projekte (Abschnitt 10.5).

2.3 Signaturen und die Relation des Algorithmus

Der Subgraph Algorithmus [1] vergleicht Adjazenzmatrizen nicht über Einbettungen, sondern über Signaturen.

Definition 2.6 (Spaltensignatur, Zeilenkomponente)**.** Für $A \in \{0,1\}^{n \times n}$ ist die Signatur der Spalte j

$$\sigma_j(A) = \sum_{i=0}^{n-1} A_{ij}\, 2^i + j\, 2^n.$$

Die *Zeilenkomponente* ist $\rho_j(A) = \sigma_j(A) \bmod 2^n = \sum_{i=0}^{n-1} A_{ij} 2^i$, also die binär kodierte Spalte ohne Positionsanteil. Wir schreiben $\rho(A) = (\rho_0(A), \dots, \rho_{n-1}(A))$.

Die Zeilenkomponente $\rho_j(A)$ kodiert bijektiv die Menge $\{i : A_{ij} = 1\}$, die In-Nachbarschaft des Knotens j. Das ist die Brücke zur mengenbasierten Verallgemeinerung in Abschnitt 6.3.

Definition 2.7 (Rotation, Teilfolgen-Test)**.** Für eine Folge $b = (b_0, \ldots, b_{q-1})$ ist die Rotation um r nach rechts $\mathrm{rot}_r(b)_i = b_{(i-r) \bmod q}$. Für zwei Folgen a (Länge p) und b (Länge $q \geq p$) gilt $a \preceq b$, wenn es eine Rotation $r \in \{0, \ldots, q-1\}$ gibt, für die a und $\mathrm{rot}_r(b)$ eine gemeinsame *zusammenhängende* Teilfolge der Länge mindestens 2 haben. Für Matrizen A, B mit $n_A \leq n_B$ gilt $A \preceq B$, wenn $\rho(A) \preceq \rho(B)$.

Der Name des Hilfsverfahrens `computeLCS` ist irreführend: Der Code berechnet den längsten gemeinsamen *Teilstring* (zusammenhängend), nicht die längste gemeinsame Teilfolge; bei einem Unterschied wird der Tabellenwert auf 0 zurückgesetzt (Abschnitt 5.2). Die Relation $\preceq$ ist *nicht* die klassische Teilgraph-Relation; ein Beispiel zeigt den Unterschied.

Beispiel 2.1 ($K_2 \sqsubseteq_{\text{mono}} K_3$, aber $K_2 \not\preceq K_3$)**.** Seien A und B die Adjazenzmatrizen von K_3 und K_2:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Klassisch gilt $K_2 \sqsubseteq_{\text{mono}} K_3$. Die Zeilenkomponenten sind $\rho(A) = (6, 5, 3)$ und $\rho(B) = (2, 1)$. Die Folgen haben kein gemeinsames Element, die gemeinsame zusammenhängende Teilfolge hat Länge $0 < 2$. Der Algorithmus meldet daher keine Enthaltensein-Beziehung (`KEEP_BOTH`).

Das Beispiel ist keine Kritik am Algorithmus, sondern eine Warnung vor einer Verwechslung: Der Algorithmus beantwortet die in [1] definierte Frage, nicht das klassische Teilgraphproblem. Kapitel 10 zeigt die praktische Folge.

2.4 Die Paarcharakterisierung

Die Relation $\preceq$ lässt sich ohne Rotationen und ohne Tabelle beschreiben. Dieser Satz ist der gemeinsame Nenner mehrerer Wiederverwendungen (**W2**, **W9**, **W10**, **W11**, Kapitel 8).

Definition 2.8 (Paarmengen)**.** Für eine Folge $s = (s_0, \ldots, s_{m-1})$ über einer beliebigen Menge sind

$$P_{\text{lin}}(s) = \{(s_j, s_{j+1}) : 0 \leq j \leq m-2\}, \qquad P_{\text{cyc}}(s) = \{(s_j, s_{(j+1) \bmod m}) : 0 \leq j \leq m-1\}.$$

Für $m \leq 1$ sind beide leer.

Satz 2.2 (Paarcharakterisierung)**.** Seien a und b Folgen der Längen $p \leq q$. Dann gilt $a \preceq b$ genau dann, wenn $P_{\text{lin}}(a) \cap P_{\text{cyc}}(b) \neq \emptyset$.

Beweis. ($\Rightarrow$) Gilt $a \preceq b$, so gibt es r und eine gemeinsame zusammenhängende Teilfolge der Länge ≥ 2 von a und $\mathrm{rot}_r(b)$. Sie enthält zwei aufeinanderfolgende Einträge: $a_i = \mathrm{rot}_r(b)_j$ und $a_{i+1} = \mathrm{rot}_r(b)_{j+1}$ mit $j+1 \leq q-1$. Wegen $\mathrm{rot}_r(b)_j = b_{(j-r) \bmod q}$ ist $(a_i, a_{i+1}) = (b_k, b_{(k+1) \bmod q})$ mit $k = (j-r) \bmod q$. Das Paar liegt in $P_{\text{lin}}(a) \cap P_{\text{cyc}}(b)$.

($\Leftarrow$) Sei $(a_i, a_{i+1}) = (b_k, b_{(k+1) \bmod q})$. Wegen $P_{\text{lin}}(a) \neq \emptyset$ ist $p \geq 2$, also $q \geq 2$. Mit $r = (q-k) \bmod q$ steht b_k an Position 0 und $b_{(k+1) \bmod q}$ an Position 1 von $\mathrm{rot}_r(b)$. Damit ist (a_i, a_{i+1}) eine gemeinsame zusammenhängende Teilfolge der Länge 2 von a und $\mathrm{rot}_r(b)$. □

Aus dem Satz folgen zwei Dinge. Erstens genügt zur Entscheidung von $\preceq$ ein Mengenschnitt, in erwarteter Zeit $O(p+q)$ statt $O(q \cdot pq)$ durch alle Rotationen mit je einer Tabelle. Zweitens verwendet die Aussage nur die Gleichheit der Folgenglieder, nicht ihre Natur. Sie gilt deshalb ebenso für Folgen von Tupeln (Schichtstapel, Modus *kohärent*) und für Folgen von Mengen (Abschnitt 6.3) und lässt sich in einen invertierten Index umsetzen (Abschnitt 9.5). Die Aussage wurde zusätzlich an 200 000 Zufallspaaren kurzer Folgen gegen die Rotations-und-Tabellen-Fassung geprüft, ohne Abweichung (Kapitel 14).

2.5 Das Subgraph-Orakel als abstrakte Schnittstelle

Die Anwendungen hängen nicht vom Algorithmus ab, sondern von einer Schnittstelle.

Definition 2.9 (Subgraph-Orakel)**.** Ein *Subgraph-Orakel* ist ein Verfahren $\mathcal{O}$, das für ein Paar ungerichteter Graphen (G, H) entweder *nein* oder *ja* liefert, bei *ja* optional mit einer Einbettung von H in G. $\mathcal{O}$ heißt *korrekt*, wenn aus *ja* stets $H \sqsubseteq_{\text{mono}} G$ folgt und eine gelieferte Einbettung tatsächlich eine ist, und *vollständig*, wenn aus $H \sqsubseteq_{\text{mono}} G$ stets *ja* folgt.

Der SAT-Solver braucht ein korrektes und vollständiges Orakel mit Einbettung, denn nur dann folgt aus Korollar 2.1, dass *ja* erfüllbar und *nein* unerfüllbar bedeutet. `ExactSubgraph` erfüllt das, der Signaturmodus nicht (Beispiel 2.1, Abschnitt 10.4). GenDB braucht dagegen kein Orakel, sondern die Relation $\preceq$ selbst und ihre mehrschichtigen Fassungen.

Kapitel 3

Projektlandschaft und Wiederverwendungsformen

3.1 Die Repositories

Tabelle 3.1 fasst Aufgabe und Rolle der fünf Projekte zusammen; die letzte Spalte beantwortet, wer den Algorithmus *implementiert* und wer ihn nur benutzt. Der Quelltext des Experimente-Frameworks wurde nicht geprüft; Kapitel 11 beschreibt es nach seiner eigenen wissenschaftlichen Arbeit.

Repository	Sprache	Aufgabe	Algorithmus enthalten?
`github.com/naphez86/subgraph`	Python	Referenzimplementierung, Klasse `Subgraph`	ja (Signaturen, Rotation)
`github.com/naphez86/csubgraph`	C++17	statische Bibliothek mit `SubgraphAlgorithm`, `MultiOmics`, `ExactSubgraph`; Programme `subgraph-cli` und `subgraph`	ja (drei Module)
`github.com/naphez86/gen-db`	Python (FastAPI, PostgreSQL)	Datenbank biologischer Netzwerke mit Subgraph-, Multi-Omics- und universeller Suche	ja, als Neuimplementierung der Relation (`multiomics_python`, `universal_core`)
`github.com/naphez86/subgraph-sat-solver`	C++17	Reduktionskette CNF → 3-SAT → Clique → Subgraph	nein (ruft externes Programm)
`github.com/naphez86/subgraph-sat-solver-experiments`	Python	Messframework	nein (startet den Solver)

Tabelle 3.1: Die fünf Projekte. Gegenüber der früheren Fassung dieser Arbeit enthält GenDB *doch* eigene Realisierungen der Relation.

3.2 Abhängigkeiten

Abbildung 3.1 zeigt die Abhängigkeiten. Durchgezogene Pfeile sind Laufzeit- oder Bauabhängigkeiten, gestrichelte Pfeile Rückfallwege oder Geschwisterbeziehungen.

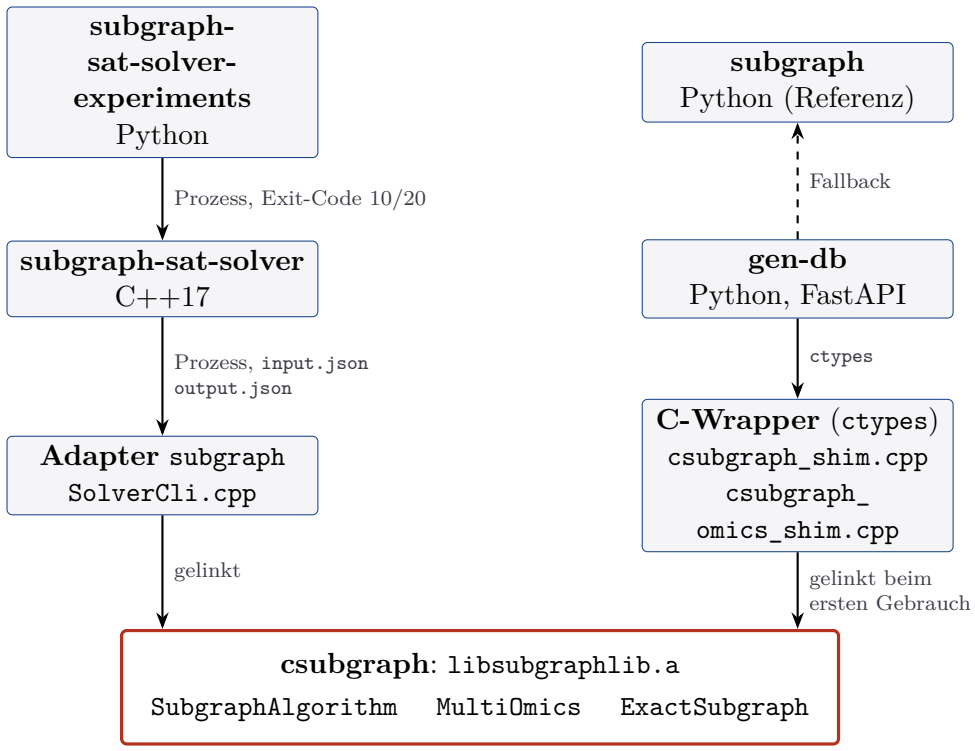


Abbildung 3.1: Abhängigkeiten der Projekte. Der gestrichelte Bogen von `csubgraph` zu `subgraph` deutet die Portierung an (**W1**); die Abhängigkeit des Solvers vom Adapter ist eine Prozessgrenze.

3.3 Eine Systematik der Wiederverwendung

Die achtzehn Wiederverwendungen unterscheiden sich darin, *was* sie vom Ursprung übernehmen. Die Arbeit verwendet sieben Formen.

Die in Tabelle 3.2 vorgenommene Systematisierung der insgesamt achtzehn Wiederverwendungen bildet das architektonische und methodische Rückgrat dieser Arbeit. Sie verdeutlicht, dass Wiederverwendung im wissenschaftlichen Softwarebau kein monolithischer Block ist, sondern sich auf fundamental unterschiedliche Dimensionen und Abstraktionsebenen stützen kann. Die zugrundeliegende Taxonomie unterscheidet exakt sieben Formen, die jeweils eigene Eigenschaften hinsichtlich Kopplung, Laufzeit, Verifikationsaufwand und Wartbarkeit aufweisen.

Detaillierte Betrachtung der sieben Wiederverwendungsformen

– **Portierung (W1):** Bei dieser Form wird ein bestehender Algorithmus in eine andere Programmiersprache übertragen, wobei die Semantik und die mathematischen Ergebnisse exakt gewahrt bleiben. Im Zentrum steht hier die C++-Neuimplementierung des Algorithmus, wodurch zwar die gleiche funktionale Kor-

rektheit wie im Python-Original garantiert wird, sich jedoch drastische Verbesserungen in der Ausführungsgeschwindigkeit und Speichereffizienz ergeben.

- **Verallgemeinerung (W2, W10):** Hierbei wird die Kernlogik auf einen erweiterten Eingabebereich oder komplexere Datenstrukturen (wie geschichtete Graphen oder abstrakte Mengenfolgen) ausgedehnt. Das wesentliche Kennzeichen ist, dass der ursprüngliche Spezialfall als exakter Unterfall enthalten bleibt und zwingend die identischen Resultate liefert. Dies betrifft unter anderem die MultiOmics-Schichtstapel und die formalen Mengenfolgen.

- **Neuimplementierung als Fallback (W9):** Tritt der Fall ein, dass fremde Code-Abhängigkeiten in einer bestimmten Umgebung (wie Gen-DB) vermieden werden sollen oder müssen, wird die Relation in der Zielsprache eigenständig neu implementiert. Diese Fassungen besitzen keinen gemeinsamen Quelltext mit dem Original, weshalb ihre Korrektheit nicht strukturell garantiert ist, sondern permanent durch automatisierte Tests gegen das Original abgesichert werden muss.

- **Bibliotheksnutzung (W6, W7, W8):** Diese Form beschreibt die klassische Einbindung von Programmbibliotheken im selben Ausführungsprozess. Durch direkte API-Aufrufe, C-Wrapper oder Mechanismen wie `ctypes` wird eine hohe Ausführungsgeschwindigkeit bei gleichzeitiger Bindung an eine bestimmte Application Binary Interface (ABI) erreicht. Die Python-Paketabhängigkeit von Gen-DB fällt ebenso hierunter wie die performanten C-Schnittstellen.

- **Prozessnutzung (W4, W5, W14, W15, W17):** Wenn Komponenten als eigenständige Dienste oder CLI-Anwendungen gekapselt werden, erfolgt die Interaktion über standardisierte Protokolle (Dateisystem, JSON über Standard-Ein-/Ausgabe). Dies erzeugt eine saubere architektonische Systemgrenze, zieht jedoch systembedingt höhere Aufrufkosten (Inter-Process Communication, Serialization) nach sich. Beispiele reichen von den Subgraph-CLIs über Solver-Adapter bis hin zur Experimente-Infrastruktur.

- **Infrastruktur (W3, W12, W13):** In dieser Kategorie wird nicht der eigentliche mathematische Algorithmus wiederverwendet, sondern die begleitende Umgebung wie Build-Systeme, parallele Prozess-Pools (`compare_many`) oder Test-Frameworks für Differenzialtests. Diese Elemente stützen den Entwicklungsprozess und die Qualitätssicherung, ohne direkt in die Evaluation der Relation einzufließen.

- **Begriffliche Wiederverwendung (W11, W16, W18):** Die schwächste, aber konzeptionell oft wirkungsvollste Form basiert auf dem Transfer reiner Ideen und theoretischer Konzepte—wie etwa der Paarcharakterisierung oder der Extraktion von Einbettungen. Hier existiert kein gemeinsamer Quelltext; stattdessen dient das zugrundeliegende Theorem als Blaupause für eigenständige Implementierungen in anderen Domänen.

Praktische Konsequenzen für Wartung und Evolution

Die Differenzierung nach diesen sieben Formen ist keineswegs nur akademischer Natur, sondern hat unmittelbare und weitreichende Auswirkungen auf die Wartbarkeit, Robustheit und Zukunftssicherheit des Gesamtsystems. Wenn sich im Zuge der Weiterentwick-

lung Änderungen an der Kernlogik ergeben—beispielsweise eine Modifikation des Schwellenwerts (≥ 2) im Vergleichsalgorithmus oder eine Anpassung der Rotationsrichtung–, schlagen sich diese Änderungen je nach Verwendungsform völlig unterschiedlich nieder:

1. **Manuelle Nachpflege und Validierung:** Komponenten der Formen *Portierung*, *Verallgemeinerung* und *Neuimplementierung* besitzen eine direkte algorithmische Abhängigkeit. Jede Änderung am Original erfordert hier ein manuelles Nachziehen des Quelltextes und eine erneute, strenge Verifikation (z. B. durch Differenzialtests), um Regressionen auszuschließen.

2. **Automatische Vererbung:** Module, die auf *Bibliotheks-* oder *Prozessnutzung* basieren, entkoppeln die Ausführung physisch oder prozessual. Wird die zugrundeliegende Bibliothek aktualisiert oder neu kompiliert, erben diese Module die algorithmischen Anpassungen vollkommen automatisch, ohne dass Anpassungen an ihrer eigenen Schnittstelle nötig sind.

3. **Vollständige Unabhängigkeit:** Schließlich bleiben Anteile der Kategorien *Infrastruktur* und *Begriffliche* von rein algorithmischen Änderungen inhaltlich gänzlich unberührt, da sie auf abstrakteren Ebenen operieren oder lediglich organisatorische Funktionen erfüllen.

Durch dieses Zusammenspiel erlaubt es der Katalog, die evolutionären Kosten von Softwareänderungen präzise abzuschätzen und gezielt dort Test- und Prüfkapazitäten zu konzentrieren, wo manuelle Nachbesserungen zwingend erforderlich sind.

Form	Was wird übernommen?	Kennzeichen	Beispiele
Portierung	Algorithmus und Ergebnisse, neue Sprache	gleiche Funktion, andere Laufzeit	**W1**
Verallgemeinerung	Algorithmus, auf größeren Eingabebereich	Spezialfall liefert das alte Ergebnis	**W2**, **W10**
Neuimplementierung als Fallback	Relation in Sprache der Anwendung	muss gegen Original geprüft werden	**W9**
Bibliotheksnutzung	Code der Bibliothek, im selben Prozess	schnell, ABI-gebunden	**W6**, **W7**, **W8**
Prozessnutzung	Programm als Dienst	klare Grenze, hohe Aufrufkosten	**W4**, **W5**, **W14**, **W15**, **W17**
Infrastruktur	Bau, Adapter, Bibliotheksform, aber nicht der Algorithmus	Teil des Pakets, nicht der Relation	**W3**, **W12**, **W13**
Begriffliche	Idee (Paarcharakterisierung) statt Code	kein gemeinsamer Quelltext	**W11**, **W16**, **W18**

Tabelle 3.2: Sieben Formen der Wiederverwendung. Kapitel 8 ordnet jede der achtzehn Wiederverwendungen genau einer Form zu.

Die Unterscheidung hat eine praktische Folge: Wer den Algorithmus ändert (etwa seine Schwelle ≥ 2 oder die Rotationsrichtung), muss alle Wiederverwendungen der Formen Portierung, Verallgemeinerung und Neuimplementierung nachziehen und gegen das Original prüfen. Bibliotheks- und Prozessnutzung erben die Änderung dagegen automatisch, Infrastruktur und Begriffliche gar nicht.

Teil II

Der Kern

Kapitel 4

Die Python-Referenz

Der Subgraph Algorithmus entstand als Python-Klasse `Subgraph` in `github.com/naphez86/subgraph`. Sie ist die Referenz, an der alle späteren Wiederverwendungen gemessen werden: Weicht eine Portierung von ihr ab, ist die Portierung zu prüfen, nicht die Referenz (mit einer Ausnahme, Abschnitt 5.3). Dieses Kapitel beschreibt die Referenz so genau, dass die folgenden Kapitel auf ihr aufbauen können: ihren Aufbau, den Rechenweg an einem durchgerechneten Beispiel, die formalen Eigenschaften der berechneten Relation, ihre Rolle in der Wiederverwendungskette und ihre Grenzen.

Die Beschreibung folgt der Referenz und ihrer wörtlichen Portierung nach C++ (Abschnitt 5.2). Alle Rechnungen und Beispiele dieses Kapitels wurden mit der C++-Bibliothek nachgeprüft; der Pseudocode in Listing 4.1 gibt den Ablauf sinngemäß wieder und ist kein Zitat des Python-Quelltexts.

4.1 Aufbau

Die Datei `src/subgraph.py` (326 Zeilen) enthält eine Klasse mit folgenden Methoden.

Methode	Aufgabe
`_compute_column_signature`	Spaltensignatur $\sigma_j = \sum_i A_{ij} 2^i + j\,2^n$ mit Python-Ganzzahlen beliebiger Größe
`_find_signature_rotation_match`	prüft alle n_B Rotationen von $\rho(B)$ gegen $\rho(A)$
`_compare_signature_sequences`	Tabelle für den längsten gemeinsamen *zusammenhängenden* Teilabschnitt; Ergebnis *wahr*, wenn Länge ≥ 2
`compare_graphs`	Entscheidung `keep_A`, `keep_B`, `keep_both`, `equal_keep_A`, `equal_keep_B`
`compare_graphs_with_adj_list`	Variante für dichte Graphen: Kantenmengeninklusion *ohne* Rotation (andere Relation)
`analyze_complexity`	textuelle Aufwandsanalyse, $O(n^3)$

Tabelle 4.1: Aufbau der Python-Referenz.

Die Methoden bilden eine Pipeline (Abbildung 4.1): Zwei Adjazenzmatrizen werden zu Signaturen, die Signaturen zu Zeilenkomponenten, die Zeilenkomponenten zu einem Wahrheitswert je Richtung und die beiden Wahrheitswerte zu einer Entscheidung. Nur

die beiden mittleren Stufen (Rotation und Tabelle) kosten nennenswert Rechenzeit; die erste und die letzte Stufe sind linear in der Eingabegröße.



Abbildung 4.1: Pipeline der Referenz. Unter den Pfeilen steht der Aufwand der jeweiligen Stufe für gleich große Graphen mit n Knoten; die letzte Stufe zählt gegebenenfalls die Kanten.

Die Entscheidung in `compare_graphs` folgt dem Schema: Gilt $A \preceq B$ und $B \preceq A$ (nur bei gleicher Größe möglich), entscheidet die Kantenzahl über `equal_keep_A` oder `equal_keep_B`; gilt nur $A \preceq B$, bleibt B; gilt nur $B \preceq A$, bleibt A; sonst bleiben beide. Die Python-Fassung kennt *keinen* Ergebniswert für identische Matrizen: Identische Graphen ($n \geq 2$) erscheinen als `equal_keep_A`.

4.2 Der Rechenweg im Einzelnen

Signatur und Zeilenkomponente. Die Signatur σ_j (Definition in Abschnitt 2.3) fasst zwei Informationen in einer Zahl zusammen: das Bitmuster der Spalte j in den unteren n Bit und die Spaltenposition j darüber. Das Positionsglied macht die Signaturen aller Spalten eines Graphen paarweise verschieden; für den Vergleich wird es sofort wieder entfernt, denn zwei Graphen sollen auch dann verglichen werden können, wenn sich dieselbe Spalte in beiden an verschiedenen Positionen befindet. Übrig bleibt die Zeilenkomponente $\rho_j(A)$, die den Spalteninhalt als Zahl kodiert. Bei Python-Ganzzahlen beliebiger Länge gibt es dabei keine Größenbeschränkung; die Referenz hat deshalb, anders als die C++-Fassung, keine Knotengrenze.

Rotation. Die Folge $\rho(B)$ wird nacheinander um $r = 0, 1, \ldots, n_B - 1$ Positionen nach rechts rotiert. Eine Rotation entspricht dem zyklischen Verschieben des Anfangspunkts: Der Algorithmus verlangt nicht, dass das kleinere Muster am Anfang der größeren Folge beginnt, sondern an irgendeiner Stelle des Kreises.

Tabelle. Für zwei Folgen a (Länge p) und b (Länge q) wird eine Tabelle $T \in \mathbb{N}^{(p+1)\times(q+1)}$ ausgefüllt. Bei gleichen Folgengliedern $a_{i-1} = b_{j-1}$ ist $T_{ij} = T_{i-1,j-1} + 1$, sonst $T_{ij} = 0$. Das Maximum der Tabelle ist die Länge der längsten gemeinsamen *zusammenhängenden* Teilfolge. Der Rücksetzwert 0 unterscheidet diese Tabelle von der klassischen Tabelle für die längste gemeinsame Teilfolge; deshalb ist der Methodenname `computeLCS` der C++-Fassung irreführend (Befund B13, Kapitel 15).

Schwelle. Eine Rotation gilt als Treffer, wenn das Tabellenmaximum mindestens 2 beträgt. Die Schwelle 2 ist ein Merkmal des Verfahrens; ihre Begründung liegt in [1] und nicht in dieser Arbeit. Satz 2.2 zeigt, was sie bewirkt: Für die Entscheidung genügt ein einziges Paar benachbarter Zeilenkomponenten, das in beiden Folgen vorkommt.

Der gesamte Ablauf lässt sich in wenigen Zeilen Pseudocode fassen. Er ist hier sinngemäß aus Beschreibung und Portierung wiedergegeben und nicht aus dem Python-Quelltext zitiert.

Listing 4.1: Ablauf von `compare_graphs` (Pseudocode, sinngemäß)

```
def enthalten(rhoA, rhoB): # A <= B ?
    if len(rhoB) < len(rhoA): return False
    for r in range(len(rhoB)):
        rotiert = rotiere_rechts(rhoB, r)
        if laengster_gemeinsamer_abschnitt(rhoA, rotiert) >= 2:
            return True
    return False

def compare_graphs(A, B):
    rhoA, rhoB = zeilenkomponenten(A), zeilenkomponenten(B)
    a_in_b, b_in_a = enthalten(rhoA, rhoB), enthalten(rhoB, rhoA)
    if a_in_b and b_in_a: return gleich_gross_nach_kanten(A, B)
    if a_in_b: return "keep_B"
    if b_in_a: return "keep_A"
    return "keep_both"
```

4.3 Ein durchgerechnetes Beispiel

Das folgende Beispiel zeigt jede Stufe der Pipeline. A sei die gerichtete Kette $0 \to 1 \to 2 \to 3$ und B die Kette $0 \to 1 \to 2$, also

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

In Spalte j von A steht genau dann eine Eins in Zeile i, wenn die Kante $i \to j$ existiert. Spalte 0 ist leer, Spalte 1 enthält die Eins in Zeile 0, Spalte 2 in Zeile 1 und Spalte 3 in Zeile 2. Daraus folgen die Zeilenkomponenten $\rho(A) = (0, 1, 2, 4)$ und mit dem Positionsglied $j \cdot 2^4$ die Signaturen $\sigma(A) = (0, 17, 34, 52)$. Für B ($n = 3$) ergibt sich analog $\rho(B) = (0, 1, 2)$ und $\sigma(B) = (0, 9, 18)$.

Wegen $n_B < n_A$ ist nur die Richtung $B \preceq A$ möglich; die Richtung $A \preceq B$ entfällt, weil A länger ist als B. Tabelle 4.2 zeigt die vier Rotationen von $\rho(A)$ und die jeweils längste gemeinsame zusammenhängende Teilfolge mit $\rho(B) = (0, 1, 2)$.

Es gilt also $B \preceq A$ und nicht $A \preceq B$; die Entscheidung lautet `keep_A` (C++: `KEEP_A`, Code 0). Das stimmt mit der anschaulichen Erwartung überein: Die Kette der Länge drei ist eine Teilstruktur der Kette der Länge vier. Die Paarcharakterisierung führt zum selben Ergebnis, ohne Rotation und Tabelle: $P_{\text{lin}}(\rho(B)) = \{(0,1), (1,2)\}$ und $P_{\text{cyc}}(\rho(A)) = \{(0,1), (1,2), (2,4), (4,0)\}$ haben die Paare $(0,1)$ und $(1,2)$ gemeinsam. Beide Rechnungen wurden mit der C++-Bibliothek nachvollzogen (`calculateSignatures`, `compareGraphs`); sie liefert dieselben Signaturen und `KEEP_A`. Das Beispiel ist zugleich das Beispiel der Dokumentation von `csubgraph`.

r	$\mathrm{rot}_r(\rho(A))$	Längste gemeinsame Teilfolge	Länge
0	$(0,1,2,4)$	$(0,1,2)$	$3 \geq 2$, Treffer
1	$(4,0,1,2)$	$(0,1,2)$	$3 \geq 2$, Treffer
2	$(2,4,0,1)$	$(0,1)$	$2 \geq 2$, Treffer
3	$(1,2,4,0)$	$(1,2)$	$2 \geq 2$, Treffer

Tabelle 4.2: Rotationen im Beispiel $B \preceq A$. Schon $r = 0$ genügt; die weiteren Zeilen zeigen, dass hier jede Rotation einen Treffer liefert.

4.4 Eigenschaften der berechneten Relation

Die Relation $\preceq$ wird in den späteren Kapiteln als Black Box benutzt. Dieser Abschnitt hält fest, was man von ihr erwarten darf und was nicht. Die Aussagen gelten für die Referenz und, wegen der wörtlichen Portierung, ebenso für alle Fassungen, die dieselbe Relation berechnen (Satz 2.2).

Lemma 4.1 (Reflexivität erst ab zwei Knoten)**.** Für jede Adjazenzmatrix A mit n Knoten gilt $A \preceq A$ genau dann, wenn $n \geq 2$.

Beweis. Sei $n \geq 2$ und $a = \rho(A)$. Das Paar (a_0, a_1) liegt in $P_{\mathrm{lin}}(a)$ und, weil $P_{\mathrm{lin}} \subseteq P_{\mathrm{cyc}}$ für $m \geq 2$ gilt, auch in $P_{\mathrm{cyc}}(a)$. Nach Satz 2.2 folgt $A \preceq A$. Für $n = 1$ ist $P_{\mathrm{lin}}(a) = \emptyset$, der Schnitt also leer, und $A \not\preceq A$. □

Das Lemma erklärt die einzige Abweichung zwischen Python und C++ (Abschnitt 5.3): Zwei identische 1×1-Graphen stehen in keiner Richtung in Relation, die Referenz meldet `keep_both`. Die C++-Fassung fängt den Fall mit einer vorgeschalteten Gleichheitsprüfung ab. Aus demselben Grund bleibt `IDENTICAL` für $n \geq 2$ ohne Folge: Dort erkennt die Relation die Gleichheit ohnehin.

Beispiel 4.1 (Die Relation hängt von der Knotennummerierung ab)**.** Der ungerichtete Pfad mit drei Knoten hat zwei verschiedene Beschriftungen, je nachdem, ob der mittlere Knoten die Nummer 1 oder die Nummer 0 trägt. Die Zeilenkomponenten sind $(2,5,2)$ beziehungsweise $(6,1,1)$. Die Paarmengen $P_{\mathrm{lin}}(2,5,2) = \{(2,5),(5,2)\}$ und $P_{\mathrm{cyc}}(6,1,1) = \{(6,1),(1,1),(1,6)\}$ sind disjunkt; ebenso in der Gegenrichtung. Der Algorithmus meldet `KEEP_BOTH`, obwohl beide Graphen bis auf die Nummerierung gleich sind.

Das ist kein Fehler im Sinn der Spezifikation, aber eine wichtige Eigenschaft: Die Relation $\preceq$ vergleicht *beschriftete* Graphen, nicht Isomorphieklassen. Eine vollständige Aufzählung der ungerichteten Graphen auf vier Knoten (Kapitel 14) zeigt das Ausmaß: Es gibt 492 geordnete Paare verschiedener Beschriftungen isomorpher Graphen; bei 434 davon, also bei 88 %, meldet der Algorithmus `KEEP_BOTH`. Wer die Relation auf Graphen anwendet, deren Knotenreihenfolge zufällig ist, erhält deshalb viele „falsch negative“ Antworten im Sinn des klassischen Teilgraphproblems; Beispiel 2.1 zeigte bereits eine andere Quelle dafür.

Beispiel 4.2 (Die Relation ist nicht transitiv)**.** Seien $A = K_2$ (Zeilenkomponenten $(2,1)$), B der ungerichtete Graph auf drei Knoten mit der einzigen Kante $\{0,1\}$ (Zeilenkomponenten $(2,1,0)$) und C der ungerichtete Graph auf vier Knoten mit der einzigen Kante $\{0,2\}$ (Zeilenkomponenten $(4,0,1,0)$). Dann gilt:

– $A \preceq B$, denn $(2,1) \in P_{\text{lin}}(2,1) \cap P_{\text{cyc}}(2,1,0)$;

– $B \preceq C$, denn $(1,0) \in P_{\text{lin}}(2,1,0) \cap P_{\text{cyc}}(4,0,1,0)$;

– $A \not\preceq C$, denn $(2,1) \notin P_{\text{cyc}}(4,0,1,0) = \{(4,0),(0,1),(1,0),(0,4)\}$.

Die C++-Bibliothek bestätigt es: `compareGraphs` liefert für (A,B) und (B,C) jeweils `KEEP_B`, für (A,C) aber `KEEP_BOTH`.

Klassisch ist K_2 sogar in C enthalten. Der Fall zeigt noch einmal, dass $\preceq$ eine eigene Relation ist und keine Näherung des Teilgraphtests. Für Anwendungen, die „behalte die umfassendere Struktur“ wörtlich nehmen und Strukturen aussortieren, hat die fehlende Transitivität eine praktische Folge: Das Ergebnis hängt von der Reihenfolge der Vergleiche ab. Verarbeitet man im Beispiel zuerst (A,B) und dann (B,C), bleibt nur C übrig; verarbeitet man zuerst (B,C) und dann (A,C), bleiben A und C übrig, denn für (A,C) lautet das Ergebnis `KEEP_BOTH`. Wo die Relation dagegen nur einzeln ausgewertet wird, etwa bei einer Anfrage gegen alle gespeicherten Stapel (Abschnitt 9.5), hat die fehlende Transitivität keine Folge, denn dort wird jedes Paar unabhängig von den anderen entschieden.

Zwei weitere Eigenschaften folgen unmittelbar aus den Definitionen und dem Quelltext.

– *Größenordnung:* $A \preceq B$ verlangt $n_A \le n_B$. Wechselseitige Enthaltung ($A \preceq B$ und $B \preceq A$) setzt deshalb $n_A = n_B$ voraus; nur dann kann `equal_keep_*` eintreten.

– *Spiegelung der Entscheidung:* Die beiden Richtungen $A \preceq B$ und $B \preceq A$ werden mit demselben Verfahren geprüft. Vertauscht man A und B, vertauschen sich daher `keep_A` und `keep_B`. Bei `equal_keep_*` entscheidet die Kantenzahl, und bei Gleichstand bleibt jeweils das *erste* Argument (im Quelltext von `compareGraphs`: `edgesA >= edgesB`).

4.5 Laufzeit und Speicher der Referenz

Die Referenz prüft je Richtung höchstens n_B Rotationen, jede mit einer Tabelle der Größe $(n_A+1)(n_B+1)$. Für gleich große Graphen sind das $O(n^3)$ Schritte und $O(n^2)$ Speicher für die Tabelle. Zählt man Operationen auf Maschinenwörtern statt auf Zahlen, kommt bei Python-Ganzzahlen ein weiterer Faktor hinzu: Ein Vergleich zweier Zeilenkomponenten kostet bis zu $\lceil n/64 \rceil$ Wortvergleiche. Das ist eine Rechnung aus der Zahldarstellung und keine Messung; für die Größen, die in Gen-DB vorkommen, spielt sie keine Rolle.

Die Messungen in Gen-DB (Abschnitt 9.9) zeigen, was der Preis der Referenz in der Praxis ist: Ein einzelner Vergleich dauert im Mikro-Benchmark im Mittel 2,69 ms in Python und 0,074 ms in C++. Bei der Suche über alle Kandidaten verteilt sich die Zeit anders: 1,27 ms je Kandidat gegenüber 0,20 ms. Dass der Gewinn im Gesamtlauf kleiner ausfällt als im Einzelvergleich, hat mit der Kopplung zu tun und nicht mit dem Algorithmus (Kapitel 12).

4.6 Die Rolle der Referenz in der Kette

Die Referenz ist in der Wiederverwendungskette an fünf Stellen beteiligt. Tabelle 4.3 ordnet sie, und die letzte Spalte sagt, was passiert, wenn man die Referenz ändert.

Nr.	Rolle der Referenz	Art der Abhängigkeit	Folge einer Änderung der Referenz
W1	Vorlage der Portierung	Quelltext wurde übersetzt	manuell nachziehen, Differenzialtest wiederholen
W6	Fallback in GenDB	Paket, Version `@main`	wirkt *sofort und ungeprüft* (Befund B4)
W9, **W10**	Spezifikation der Relation	kein gemeinsamer Quelltext	manuell nachziehen
W13	Vergleichsorakel der Differenzialtests	wird als Prüfling aufgerufen	Tests sind dann gegen eine *neue* Relation gerichtet
W18	Gegenstand der Theorie	Satz 2.2 bezieht sich auf $\preceq$	Beweis prüfen, Sätze neu aussprechen

Tabelle 4.3: Beteiligung der Python-Referenz an der Wiederverwendungskette.

Die Tabelle zeigt zwei Eigenheiten. Erstens ist die Referenz zugleich Ursprung und Prüfmaßstab: Sie ist die Vorlage von **W1**, **W9** und **W10** und der Maßstab, an dem dieselben Fassungen in den Differenzialtests gemessen werden. Ändert man sie, verschiebt sich beides gleichzeitig, und ein Differenzialtest kann nichts mehr melden, solange die Fassungen nicht gleichzeitig mitgezogen werden. Zweitens ist **W6** die einzige Stelle, an der eine Änderung der Referenz ohne Zutun wirksam wird; deshalb steht die Pinnung auf eine feste Version (B4) unter den Verbesserungen mit mittlerer Priorität.

4.7 Grenzen der Referenz

Die Referenz beantwortet die in [1] definierte Frage. Vier Eigenschaften sollte man kennen, bevor man sie benutzt.

1. Sie entscheidet *nicht* das klassische Teilgraphproblem (Beispiel 2.1, Beispiel 4.2). Wo ein exakter Test gebraucht wird, ist `ExactSubgraph` zu verwenden (Abschnitt 6.2).
2. Sie ist von der Knotenreihenfolge abhängig (Beispiel 4.1).
3. Sie verhält sich bei identischen 1×1-Graphen anders als die Portierung (Lemma 4.1, Befund B7).
4. Sie ist die langsamste der Fassungen: Der Preis der Python-Ganzzahlen und der Interpretation ist der Faktor ≈ 37 im Mikro-Benchmark.

Die Methode `compare_graphs_with_adj_list` gehört nicht zur untersuchten Relation: Sie vergleicht Kantenmengen und benutzt keine Rotation (Tabelle 4.1). Sie kommt in keiner der untersuchten Wiederverwendungen vor.

Kapitel 5

Die C++-Portierung: `csubgraph`

`csubgraph` ist die erste Wiederverwendung (**W1**) und zugleich der Kern aller weiteren. Es enthält drei Module (Abbildung 5.1); dieses Kapitel behandelt das erste.



Abbildung 5.1: Die drei Module von `csubgraph`. `MultiOmics` ruft Funktionen von `SubgraphAlgorithm` direkt auf (`computeLCS`, `rotateSequence`, `calculateSignatures`, `extractRowComponents`); `ExactSubgraph` nutzt vom Signaturverfahren nichts.

5.1 Bauziele

Ziel	Art	Verwendung
`subgraphlib`	statische Bibliothek	Kern; vom Adapter und (über C-Wrapper) von Gen-DB benutzt
`subgraph-cli`	Programm, JSON über stdin/stdout	Einzelvergleich mit `SubgraphAlgorithm`; benötigt `nlohmann/json` [9]
`subgraph`	Programm, Dateiprotokoll	Adapter für den SAT-Solver (`SolverCli.cpp`)
`subgraph-tests`	Programm	GoogleTest-Suite [8]

Tabelle 5.1: Bauziele von `csubgraph` (aus `CMakeLists.txt`).

Unter GCC und Clang werden Bibliothek und Tests mit `--coverage` übersetzt; Anwendungen, die die Bibliothek in eine gemeinsame Bibliothek linken, müssen deshalb `-lgcov` hinzufügen (Abschnitt 9.2). Die CMake-Datei setzt *nicht* `POSITION_INDEPENDENT_CODE`; das verhindert unter Linux das Einbinden in eine gemeinsame Bibliothek (Kapitel 15).

5.2 Das Modul `SubgraphAlgorithm`

Listing 5.1: Öffentliche Schnittstelle von `SubgraphAlgorithm` (gekürzt)

```cpp
class SubgraphAlgorithm {
public:
    enum class Result { KEEP_A, KEEP_B, KEEP_BOTH, IDENTICAL, EQUAL_KEEP_A,
        ↪ EQUAL_KEEP_B };

    static std::vector<uint64_t> calculateSignatures(const std::vector<std::vector<
        ↪ int>>& m);
    static std::vector<uint64_t> extractRowComponents(const std::vector<uint64_t>&
        ↪ sigs, size_t n);
    static std::vector<uint64_t> rotateSequence(const std::vector<uint64_t>& seq,
        ↪ size_t rotation);
    static size_t computeLCS(const std::vector<uint64_t>& a, const std::vector<
        ↪ uint64_t>& b);
    static Result compareGraphs(const std::vector<std::vector<int>>& a,
                                const std::vector<std::vector<int>>& b); // wirft std::
                                    ↪ invalid_argument
    static bool isValidAdjacencyMatrix(const std::vector<std::vector<int>>& m);
    static std::string resultToString(Result r);
};
```

Die Portierung ist wörtlich: `calculateSignatures` berechnet σ_j in `uint64_t`, `extractRowComponents` maskiert mit $2^n - 1$, `rotateSequence` rotiert nach rechts, `computeLCS` ist dieselbe Tabelle wie in Python (bei Unterschied Rücksetzen auf 0) und `compareGraphs` dieselbe Entscheidung. Eine Änderung gibt es: C++ liefert zusätzlich `IDENTICAL`, wenn beide Matrizen gleiche Größe und gleiche Zeilenkomponenten haben; das wird vor den Rotationen geprüft.

Code	Name	Bedeutung
0	`KEEP_A`	$B \preceq A$: B ist in A enthalten, behalte A
1	`KEEP_B`	$A \preceq B$: A ist in B enthalten, behalte B
2	`KEEP_BOTH`	keine Beziehung
3	`IDENTICAL`	identische Matrizen (nur C++)
4	`EQUAL_KEEP_A`	wechselseitig enthalten, behalte A (bei Kantengleichstand)
5	`EQUAL_KEEP_B`	wechselseitig enthalten, behalte B

Tabelle 5.2: Ergebniswerte von `compareGraphs` und ihre Codes.

Laufzeit und Grenze. Je Richtung werden höchstens n_B Rotationen mit je einer $O(n_A n_B)$-Tabelle geprüft; für gleiche Größen ergibt das $O(n^3)$. Eine Knotengrenze prüft `SubgraphAlgorithm` *nicht* selbst: Die Zeilenkomponente braucht n Bit, und die Maske $(1 \ll n) - 1$ ist nur für $n \leq 63$ definiert. Die Grenze wird erst in `MultiOmics::isValidLayerStack` und im Adapter durchgesetzt (Kapitel 15).

5.3 Abweichungen zwischen Python und C++

Ein früherer Differenzialtest verglich beide Fassungen an 20 000 Zufallspaaren (Kapitel 14): 127 Abweichungen, *alle* bei zwei identischen 1×1-Graphen. Der Quelltext erklärt

sie vollständig. Python braucht für `keep_*` eine gemeinsame Teilfolge der Länge ≥ 2; bei $n = 1$ ist das unmöglich, also `keep_both`. C++ prüft vorher die Gleichheit der Zeilenkomponenten und liefert `IDENTICAL`. Die C++-Antwort ist hier die fachlich richtige; die Referenz ist an dieser Stelle fehlerhaft. Gen-DB übersetzt `IDENTICAL` deshalb in `equal_keep_A` oder `equal_keep_B` nach Kantenzahl und liefert damit für Graphen mit $n \geq 2$ dasselbe wie die Referenz.

Kapitel 6

Erweiterungen: MultiOmics und ExactSubgraph

6.1 Das Modul `MultiOmics`

`MultiOmics` (**W2**) verallgemeinert die Relation auf Schichtstapel. Es kennt zwei Modi und zwei Strategien.

Listing 6.1: Schnittstelle von `MultiOmics` (gekürzt)

```
class MultiOmics {
public:
    using Matrix = std::vector<std::vector<int>>; using LayerStack = std::vector<
        ↪ Matrix>;
    static constexpr std::size_t kMaxNodes = 63;
    enum class Mode { COHERENT, INDEPENDENT };
    enum class Strategy { BIGRAM, DYNAMIC };
    enum class IntegrationOp { UNION, INTERSECTION, CONSENSUS };

    static bool isValidLayerStack(const LayerStack& stack); // n <= 63, quadratisch
        ↪ , 0/1
    static Matrix integrateConsensus(const LayerStack&, std::size_t k); // Union: k
        ↪ =1, Schnitt: k=L
    static bool contains(const LayerStack& a, const LayerStack& b, Mode, Strategy);
    static Result compareLayered(const LayerStack& a, const LayerStack& b, Mode,
        ↪ Strategy);
};
```

Im Modus `INDEPENDENT` muss jede Schicht für sich die Relation erfüllen; im Modus `COHERENT` werden die Zeilenkomponenten aller Schichten je Spalte zu einem Tupel zusammengefasst, das injektiv auf eine Zahl abgebildet wird, und die Relation wird auf der Folge dieser Zahlen geprüft. Die Strategie `DYNAMIC` verwendet die Rotations- und Tabellenfassung von `SubgraphAlgorithm` (ruft also `computeLCS` und `rotateSequence` auf), die Strategie `BIGRAM` die Paarcharakterisierung (Satz 2.2): Es genügt, dass ein benachbartes Paar von a als zyklisch benachbartes Paar in b vorkommt. Die Strategien liefern nach Satz 2.2 dieselben Ergebnisse. Bei $L = 1$ stimmt `compareLayered` mit `compareGraphs` überein, abgesehen von der Gleichheitsprüfung, die dasselbe `IDENTICAL` liefert.

6.2 Das Modul `ExactSubgraph`

`ExactSubgraph` (**W3**) entscheidet $H \sqsubseteq_{\text{mono}} G$ und liefert die Einbettung. Es gehört nicht zum Signaturverfahren: Es wurde für die Wiederverwendung im SAT-Solver ergänzt, weil die Reduktion ein korrektes und vollständiges Orakel braucht (Definition 2.9). Von `csubgraph` wird nur die Bibliotheksform wiederverwendet, nicht der Algorithmus.

Listing 6.2: Schnittstelle von `ExactSubgraph` (gekürzt)

```
class BitGraph { // ungerichtet, einfach, Schleifen ignoriert
public: explicit BitGraph(size_t n = 0); void addEdge(size_t u, size_t v);
        bool hasEdge(size_t u, size_t v) const; // Speicher: n * ceil(n/64) * 8
            ↪ Byte
};
class ExactSubgraph {
public:
    class Timeout : public std::runtime_error { /* ... */ };
    // true, wenn pattern in host enthalten ist; bei Erfolg optional mapping:
        ↪ Knoten von pattern -> Knoten von host
    static bool contains(const BitGraph& host, const BitGraph& pattern,
                         std::vector<size_t>* mapping = nullptr, double timeoutSec =
                             ↪ 0.0);
};
```

Ist das Muster vollständig ($|E_H| = k(k-1)/2$), wird eine k-Clique gesucht (Klasse `CliqueSearch`), sonst läuft eine allgemeine Rückverfolgungssuche (`GeneralSearch`). Für die Reduktion ist nur der erste Fall relevant.

6.2.1 Die Cliquensuche

Die Suche kombiniert einen Gradfilter (k-Kern), eine einmalig berechnete gierige Färbung als Schranke und eine Verzweigung auf die kleinste Farbklasse.

Lemma 6.1 (k-Kern)**.** Ein Knoten v mit weniger als $k-1$ Nachbarn in der Kandidatenmenge C liegt in keiner k-Clique in C. Das iterierte Entfernen solcher Knoten verändert die Menge der k-Cliquen in C nicht.

Beweis. Liegt v in einer k-Clique $Q \subseteq C$, sind die $k-1$ übrigen Knoten von Q Nachbarn von v in C. Hat v weniger Nachbarn, ist das unmöglich. Da keine k-Clique den entfernten Knoten enthält, bleibt jede k-Clique erhalten; die Gradbedingung wird für die kleinere Menge erneut geprüft, und die Argumentation wiederholt sich. □

Lemma 6.2 (Färbungsschranke)**.** Die gierige Färbung teilt die Knoten in Klassen, die jeweils unabhängige Mengen sind. Eine Clique enthält höchstens einen Knoten je Klasse; eine r-Clique in C benötigt daher mindestens r Klassen mit nichtleerem Schnitt mit C.

Beweis. Der Quelltext fügt v nur in eine Klasse ein, in der v keinen Nachbarn hat; jede Klasse ist also unabhängig. Zwei Knoten einer Clique sind benachbart und liegen deshalb in verschiedenen Klassen. □

Satz 6.1 (Korrektheit und Vollständigkeit der Cliquensuche)**.** `CliqueSearch::run` liefert genau dann `true`, wenn G eine k-Clique enthält, und die gelieferte Knotenfolge ist dann eine k-Clique.

Beweis. Korrektheit. Die Rekursion `solve(cand, need)` wählt $v \in$ `cand` und ruft sich mit `cand`$\cap N(v)$ und `need`-1 auf. Per Induktion ist jeder Knoten von `path_` mit allen früheren benachbart. Bei `need` $= 0$ hat `path_` also k paarweise benachbarte Knoten.

Vollständigkeit. Nach Lemma 6.1 geht durch den Gradfilter keine k-Clique verloren. In `solve` wird die Klasse c mit den wenigsten Kandidaten gewählt. Eine `need`-Clique in `cand` enthält nach Lemma 6.2 entweder genau einen Knoten aus c oder keinen. Im ersten Fall wird sie beim Probieren der Kandidaten von c gefunden, im zweiten nach dem Entfernen von c in der nächsten Schleifenrunde. Die Abbruchbedingung `nonEmpty` $<$ `need` ist nach Lemma 6.2 zulässig; bei `nonEmpty` $=$ `need` muss jede nichtleere Klasse genau einen Knoten beitragen, insbesondere c, und alle Kandidaten von c sind bereits erfolglos probiert. Die Rekursion terminiert, weil `need` oder die Kandidatenmenge schrumpft. □

Bemerkung 6.1 (Zeitlimit)**.** Die Klasse `Deadline` prüft die Uhr nur bei jedem 1024. Aufruf von `tick()`, und `tick()` wird nur in der Suche selbst aufgerufen, nicht im k-Kern und nicht in der Färbung. Bei sehr großen Graphen kann die Vorverarbeitung das Limit überschreiten. Überschreitet die Suche es, wirft sie `ExactSubgraph::Timeout`.

6.2.2 Die allgemeine Suche

Für nicht vollständige Muster ordnet `GeneralSearch` die Musterknoten gierig (zuerst der Knoten mit den meisten bereits geordneten Nachbarn, dann der mit dem höchsten Grad), schneidet beim Fortsetzen die Nachbarschaftsmengen der bereits abgebildeten Nachbarn bitweise und verwirft Kandidaten mit kleinerem Grad als der Musterknoten. Auch sie ist im ungünstigsten Fall exponentiell und wird in den Kopplungen dieser Arbeit nicht gebraucht.

6.3 Die mengenbasierte Verallgemeinerung

Die Paarcharakterisierung benutzt nur die Gleichheit von Folgengliedern. Gen-DB nutzt das in `universal_core` (**W10**) aus: Eine *Schicht* ist eine Folge endlicher Mengen $s_j \subset \mathbb{N}$, ein *Stapel* besteht aus L Schichten gleicher Länge, und das Enthaltensein ist

$$M \sqsubseteq_{\mathrm{ind}} M' \iff |M| \le |M'| \wedge \forall \ell : \; P_{\mathrm{lin}}(s^{(\ell)}) \cap P_{\mathrm{cyc}}(s'^{(\ell)}) \neq \emptyset,$$

bzw. im kohärenten Modus mit den Tupelfolgen aller aktiven Schichten. Ein Matrixstapel ist der Spezialfall $s_j^{(\ell)} = \{i : A_{ij}^{(\ell)} = 1\}$ in lokalen Koordinaten; weil Mengen keine 64-Bit-Zahlen brauchen, entfällt die Knotengrenze. Die Ergebnisnamen sind die der C++-Bibliothek, sodass beide gegeneinander geprüft werden können.

Kapitel 7

Außenflächen des Kerns

Der Kern hat vier Außenflächen. Jede ist eine eigene Wiederverwendungsgrenze mit eigenem Vertrag.

Fläche	Art	Vertrag	Nutzer
C++-API	Bibliotheksaufruf	Klassen, Ausnahmen	Adapter, Wrapper, Tests
JSON-Programm	stdin/stdout	`graph_a`, `graph_b` → `result`, `result_code`	frühe Gen-DB-Läufe
Datei-Adapter	Dateipaar, Exit-Code	`G`, `H` → `is_subgraph`, `mapping`	Solver
C-ABI	`extern "C"`	`int`-Felder, Fehlerpuffer, Versionsfunktion	Gen-DB per `ctypes`

Tabelle 7.1: Die vier Außenflächen von `csubgraph` und ihre Nutzer.

7.1 Das JSON-Programm `subgraph-cli` (W4)

Listing 7.1: Protokoll von `subgraph-cli`: Eingabe und Ausgabe

```
{ "graph_a": [[0,1,0],[0,0,1],[0,0,0]], "graph_b": [[0,1],[0,0]] }

{ "result": "KEEP_A", "result_code": 0, "error": null }
```

Das Programm ruft ausschließlich `SubgraphAlgorithm::compareGraphs` auf. Es war der Aufrufweg der frühen Gen-DB-Läufe und wurde von der Bibliotheksanbindung abgelöst, weil Prozessstart und JSON-Kodierung je Vergleich die Kosten des Rechenkerns überdeckten (Abschnitt 9.9).

7.2 Der Datei-Adapter `SolverCli` (W5)

Der Adapter `subgraph` (`SolverCli.cpp`) spricht das Dateiprotokoll des SAT-Solvers. Er liest die Eingabe in einem Durchgang ohne DOM-Struktur, weil die Reduktionsgraphen viele Millionen Kanten haben können, wählt das Verfahren über Umgebungsvariablen und schreibt das Ergebnis im vom Solver erwarteten Format.

Umgebungsvariable	Wirkung
`SUBGRAPH_MODE`	`exact` (Standard): `ExactSubgraph`, keine Knotengrenze. `heuristic`: `compareGraphs`, höchstens 63 Knoten.
`SUBGRAPH_TIMEOUT_SEC`	Zeitlimit für den exakten Modus; Standard 60, 0 bedeutet kein Limit.

Tabelle 7.2: Konfiguration des Adapters.

Exit-Code	Bedeutung
0	Erfolg; die Ausgabedatei enthält `is_subgraph` (bei `true` im exakten Modus auch `mapping`)
1	Fehler: Datei nicht lesbar, JSON ungültig, ungültige Graphen, ungültiger `SUBGRAPH_MODE`
2	Modus `heuristic` mit mehr als 63 Knoten
3	Zeitlimit überschritten

Tabelle 7.3: Exit-Codes des Adapters.

Im Signaturmodus wird $A = G$, $B = H$ gesetzt; H in G gilt, wenn das Ergebnis `KEEP_A`, `IDENTICAL`, `EQUAL_KEEP_A` oder `EQUAL_KEEP_B` ist. Die Ausgabe hat die feste Form `{"is_subgraph": true, "mapping": [...]}` mit einem Leerzeichen nach dem Doppelpunkt; der Solver sucht diese Zeichenfolge wörtlich (Abschnitt 10.2). Das Ausgabeformat ist damit Teil des Vertrags.

7.3 Die C-ABI für Gen-DB (W7, W8)

Gen-DB besitzt zwei C-Wrapper (`csubgraph_shim.cpp`, `csubgraph_omics_shim.cpp`), die die Bibliothek aufrufen und `extern "C"` exportieren. Sie gehören zum Repository von Gen-DB, nicht zu `csubgraph`; der Kern kennt sie nicht. Abschnitt 9.2 beschreibt sie.

7.4 Testsuite

Die Dokumentation von `csubgraph` nennt 137 Testfälle in 22 Gruppen für `SubgraphAlgorithm` und `MultiOmics` und sechs Tests für `ExactSubgraph`, darunter einen Zufallsvergleich mit einer Brute-Force-Suche mit festen Seeds. Die Zahlen sind der Dokumentation entnommen; die Suite selbst wurde hier nicht ausgeführt (kein GoogleTest in der Prüfumgebung). Stattdessen laufen eigene Differenzialtests (Kapitel 14).

Teil III

Wiederverwendungen

Kapitel 8

Der Katalog der Wiederverwendungen

Dieses Kapitel führt *alle* Wiederverwendungen des Subgraph Algorithmus in den vier hochgeladenen Projekten auf, dazu die eine bekannte, aber nicht geprüfte Wiederverwendung im Experimente-Framework. Jede Wiederverwendung hat eine Nummer **W1** bis **W18**, eine Form nach Tabelle 3.2 und einen Verweis auf das Kapitel, das sie vertieft.

8.1 Übersicht

Tabelle 8.1: Katalog der Wiederverwendungen. *Wiederverwendet* nennt den Teil des Ursprungs, der tatsächlich benutzt wird.

Nr.	Wiederverwendung	Ort	Form	Wiederverwendet
Kern: `csubgraph`				
W1	C++-Neuimplementierung `SubgraphAlgorithm`	csubgraph	Portierung	gesamter Algorithmus
W2	`MultiOmics`: Schichtstapel, Modi, Strategien	csubgraph	Verallgemeinerung	`computeLCS`, `rotateSequence`, `calculateSignatures`, `extractRowComponents`
W3	`ExactSubgraph`: exakter Teilgraphtest	csubgraph	Infrastruktur	nur Bibliothek, Build, Adapter
W4	`subgraph-cli`: JSON über stdin/stdout	csubgraph	Prozessnutzung	`compareGraphs`
W5	Adapter `SolverCli`: Dateiprotokoll, Modi `exact`/`heuristic`	csubgraph	Prozessnutzung	`ExactSubgraph`, `compareGraphs`
Gen-DB				
W6	Python-Paket `subgraph` als Abhängigkeit mit Fallback	gen-db	Bibliotheksnutzung	Klasse `Subgraph`, nur über die API
W7	C++-Bibliothek über C-Wrapper und `ctypes`	gen-db	Bibliotheksnutzung	`compareGraphs`
W8	`MultiOmics` über zweiten Wrapper	gen-db	Bibliotheksnutzung	`compareLayered`

Nr.	Wiederverwendung	Ort	Form	Wiederverwendet
W9	`multiomics_python`: Neuimplementierung als Fallback	gen-db	Neuimplementierung	Relation, ohne Code
W10	`universal_core`: Relation auf Mengenfolgen	gen-db	Verallgemeinerung	Relation (Satz 2.2)
W11	`universal_index`: Paarindex ohne Einzelvergleich	gen-db	Begriffliche	Paarcharakterisierung
W12	Prozess-Pool `compare_many*`	gen-db	Infrastruktur	keine Relation, nur Aufruf
W13	Experimente und Differenzialtests	gen-db	Infrastruktur	alle Fassungen als Prüfling
Solver, Experimente, Theorie				
W14	Solver ruft `ExactSubgraph` als Orakel	subgraph-sat-solver	Prozessnutzung	exakter Test, Einbettung
W15	Solver ruft `SubgraphAlgorithm` als unvollständiges Orakel	subgraph-sat-solver	Prozessnutzung	Signaturrelation (Diagnose)
W16	Belegungsextraktion aus dem `mapping`	subgraph-sat-solver	Prozessnutzung	Einbettung als Ausgabe
W17	Experimente-Framework startet den Solver	Experimente	Prozessnutzung	nur Solver-Ausgabe
W18	Paarcharakterisierung als gemeinsame Theorie	Theorie	Begriffliche	**W2**, **W9**, **W10**, **W11**

8.2 Der Wiederverwendungsbaum

Abbildung 8.1 zeigt, wie die Wiederverwendungen aufeinander aufbauen. Wichtig ist die Unterscheidung zwischen Wiederverwendungen *des Algorithmus* (Wurzel: Python-Referenz) und *der Bibliothek* (Wurzel: `csubgraph`, unabhängig vom Algorithmus).

Die in Abbildung 8.1 dargestellte Architektur visualisiert die hierarchischen Abhängigkeiten und Verflechtungen aller im Katalog erfassten Wiederverwendungen (**W1** bis **W18**). Als zentraler Ausgangspunkt und methodische Wurzel dient die **Python-Referenz** (`subgraph`, rot hervorgehoben), welche die exakte Spezifikation der Relation in ausführbarer Form darstellt. Von diesem Ursprung aus verzweigt sich die Code- und Konzeptbasis in zwei Hauptrichtungen: einerseits in die direkte **C++-Portierung** (**W1**) sowie andererseits in die direkte Einbindung des **Python-Pakets als Fallback** in GenDB (**W6**).

Der C++-Zweig (**W1**) bildet seinerseits das Fundament für eine Vielzahl darauf aufsetzender Module und Wrapper:

- **Schnittstellen und Adapter:** Über die Zwischenstationen **W2** (`MultiOmics`), **W4** (Kommandozeilenschnittstelle) und **W5** (Prozess-Adapter) werden sowohl dichte Graphstrukturen als auch exakte Teilgraphtests angebunden.
- **C- und Omics-Wrapper:** Die Module **W7** (C-Wrapper), **W8** (Omics-Wrapper) und **W9** (Python-Omics-Neuimplementierung) leiten die Logik in Richtung theoretischer Fundierungen wie Mengenfolgen (**W10**) und den effizienten Paarindex (**W11**) weiter.

– **Solver-Integration und Experimente:** Über den exakten Test (**W3**) und den Adapter (**W5**) greifen die Solver-Komponenten (**W14**–**W16**) sowie heuristische Prüfstände (**W15**) auf die Algorithmen zu, deren Ergebnisse letztlich in die Experimente-Infrastruktur (**W17**) einfließen.

Globale Querschnittsaufgaben wie der parallele Prozess-Pool (**W12**), die Differenzialtests (**W13**) und die übergreifende Paarcharakterisierung als gemeinsame Theorie (**W18**) betreffen mehrere dieser Zweige parallel und sind im Baum als konzeptionelle Klammern zu verstehen.

8.3 Was wird tatsächlich wiederverwendet?

Der Titel dieser Arbeit könnte suggerieren, dass überall der Algorithmus benutzt wird. Das stimmt nicht. Tabelle 8.2 zeigt je Wiederverwendung, welche Bestandteile des Algorithmus tatsächlich zum Einsatz kommen.

8.4 Die Wiederverwendungen im Einzelnen

W1. Neuimplementierung in C++17; Abschnitt 5.2. Grund war die Laufzeit der Python-Fassung. Die Portierung ist wörtlich, bis auf den Ergebniswert `IDENTICAL` und die daraus folgende Abweichung bei 1×1-Graphen (Abschnitt 5.3).

W2. `MultiOmics` ruft `computeLCS`, `rotateSequence`, `calculateSignatures` und `extractRowComponents` direkt aus **W1** auf, ergänzt Schichtstapel, die Modi `COHERENT`/`INDEPENDENT` und die Strategie `BIGRAM`.

W3. Wiederverwendet werden nur Bibliothek, Build und Adapter. Der Algorithmus selbst wird *nicht* benutzt; das Modul ist ein zweiter, andersartiger Inhalt derselben Bibliothek.

W4, W5. Zwei Prozessschnittstellen für zwei Nutzer: JSON für Gen-DB (früher), Dateiprotokoll für den Solver. Beide sind dünne Hüllen um die Bibliothek.

W6. Gen-DB bindet das Python-Paket als Abhängigkeit ein und ruft ausschließlich die öffentliche API `Subgraph().compare_graphs(A, B)` auf; Abschnitt 9.1.

W7, W8. Die Bibliothek wird per C-Wrapper und `ctypes` im selben Prozess aufgerufen; Abschnitt 9.2.

W9. Python-Neuimplementierung der Mehrschicht-Relation, damit Gen-DB ohne C++ läuft; Abschnitt 9.3.

W10. Dieselbe Relation auf Mengenfolgen, ohne Knotengrenze; Abschnitt 6.3.

W11. Ein invertierter Paarindex, der die Relation nicht mehr für jedes Paar auswertet, sondern Trefferlisten liest; Abschnitt 9.5.

	Signaturen	Rotation + Tabelle	Relation $\preceq$	Ergebniswerte	Bibliothek	Paarsatz
W1 C++	ja	ja	ja	ja	–	–
W2 MultiOmics	ja	Strategie `DYNAMIC`	ja	ja	ja	Strategie `BIGRAM`
W3 Exact	–	–	–	–	ja	–
W6 Paket	ja	ja	ja	ja (Strings)	–	–
W7, **W8** Wrapper	indirekt	indirekt	indirekt	Codes 0–5	ja	–
W9 Python-Omics	ja	–	ja	ja	–	ja
W10 Mengenfolgen	–	–	ja	ja	–	ja
W11 Paarindex	–	–	ja	–	–	ja
W14 Orakel	–	–	–	–	ja	–
W15 heuristic	ja	ja	ja	ja	ja	–

Tabelle 8.2: Welche Bestandteile des Algorithmus welche Wiederverwendung benutzt. **W14** benutzt vom Algorithmus *nichts*, vom Kern nur die Bibliothek.

W12, W13. Prozess-Pool und Experimente sind Infrastruktur: Sie wiederverwenden die Aufrufschnittstelle, nicht die Relation; Abschnitte 9.6 und 9.7.

W14–W16. Der Solver benutzt den exakten Test als Orakel (**W14**), den Signaturmodus als unvollständigen Diagnosepfad (**W15**) und die Einbettung für die Belegungsextraktion (**W16**); Kapitel 10.

W17. Das Experimente-Framework startet den Solver als Prozess; Kapitel 11.

W18. Die Paarcharakterisierung (Satz 2.2) erklärt, warum **W2**, **W9**, **W10** und **W11** dieselbe Relation berechnen; sie ist die einzige *theoretische* Wiederverwendung.

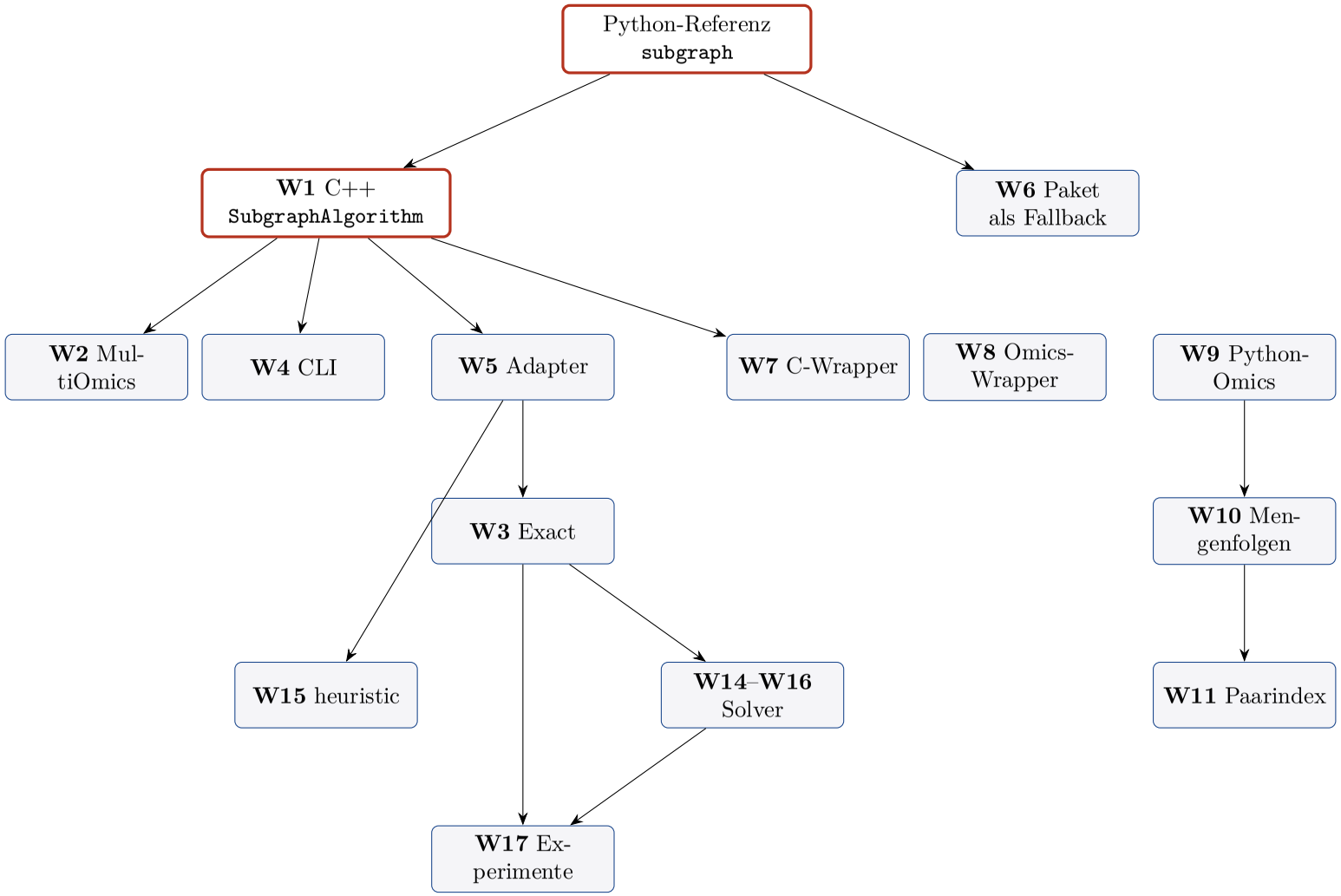


Abbildung 8.1: Abhängigkeiten zwischen den Wiederverwendungen. Pfeile bedeuten „wird wiederverwendet in“. **W12**, **W13** und **W18** betreffen mehrere Zweige und sind nicht eingezeichnet.

Kapitel 9

Gen-DB

Gen-DB ist die größte Anwendung und enthält die meisten Wiederverwendungen (**W6** bis **W13**). Die fachliche Logik (Auswahl der Kandidaten, Bewertung, biologische Relevanz) bleibt in Gen-DB; der Kern liefert nur die Relation.

9.1 Das Python-Paket als Abhängigkeit (W6)

Die Datei `pyproject.toml` fordert die Abhängigkeit aus dem Repository `subgraph @ git+https://github.com/naphez86/subgraph.git@main`. Der Quelltext benutzt ausschließlich die öffentliche API: `from subgraph import Subgraph`, dann `Subgraph().compare_graphs(A, B)`. Das Paket ist der Rückfallweg, wenn die C++-Bibliothek nicht genutzt werden kann (Abschnitt 9.3). Die Abhängigkeit zeigt auf den Zweig `main` und nicht auf eine feste Version; ein Wechsel am Zweig ändert das Verhalten ungeprüft (Kapitel 15).

9.2 C-Wrapper und `ctypes` (W7, W8)

Python kann eine statische Bibliothek nicht laden. Gen-DB (`csubgraph_native.py`) linkt deshalb beim ersten Gebrauch `libsubgraphlib.a` mit einem kleinen C-Wrapper zu einer gemeinsamen Bibliothek und lädt diese mit `ctypes` in den Prozess. Es gibt keinen Prozessstart und keine JSON-Kodierung je Vergleich.

- *Auffinden:* Die Umgebungsvariable `CSUBGRAPH_LIB_PATH` zeigt auf die `.a`-Datei oder einen Ordner, der sie enthält (Projektordner, `build/`, `build/Release`).
- *Bauen:* Aufruf von `g++ -std=c++17 -O2 -shared -fPIC` mit `--whole-archive` um die Bibliothek; ist sie mit `--coverage` gebaut, kommt `-lgcov` hinzu.
- *Cache:* Das Ergebnis liegt in einem Cache-Ordner, benannt nach einem Hash aus Bibliothek, Wrapper, Header und ABI-Version; es wird nur neu gebaut, wenn sich eines davon ändert. Das Schreiben erfolgt in eine Temporärdatei und wird atomar umbenannt, damit parallel startende Worker keine halbe Datei laden.
- *Vertrag:* `int`-Felder, ein Fehlerpuffer von 512 Byte und eine Versionsfunktion (`csub_abi_version`, `csub_omics_abi_version`). Stimmt die Version nicht, wird die Bibliothek nicht benutzt.

– *Zwei Wrapper:* Einzelvergleich (**W7**) und Multi-Omics (**W8**) sind getrennt gebaut. Enthält ein älteres Archiv die Klasse `MultiOmics` nicht (Textsuche im Archiv), bleibt der Einzelvergleich nutzbar und nur Multi-Omics fällt auf Python zurück.

Ein Schichtstapel mit L Schichten der Größe n wird als `int`-Feld der Länge Ln^2 übergeben, Eintrag (Schicht l, Zeile i, Spalte j) an Position $ln^2 + in + j$. Die Ergebniscodes 0 bis 5 werden in Python in die Namen aus Tabelle 5.2 zurückübersetzt; die Zuordnung steht an vier Stellen (Kapitel 15).

9.3 Fallbacks: W6 und W9

Der Executor (`subgraph_executor.py`) löst die Implementierung je Prozess einmal auf: Ist `CSUBGRAPH_LIB_PATH` gesetzt und die Bibliothek nutzbar, läuft C++; sonst (nicht gesetzt, Compiler fehlt, Build- oder Ladefehler) läuft Python mit einer Warnung im Log. Für Multi-Omics gilt dieselbe Auswahl getrennt: C++ (**W8**) oder `multiomics_python` (**W9**).

W9 ist eine Neuimplementierung der C++-Relation in Python (136 Zeilen, Funktionen `row_components`, `contains`, `compare_layered`). Sie entspricht **W2** mit Strategie `BIGRAM`. Weil es kein gemeinsamer Quelltext ist, verlangt sie Differenzialtests: Die Experimente prüfen 5 000 Paare in beiden Modi, also 10 000 Vergleiche, ohne Abweichung (Gen-DB-Messdatei `search_multiomics_20261008_115430`, Windows). Eine eigene Prüfung unter Linux bestätigt das (Kapitel 14).

9.4 Die Relation auf Mengenfolgen (W10)

`universal_core` implementiert die Verallgemeinerung aus Abschnitt 6.3. Die Klasse `Stack` hält L Schichten aus `frozenset`-Komponenten, `contained_independent` und `contained_coherent` werten Satz 2.2 direkt aus (`linear_pairs` gegen `cyclic_pairs`), `compare_stacks` liefert die Ergebnisnamen der C++-Bibliothek. Aktive Schichten können über eine Indexmenge gewählt werden (Projektion). Die Funktionen `stack_from_matrices` und `matrices_from_stack` verbinden den Kern mit Matrixstapeln.

Eine Abweichung ist bewusst: `compare_stacks` entscheidet `EQUAL_KEEP_*` nach der *Belegung* (Summe der Komponentengrößen), die C++-Fassung nach Kantenzahl. Für Matrixstapel in lokalen Koordinaten ist die Belegung gleich der Kantenzahl, sodass beide übereinstimmen.

9.5 Der invertierte Paarindex (W11)

`universal_index` (und in der Datenbank die Tabelle `universal_pairs`) bildet die Paarmengen aller gespeicherten Stapel auf Trefferlisten ab. Zu jedem Zahlenpaar (u, v) je Schicht ℓ und Art (cyc oder lin) steht die Liste der Schlüssel; Komponenten werden über ein injektives Wörterbuch je Schicht auf Zahlen abgebildet.

Satz 9.1 (Korrektheit des Index)**.** Sei $\mathcal{D} = \{(k, M_k)\}$ eine Datenbank von Stapeln desselben Schemas, Q eine Anfrage mit m_Q Spalten und I eine Menge aktiver Schichten. Dann ist

$$R_\ell(Q) = \bigcup_{(u,v)\in P_{\mathrm{lin}}(q^{(\ell)})} \mathrm{cyc}_\ell[(u,v)] \;\cap\; \{k : |M_k| \geq m_Q\}, \qquad \mathrm{Match}_{\mathrm{ind}}(Q, I) = \bigcap_{\ell\in I} R_\ell(Q)$$

genau die Menge der Schlüssel k mit $Q \sqsubseteq_{\text{ind}} M_k$ auf I. Im Modus *kohärent* gilt $\text{Match}_{\text{coh}}(Q, I) \subseteq \text{Match}_{\text{ind}}(Q, I)$, und die Verifikation der Kandidaten aus $\text{Match}_{\text{ind}}$ mit der kohärenten Relation liefert genau $\text{Match}_{\text{coh}}$.

Beweis. Nach Satz 2.2 gilt $q^{(\ell)} \preceq m_k^{(\ell)}$ genau dann, wenn $P_{\text{lin}}(q^{(\ell)}) \cap P_{\text{cyc}}(m_k^{(\ell)}) \neq \emptyset$ und $m_Q \leq |M_k|$. Ein Schlüssel k steht genau dann in $\text{cyc}_\ell[(u, v)]$, wenn $(u, v) \in P_{\text{cyc}}(m_k^{(\ell)})$. Also ist $k \in R_\ell(Q)$ genau dann, wenn die Schicht ℓ die Bedingung erfüllt; der Schnitt über $\ell \in I$ ist die Konjunktion. Paare mit einer im Wörterbuch unbekannten Komponente kommen in keiner Liste vor und tragen nichts bei. Kohärent: Ein gemeinsames Paar der Tupelfolgen liefert in jeder aktiven Schicht dieselbe Position als gemeinsames Paar der Einzelschichten, also $\text{Match}_{\text{coh}} \subseteq \text{Match}_{\text{ind}}$; die Verifikation entfernt genau die Kandidaten, die nicht kohärent sind. □

Der Aufwand ist $O(\nu_I(Q) + |I|\, m_Q + H)$ mit der Trefferlast H (Summe der gelesenen Listenlängen) und hängt nicht von der Zahl der gespeicherten Stapel ab, soweit sie nicht in Trefferlisten vorkommen. Gen-DB setzt das in `crud.search_universal` um. Die Messung (Abschnitt 9.9) zeigt den Effekt.

9.6 Der Prozess-Pool (W12)

Der Algorithmus ist CPU-gebunden; deshalb laufen die Vergleiche in einem `ProcessPoolExecutor`, damit der Webserver nicht blockiert. Die Zahl der Worker steht in `SUBGRAPH_MAX_WORKERS` (Standard $\max(\text{CPUs}, 2)$), die Chunkgröße ist 256. Die Funktionen `compare_many`, `compare_many_multiomics` und `compare_many_universal` verteilen eine Anfrage gegen viele Kandidaten; jeder Worker löst die Implementierung einmal auf. Der Pool kennt keine Relation, er ruft nur die jeweilige Vergleichsfunktion auf; er wäre für jeden anderen CPU-gebundenen Vergleich genauso einsetzbar. Die Indexsuche braucht den Pool nicht.

9.7 Experimente und Differenzialtests (W13)

Gen-DB enthält vier Experimentskripte (`experiment_search_workers`, `_backends`, `_multiomics`, `_universal`), die jeweils eine Aufgabe haben: Skalierung mit Workern, Python gegen C++, Multi-Omics-Suche und Indexsuche gegen Einzelvergleiche. Gemeinsam ist ihnen, dass sie alle Fassungen aus diesem Kapitel gegeneinander prüfen: Treffer müssen gleich sein, ein stiller Fallback von C++ auf Python wird erkannt, und ein Korrektheitsteil vergleicht die Relation auf Zufallspaaren, eingebetteten, identischen und umgekehrten Paaren in beiden Modi. Das ist die systematische Absicherung der Formen Neuimplementierung und Verallgemeinerung.

9.8 Ablauf einer Suche: Sequenzdiagramm

Die bisherigen Abschnitte beschreiben die Bausteine einzeln. Wie sie bei *einer* Suche zusammenspielen, zeigt das UML-Sequenzdiagramm in Abbildung 9.1. Es folgt dem Use Case *Subgraph-Suche* von der Eingabe im Browser bis zur Anzeige des Suchergebnisses. Die Vorlage ist der Quelltext von Gen-DB (`frontend/index.html`, `backend/app.py`, `crud.py`, `subgraph_executor.py`, `csubgraph_native.py`, `native/csubgraph_shim.cpp`); die Nachrichtennamen sind die dort verwendeten Funktions- und Routennamen.

Beteiligte

Das Diagramm hat neun Lebenslinien aus drei Sprach- und Prozessbereichen (Tabelle 9.1). Der Browser führt JavaScript aus, der Webserver und der Pool-Hauptprozess sind Python. Die Vergleiche laufen in *Worker-Prozessen*, die selbst Python-Prozesse sind und entweder die C++-Bibliothek über `ctypes` aufrufen (rot umrandet) oder das Python-Paket `subgraph` (Fallback). Die Grenze zwischen Python und C++ verläuft damit *innerhalb* des Worker-Prozesses: Es gibt keinen Prozessstart und keine JSON-Kodierung je Vergleich (Abschnitt 9.2).

Lebenslinie	Sprache, Prozess	Rolle bei der Suche
`Browser` (`index.html`)	JavaScript, Browser	liest Labels und Matrix, sendet die Anfrage, zeigt die Treffer an
`FastAPI` (`app.py`)	Python, Webserver	Route `POST /api/networks/search`, Validierung, Antwortschema
`crud` (`crud.py`)	Python, Webserver	Kandidatenauswahl, Auswertung der Vergleichsergebnisse, `SearchMatch`
`PostgreSQL`	SQL, Datenbankserver	Tabellen `biological_networks` und `network_matrices`
`subgraph_executor`	Python, Webserver	`compare_many`, Pool, einmalige Auflösung der Bibliothek
`Worker-Prozess`	Python, Pool-Worker	`execute_subgraph_comparison`, Aufruf über `ctypes` oder Python-Paket
`csubgraph_shim`	C-ABI (C++), im Worker	`csub_compare`, `.so`/`.dll` aus Wrapper und `libsubgraphlib.a`
`SubgraphAlgorithm`	C++17, im Worker	`compareGraphs`, Kern aus `github.com/naphez86/csubgraph`
`subgraph` (Python-Paket)	Python, im Worker	`Subgraph().compare_graphs`, Referenz aus `github.com/naphez86/subgraph`

Tabelle 9.1: Lebenslinien des Sequenzdiagramms (Abbildung 9.1).

Schritt für Schritt

Tabelle 9.2 ordnet jeden Schritt der Nummerierung im Diagramm einer Stelle im Quelltext zu.

Tabelle 9.2: Schritte der Suche (Nummern wie in Abbildung 9.1) und ihre Stellen im Quelltext.

Nr.	Schritt	Quelltext und Erläuterung
0	Eingabe prüfen	`searchSubgraph()` in `index.html`: Labels (kommagetrennt) und Matrix aus den Eingabefeldern; ohne Labels oder ohne Kante erscheint eine Meldung und es wird nichts gesendet.

Nr.	Schritt	Quelltext und Erläuterung
1	Anfrage senden, validieren	`fetch(API_BASE + '/networks/search', POST)` mit `node_labels` und `adjacency_matrix` als JSON. In `app.py` prüft das Pydantic-Schema `NetworkSearch` (`schemas.py`) die Eingabe; mindestens ein Label.
2	Fachlogik aufrufen	`search_networks` ruft `crud.search_subgraph(query_matrix, query_labels)`; Ausnahmen werden zu HTTP 500.
3–4	Kandidaten laden	Join von `biological_networks` und `network_matrices` mit `node_count >= n` und `edge_count >= e` der Query, sortiert nach Knotenzahl. Nur gleich große oder größere Netzwerke können die Query enthalten.
5	Vergleich verteilen	`compare_many(query, [Matrix je Kandidat])`; die Query ist A, der Kandidat ist B.
6	Einmalige Auflösung	`get_executor()` löst vor dem Start der Worker `_get_native()` und `_get_native_omics()` auf: Bibliothek suchen, Wrapper bauen, laden (Kapitel 13). Danach startet der `ProcessPoolExecutor` (`SUBGRAPH_MAX_WORKERS`, in der `.env` auf 2 gesetzt). Spätere Suchen überspringen den Schritt.
7	Chunks verteilen	`executor.map(_compare_to_query, …, chunksize=256)`: Jeder Worker erhält Pakete von 256 Kandidaten.
8	Eingaben prüfen	`execute_subgraph_comparison`: beide Matrizen als `numpy`-Arrays, nichtleer und quadratisch; Fehler werden als Text zurückgegeben, nicht geworfen.
9–12	Weg über C++	Fall *alt, erster Zweig*: `NativeSubgraph.compare` übergibt die Matrizen zeilenweise als `int`-Felder per `ctypes` an `csub_compare` (`csubgraph_shim.cpp`). Der Wrapper baut `std::vector<std::vector<int>>`, ruft `SubgraphAlgorithm::compareGraphs` und liefert den Code 0 bis 5, bei Fehlern -1 mit Text im 512-Byte-Puffer. Zurück im Worker übersetzt `_compare_with_csubgraph` den Code in die Namen des Python-Algorithmus (`_CSUBGRAPH_RESULTS`); `IDENTICAL` wird nach Kantenzahl zu `equal_keep_A` oder `equal_keep_B`.
13–14	Weg über Python	Fall *alt, zweiter Zweig*: `_compare_with_python` erzeugt je Prozess einmal `Subgraph()` und ruft `compare_graphs`. Dieser Weg gilt, wenn `CSUBGRAPH_LIB_PATH` leer ist oder die Bibliothek nicht nutzbar ist (Warnung im Log).
15–16	Ergebnisse sammeln	Jeder Aufruf liefert `(result, error)`; `executor.map` gibt die Liste in der Reihenfolge der Kandidaten zurück.
17	Treffer bestimmen	`crud.search_subgraph`: Fehlerfälle werden gezählt und übersprungen (ein defekter Kandidat bricht die Suche nicht ab). Treffer sind `keep_B`, `equal_keep_A` und `equal_keep_B` (`MATCH_RESULTS`); `equal_*` ergibt den Match-Typ `exact`, sonst `subgraph`.

Nr.	Schritt	Quelltext und Erläuterung
18–19	Antwort	`search_match_to_response` je Treffer, `SearchResponse(success=True, data=…)`, HTTP 200 mit JSON.
20	Anzeige	Der Browser zeigt Name, Typ, Organismus, Match-Typ, Knoten- und Kantenzahl, Labels und `subgraph_result`; bei leerer Liste „Keine Übereinstimmungen gefunden“.

Was das Diagramm über das Zusammenspiel zeigt

1. **Drei Sprachbereiche, zwei Prozessgrenzen.** Die Anfrage überquert die Netzgrenze (Browser→Webserver), die Prozessgrenze zum Pool (Schritt 7, `pickle`-Übertragung der Matrizen) und die Sprachgrenze Python→C++ (Schritt 9, Funktionsaufruf im selben Prozess). Nur die letzte ist billig; sie kostet nach der eigenen Messung etwa 0,018 ms (Tabelle 9.3).

2. **Der Algorithmus ist austauschbar, die Suche nicht.** Die Schritte 0 bis 8 und 15 bis 20 sind für beide Zweige gleich. Nur der `alt`-Rahmen (Schritte 9 bis 14) enthält Algorithmus-Code. Deshalb ist die Fachlogik (Kandidatenauswahl, Treffer, Antwort) von der Wahl zwischen **W6** und **W7** unabhängig (Kapitel 12).

3. **Die Auswahl fällt einmal, nicht je Vergleich.** Schritt 6 trifft die Entscheidung zwischen C++ und Python je Prozess einmal; im Worker kostet die Abfrage nur eine Variable. Wo der Ladevorgang im Einzelnen abläuft, beschreibt Kapitel 13.

4. **Fehler laufen als Werte, nicht als Ausnahmen.** Vom Wrapper bis `crud` werden Fehler als `(None, Text)` weitergereicht; nur ein Fehler außerhalb der Vergleiche (Datenbank, Validierung) führt zu HTTP 500.

5. **Die Reihenfolge bleibt erhalten.** Weil `executor.map` die Reihenfolge der Kandidaten bewahrt, genügt in Schritt 17 ein `zip(candidates, outcomes)`.

Die Multi-Omics-Suche (`POST /api/multiomics/search`) und die universelle Suche folgen demselben Ablauf mit anderen Funktionsnamen (`compare_many_multiomics`, zweiter Wrapper `csubgraph_omics_shim`); die Indexsuche (**W11**) ersetzt die Schritte 5 bis 16 durch Listenzugriffe in der Datenbank und braucht weder Pool noch Bibliothek (Abschnitt 9.5).

9.9 Messungen aus Gen-DBs eigenen Dateien

Die folgenden Werte stammen aus den Messdateien im Gen-DB-Repository (Windows 11, Python 3.14, vier logische CPUs). Sie wurden *nicht* neu gemessen.

Der Gesamtgewinn (6,36) ist deutlich kleiner als der Gewinn je Einzelvergleich (≈ 37), weil mit der C++-Fassung der serielle Anteil der Suche (Datenbankabfrage, Prozesskommunikation, Trefferliste) 66,7 % der Zeit ausmacht: Der Rechenkern ist nicht mehr der Engpass. Das ist eine Aussage über die *Kopplung*, nicht über den Algorithmus.

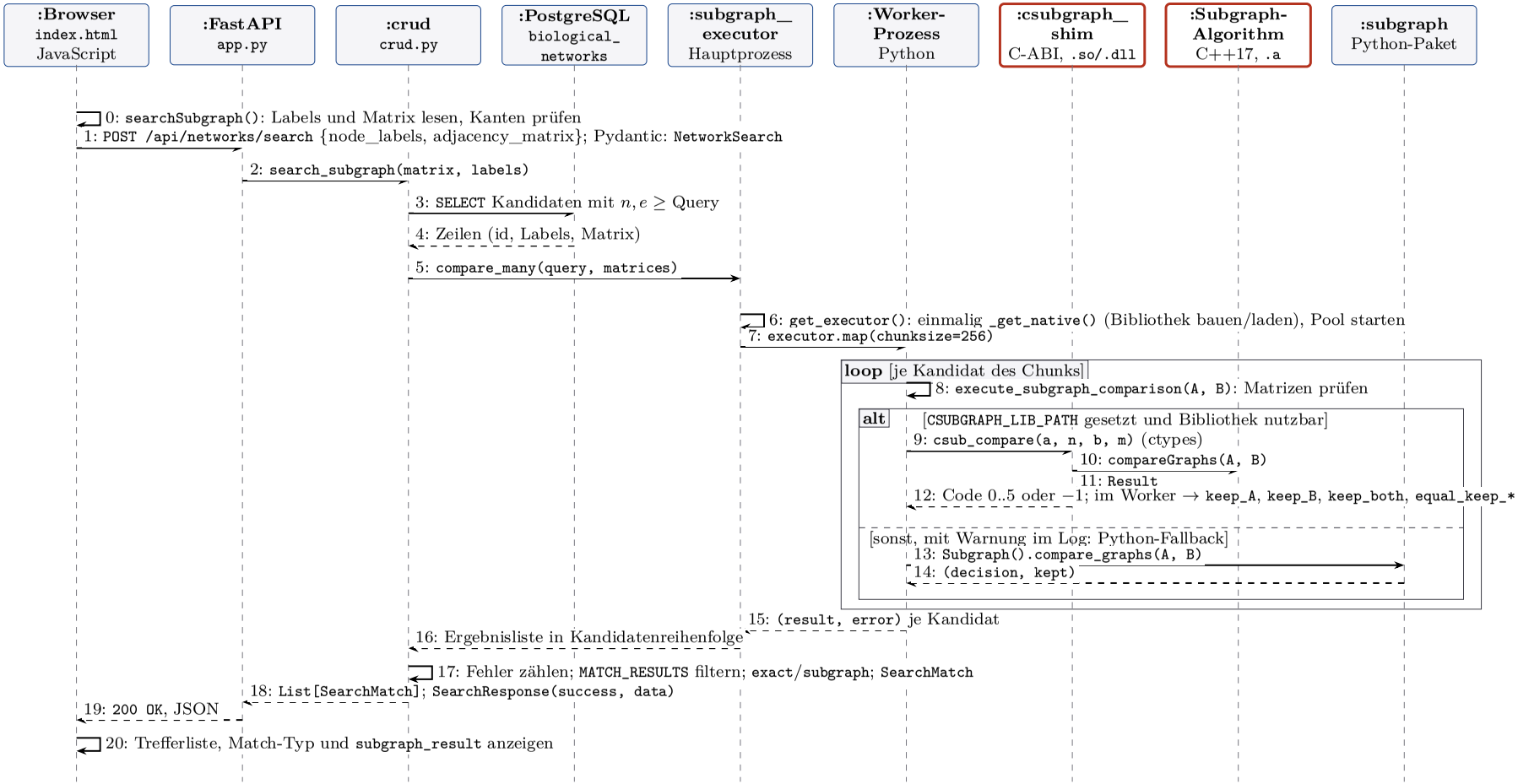


Abbildung 9.1: UML-Sequenzdiagramm der Subgraph-Suche in Gen-DB (Nummern wie in Tabelle 9.2). Durchgezogene Pfeile sind Aufrufe, gestrichelte Rückgaben. Rot umrandet: C/C++-Code, der im Worker-Prozess über `ctypes` aufgerufen wird; alle anderen Lebenslinien sind Python (bzw. JavaScript und SQL).

Größe	Python (**W6**)	C++ (**W7**)	Verhältnis
Gesamtzeit, 6 Anfragen, 2 Worker	1180,3 s	185,6 s	6,36
Kandidaten insgesamt	931 899 (beide)		–
Zeit je Kandidat (Gesamtzeit/Kandidaten)	1,27 ms	0,20 ms	–
Anteil der Vergleichsphase an der Gesamtzeit	88,4 %	33,3 %	–
Mikro-Benchmark je Einzelvergleich (Mittel)	2,69 ms	0,074 ms	≈ 37
Aufrufkosten der Brücke (Mittel)	–	0,018 ms	–

Tabelle 9.3: Python gegen C++ in Gen-DB (`search_backends_20261007_104804`). Beide Implementierungen lieferten dieselben Treffer, ohne fehlgeschlagene Vergleiche.

Die Indexsuche (**W11**) vermeidet die Vergleiche ganz: In der Messreihe `search_universal_20261010_103804` (Windows, 112 Anfragen je Variante) betrug die Median-Zeit je Anfrage 0,107 s für den Index gegenüber 12,3 s für Einzelvergleiche in Python ($\approx$ 115-fach), die Summe 21,0 s gegenüber 1689,1 s ($\approx$ 80-fach). Die Trefferlisten waren in allen 112 Fällen gleich (Hash der Trefferlisten), der Korrektheitsteil mit 5 000 Paaren meldete keinen Verstoß.

Kapitel 10

Der Subgraph-SAT-Solver

Der Solver ist die mathematisch präziseste Nutzung. Er braucht kein Signaturverfahren, sondern ein Orakel nach Definition 2.9 und benutzt deshalb `ExactSubgraph` (**W14**); den Signaturalgorithmus kennt er nur als Diagnosepfad (**W15**). Er enthält keinen Subgraph Algorithmus: In den Dateien `Cnf`, `Reduction`, `SubgraphSATSolver` und `main` kommen weder `SubgraphAlgorithm` noch `ExactSubgraph` vor, und `CMakeLists.txt` bindet `csubgraph` nicht ein.

10.1 Die Reduktionskette

Der Solver führt `CNF` → `3-SAT` → `Clique` → `Subgraph` aus.

1. `CNFFormula::to_3sat` teilt Klauseln mit mehr als drei Literalen: Aus $(r_0 \vee r_1 \vee \cdots)$ entsteht $(r_0 \vee r_1 \vee x)$ und die Restklausel $(r_2 \vee \cdots \vee \neg x)$, mit frischer Variable x, iterativ. Danach entfernt `simplify` Tautologien und doppelte Klauseln.

2. `ThreeSATToCliqueReducer` baut G_φ nach Satz 2.1; Knoten sind die Paare (Klausel, Position) in Klausel-vor-Position-Reihenfolge, Kanten verbinden Knoten verschiedener Klauseln mit nicht komplementären Literalen.

3. `CliqueToSubgraphReducer` liefert das Muster K_m (m = Klauselzahl).

4. Beide Graphen werden als JSON geschrieben (Quelle G, Ziel $H = K_m$).

Lemma 10.1 (Klauselteilung)**.** Sei φ' aus φ durch `to_3sat` entstanden. Erfüllt eine Belegung α die Formel φ', so erfüllt α (eingeschränkt auf die ursprünglichen Variablen) auch φ.

Beweis. Sei $C = (r_1 \vee \cdots \vee r_k)$ mit $k > 3$ in φ in die Klauseln $c_1, \ldots, c_{k-2}$ geteilt worden, wobei c_i für $i < k-2$ die Form $(u_i \vee v_i \vee x_i)$ hat und c_{k-2} aus den übrig gebliebenen Literalen besteht; jedes u_i, v_i ist ein ursprüngliches Literal von C oder ein $\neg x_j$ mit $j < i$. Angenommen, alle r_j wären unter α falsch, aber alle c_i wahr. Per Induktion über i folgt x_i wahr: Die Literale u_i, v_i sind falsch (ursprünglich falsch, oder $\neg x_j$ mit x_j wahr nach Induktionsannahme), also muss x_i wahr sein. Die letzte Klausel c_{k-2} enthält dann nur falsche Literale (ursprüngliche oder $\neg x_j$ mit wahrem x_j). Widerspruch. Also ist jede ursprüngliche Klausel erfüllt. Die übrigen Klauseln bleiben unverändert. □

Die umgekehrte Richtung (jede erfüllende Belegung von φ lässt sich auf die frischen Variablen fortsetzen) wird für die Korrektheit der Ablehnung gebraucht und gilt ebenfalls: Ist C durch α erfüllt, etwa durch r_t, setzt man x_i wahr genau für die Klauseln c_i, die vor der Position von r_t liegen und nicht schon durch ihre Literale erfüllt sind; auf die Angabe des vollen Beweises wird verzichtet (die 300 Zufallsformeln der Prüfung in Kapitel 14 sind damit konsistent).

10.2 Dateiprotokoll und Adapter

Der Solver schreibt `input.json` mit den Schlüsseln `G` und `H` (je `nodes` und `edges`), startet das Programm aus `--subgraph-binary` (Linux: `fork`/`execl`, Windows: `system`) mit zwei Dateinamen und liest `output.json`. Rückgabecode $\neq 0$ ergibt den Status `SAT_ERROR`; ein Zeitlimit des Adapters (Exit-Code 3) wird also *nicht* als `UNSAT` gemeldet. Die Antwort wird nicht mit einem JSON-Parser, sondern durch Textsuche nach `"is_subgraph": true` und `"mapping"` ausgewertet. Dazu muss der Adapter diese Schreibweise (ein Leerzeichen nach dem Doppelpunkt) einhalten (Abschnitt 7.2).

Bemerkenswert ist, dass der Solver kein `nlohmann/json` einbindet, obwohl `CMakeLists.txt` es anfordert und notfalls herunterlädt; die Abhängigkeit ist überflüssig.

10.3 Belegungsextraktion und Korrektheit (W16)

Bei *ja* liefert der Adapter ein `mapping`: Für jeden Knoten von K_m den Index des Bildknotens in G_φ. Die Knoten sind in der Reihenfolge Klausel vor Position nummeriert; der Solver baut dieselbe Nummerierung aus der geteilten, vereinfachten Formel neu auf.

Satz 10.1 (Korrektheit der Belegungsextraktion)**.** Sei φ'' die Formel nach `to_3sat` und `simplify` mit $m \geq 1$ Klauseln und $\iota : K_m \to G_{\varphi''}$ eine Einbettung. Setzt man für jeden Bildknoten $\iota(v) = (c, p)$ die Variable von $\lambda(c, p)$ so, dass das Literal wahr wird, und alle übrigen Variablen beliebig (der Code: wahr), so ist die Belegung wohldefiniert und erfüllt φ''. Ihre Einschränkung auf die ursprünglichen Variablen erfüllt die ursprüngliche Formel.

Beweis. Die m Bildknoten sind paarweise benachbart, liegen also in paarweise verschiedenen Klauseln. Bei m Klauseln enthält jede Klausel genau einen Bildknoten. Zwei Bildknoten tragen keine komplementären Literale, also ist keine Variable auf zwei verschiedene Werte gesetzt (wohldefiniert; die Regel „erste Zuweisung gilt“ im Code wird dadurch folgenlos). Jede Klausel enthält ein wahr gesetztes Literal, also ist φ'' erfüllt. Da `simplify` nur Tautologien und Duplikate entfernt, erfüllt die Belegung auch φ' (die entfernten Klauseln sind entweder immer wahr oder gleich einer behaltenen), nach Lemma 10.1 also die ursprüngliche Formel. □

Der Solver prüft die Belegung zusätzlich (`validate_result`, Standard an) und meldet bei Fehlschlag `SAT_ERROR` statt `SATISFIABLE`. Damit ist ein falsches `SAT` ausgeschlossen, selbst wenn das Orakel oder die Extraktion fehlerhaft wären. Fehlt das `mapping` (etwa im Signaturmodus), bleibt es bei der Belegung „alle Variablen wahr“; diese Belegung wird ebenfalls geprüft.

10.4 Ein durchgerechnetes Beispiel

Die Formel $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ steht als DIMACS-Datei vor.

Listing 10.1: Eingabe `f1.cnf`

```
p cnf 3 3
1 2 0
-1 3 0
-2 -3 0
```

Reduktion. Die Formel ist schon 3-SAT, $m = 3$. Die sechs Knoten sind (Klausel, Position) mit Literalen: v_0: x_1, v_1: x_2 (Klausel 1), v_2: $\neg x_1$, v_3: x_3 (Klausel 2), v_4: $\neg x_2$, v_5: $\neg x_3$ (Klausel 3). Kanten verbinden Knoten verschiedener Klauseln mit nicht komplementären Literalen: v_0v_3, v_0v_4, v_0v_5, v_1v_2, v_1v_3, v_1v_5, v_2v_4, v_2v_5, v_3v_4. Das sind 9 Kanten (die Ausgabe des Reducers nennt „6 vertices, 9 edges“). Gesucht ist $K_3 \sqsubseteq_{\text{mono}} G_\varphi$.

Orakel. Der Adapter erhält diese beiden Graphen (Datei `in.json`) und liefert im exakten Modus:

Listing 10.2: Ausgabe des Adapters im Modus `exact` (echter Lauf)

```
{"is_subgraph": true, "mapping": [0, 3, 4]}
```

Die Bildknoten v_0, v_3, v_4 sind paarweise benachbart (v_0v_3, v_0v_4, v_3v_4) und bilden die Clique.

Extraktion. v_0 hat das Literal x_1, also x_1 = wahr; v_3 hat x_3, also x_3 = wahr; v_4 hat $\neg x_2$, also x_2 = falsch. Der Solver meldet `SATISFIABLE` mit $\{x_1{=}T, x_2{=}F, x_3{=}T\}$. Probe: $(T \vee F)$, $(F \vee T)$, $(T \vee F)$ sind wahr.

Der Signaturmodus am selben Beispiel. Mit `SUBGRAPH_MODE=heuristic` liefert derselbe Adapter für dieselbe Eingabe `{"is_subgraph": false}`. Der Solver würde `UNSATISFIABLE` melden, obwohl die Formel erfüllbar ist: eine konkrete Folge der Unvollständigkeit aus Beispiel 2.1.

Unerfüllbares Gegenstück. Für $(x_1) \wedge (\neg x_1)$ entsteht ein Graph mit zwei Knoten ohne Kante; K_2 ist nicht enthalten, der Solver meldet `UNSATISFIABLE`.

10.5 Der Signaturmodus als Diagnosepfad (W15) und Grenzen

Der Signaturmodus ist unvollständig im Sinne von Definition 2.9: Bei 300 zufälligen CNF-Formeln lieferte der Modus `heuristic` in 130 Fällen `UNSAT` für erfüllbare Formeln und endete in 5 Fällen mit einem Fehlerstatus (Kapitel 14); die Ursache der fünf Fehler ist nicht untersucht. Der Modus `exact` lieferte dagegen in allen 300 Fällen die richtige Antwort und nie eine ungültige Belegung. Der Signaturmodus hat außerdem die Grenze von 63 Knoten, die bei Reduktionsgraphen schnell überschritten wird.

Der exakte Modus ist im ungünstigsten Fall exponentiell. Der Kommentar im Solver-Header, der von „polynomieller Zeit“ und $O(n^3)$ spricht, trifft nur auf den Signaturmodus zu und ist für den exakten Modus falsch. Das Zeitlimit des Solvers (`subgraph_timeout_sec`, `--timeout`) wird nicht an den Adapter weitergereicht; dieser verwendet `SUBGRAPH_TIMEOUT_SEC` mit Standard 60 Sekunden.

Kapitel 11

Das Experimente-Framework

Das Experimente-Framework (**W17**) ist das Projekt `github.com/naphez86/subgraph-sat-solver-experiments`. Es erzeugt Boolean Circuits, übersetzt sie in CNF, startet den Subgraph-SAT-Solver als Prozess, misst die Laufzeit und legt die Ergebnisse als JSON, CSV und Diagramme ab. Dieses Kapitel beschreibt die Experimente ausführlich: ihren Aufbau, die Instanzen, die Messmethodik, alle Ergebnisse beider Messreihen und ihre Grenzen.

Als Quelle dient die eigene wissenschaftliche Arbeit des Projekts, *Experimentelle Analyse: Subgraph-SAT-Solver auf Boolean Circuits* (`science/subgraph-sat-solver-experiments.tex`). Alle Zahlen dieses Kapitels sind dieser Arbeit entnommen; sie wurden *nicht* neu gemessen, und der Quelltext des Frameworks wurde im Rahmen der vorliegenden Arbeit weder neu ausgeführt noch geprüft (vgl. die Aussagearten in Abschnitt 1.2). Ausgenommen sind die wenigen Rechnungen, die ausdrücklich als solche gekennzeichnet sind.

Für die Wiederverwendungskette ist das Framework das letzte Glied: Es benutzt weder den Subgraph Algorithmus noch die Bibliothek `csubgraph`, sondern nur die Außenfläche des Solvers (Abschnitt 11.2). Was die Messungen über den Algorithmus aussagen und was nicht, ist deshalb eine eigene Frage, die Abschnitt 11.10 beantwortet.

11.1 Gegenstand und die beiden Messreihen

Der Subgraph-SAT-Solver reduziert eine CNF-Formel in drei Schritten auf ein Subgraph-Problem (Abschnitt 10.1). Die Experimente sollen zeigen, wie sich dieser Solver auf wachsenden Instanzen verhält. Die Instanzen sind Boolean Circuits, die durch die Tseitin-Transformation [4] in CNF übersetzt werden; die Target-AND-Circuits gehen auf die Masterarbeit des Autors [6] zurück.

Abbildung 11.1 zeigt den Weg einer Instanz durch das Framework. Nur die Mitte (der Solver-Prozess) gehört nicht zum Framework; sein Inneres, die Reduktionskette und der Adapter aus Kapitel 10, ist für das Framework unsichtbar.

Es gibt zwei Messreihen, die sich in Messaufbau, Instanzen und Auswertung unterscheiden (Tabelle 11.1). Die *erste Messreihe* vom 14. Juli 2026 umfasst 288 Läufe in vier Suites. Die *neue Version* (Version 2.0.0 des Frameworks) wurde am 11. Oktober 2026 ausgeführt und setzt die sieben Verbesserungsvorschläge aus der Diskussion der ersten Messreihe um. Sie liegt nur als Zwischenstand des Full-Presets vor.

Die Zahlen beider Reihen dürfen nicht ohne Weiteres verglichen werden: Die Formeln tragen zwar dieselben Namen (etwa `c17` oder `TargetAND_m10`), sind aber wegen der kor-

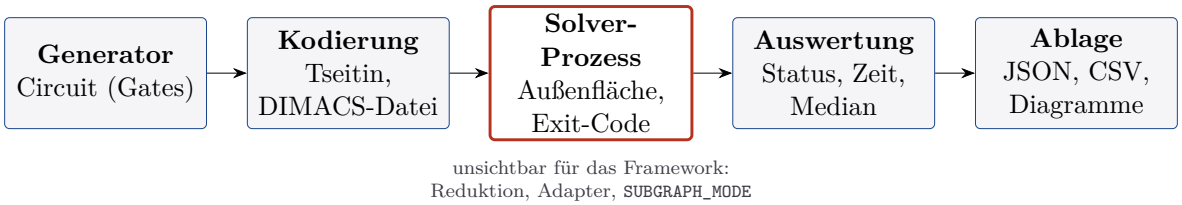


Abbildung 11.1: Weg einer Instanz durch das Experimente-Framework. Der Solver-Prozess ist die einzige Kopplung (**W17**).

	Erste Messreihe	Neue Version 2.0.0
Datum	14. Juli 2026, 09:40:03 bis 09:40:48 Uhr	11. Oktober 2026
Umfang	288 Läufe in vier Suites (Quick, Standard, Full, Performance)	Full-Preset mit 46 Instanzen, ausgewertet 44 (Zwischenstand)
Solver	Subgraph-SAT-Solver	Subgraph-SAT-Solver und Python-Referenzsolver
Zweck	Laufzeit über Instanzgröße (F1 bis F4)	dieselben Fragen, mit behobenen Mängeln
Befund	Messwerte von Prozessstart und Zeitauflösung dominiert	Laufzeit wächst deutlich mit der Instanzgröße

Tabelle 11.1: Die beiden Messreihen des Experimente-Frameworks im Überblick.

rigierten Kodierung und der Ausgangsbedingung *andere* Formeln, und die Umgebung der ersten Reihe wurde nicht dokumentiert. Verglichen werden können nur qualitative Befunde (Abschnitt 11.9).

11.1.1 Fragestellungen

Die Experimente sollen vier Fragen beantworten.

F1 Wie verhält sich die Laufzeit des Solvers bei wachsender Zahl m der Eingänge (Target-AND-Circuits)?

F2 Wie verhält sie sich bei wachsender Tiefe d zufälliger Zwischenschichten (Target-AND-Random)?

F3 Wie verhält sich der Solver auf industriellen Standard-Benchmarks (ISCAS'85)?

F4 Wie reproduzierbar sind die Messungen über verschiedene Suites hinweg?

11.2 Kopplung an den Solver (W17)

Das Framework startet den Solver als eigenen Prozess (`subprocess`), übergibt die DIMACS-Datei als letztes Argument und wertet Standardausgabe, Fehlerausgabe und Rückgabecode aus. Es kennt weder die Theorie noch die Bibliothek; es sieht nur die Außenfläche des Solvers. Tabelle 11.2 stellt zusammen, was über diese Fläche läuft.

Größe	Inhalt
Eingabe	DIMACS-CNF-Datei (letztes Argument), zusätzlich der Pfad des Solvers (`--solver-path`)
Rückgabecode	10: `SAT`, 20: `UNSAT`; 0 ohne erkennbares Ergebnis; jeder andere Wert ergibt `ERROR`
Ausgabe	Zeile `Status:` des Subgraph-SAT-Solvers oder `s`-Zeile im Format der SAT-Competition (`s SATISFIABLE`)
Beobachtete Status	`SAT`, `UNSAT`, `TIMEOUT`, `ERROR`, `NO_OUTPUT`
Zeitlimit	erste Reihe 30 s, neue Version 60 s (ein Timer-Thread beendet den Prozess)

Tabelle 11.2: Außenfläche des Solvers aus Sicht des Frameworks.

Die Statusbestimmung der neuen Version ist für die Kette wichtig, weil sie genau die Verwechslung vermeidet, die Befund B2 (Kapitel 15) beschreibt. Der Subgraph-SAT-Solver endet mit dem Code 20 auch bei `UNKNOWN` und bei Fehlern. Das Framework entscheidet deshalb in dieser Reihenfolge: zuerst eine `s`-Zeile im Format der SAT-Competition, dann die Zeile `Status:` des Solvers, dann eine alleinstehende Ergebniszeile und zuletzt den Rückgabecode. Die Zeile `Status:` hat damit Vorrang vor dem Code. Ein Rückgabecode außerhalb von $\{0, 10, 20\}$ ergibt immer `ERROR`; ein Lauf ohne erkennbares Ergebnis und mit dem Code 0 ergibt `NO_OUTPUT`. Die erste Messreihe suchte dagegen frei nach der Zeichenkette `SAT` in Standard- und Fehlerausgabe und wertete `UNSAT` als Gegenteil; das traf auch auf Banner-Zeilen zu und lieferte, wie Abschnitt 11.7.4 zeigt, in keinem Lauf ein Ergebnis.

Folgen der schmalen Kopplung. Weil das Framework nur die Außenfläche sieht, kann es Zustände nicht unterscheiden, die der Solver gleich meldet. Ein Zeitlimit im Adapter erscheint dem Framework als Fehler des Solvers (Abschnitt 10.2); ein leeres `stdout` ist kein Hinweis auf Erfüllbarkeit. Welches Subgraph-Programm der Solver benutzt und in welchem Modus (`SUBGRAPH_MODE`, Tabelle 7.2), geht aus der Arbeit des Frameworks nicht hervor; ebenso wenig die Version des Solvers (Abschnitt 11.8.1). Ein Lauf im Modus `heuristic` misst aber einen anderen Algorithmus als ein Lauf im Modus `exact`. Das ist die Begründung für den Ausblick, Modus, Zeitlimits und Versionsstände in jede Messreihe aufzunehmen (Kapitel 18).

Die Kopplung hat auch eine Stärke: Das Framework lässt sich ohne Änderung auf jeden Solver richten, der das Aufrufprotokoll einhält. Die neue Version nutzt das, indem sie einen zweiten Solver auf denselben Dateien misst (Abschnitt 11.5.4).

11.3 Die Instanzen

Definition 11.1 (Boolean Circuit)**.** Ein Boolean Circuit mit m Eingängen ist eine geordnete Liste von Gates $g_1, \ldots, g_k$. Jedes Gate hat einen Typ aus $\{\mathrm{NOT}, \mathrm{AND}, \mathrm{OR}, \mathrm{NAND}, \mathrm{NOR}, \mathrm{XOR}\}$ und bezieht seine Eingaben von Eingängen oder von früheren Gates. Die *Größe* des Circuits ist die Gate-Anzahl k.

Die Übersetzung in CNF folgt der Tseitin-Transformation: Jedes Gate erhält eine eigene Variable y, und lokale Klauseln erzwingen $y \leftrightarrow \mathrm{op}(x_1, \ldots)$. Die Formelgröße wächst

dadurch linear in der Gate-Anzahl. Tabelle 11.3 zeigt die fünf Instanzfamilien; die Abschnitte danach beschreiben sie einzeln.

Familie	Parameter	Eigenschaft	Reihe
Target-AND	$m = 4, 6, \ldots, 20$	nur NOT und AND, eindeutige erfüllende Belegung	beide
Target-AND-Random	$m = 10$, $d = 1, \ldots, 12$	zufällige Zwischenschichten, fester Seed 42	beide
ISCAS'85	12 Circuits, 4 bis 3 512 Gates	industrielle Standard-Schaltkreise	beide
Miter	9 Instanzen	unerfüllbar	nur neue Version
Skalierung	$m = 100$ bis 50 000	Target-AND mit sehr vielen Eingängen	nur neue Version

Tabelle 11.3: Instanzfamilien der Experimente.

11.3.1 Target-AND

Ein Target-AND-Circuit mit m Eingängen $x_0, \ldots, x_{m-1}$ ist so gebaut, dass sein Ausgang nur unter genau einer Belegung a^* wahr wird. Diese Belegung bestimmt die Halbierungsstrategie des LBV-Verfahrens [6]: Ausgehend von der Nullbelegung werden in jeder Runde $r = 1, 2, \ldots$ die ersten $\lfloor m/2^r \rfloor$ Eingänge invertiert, bis die Blockgröße 0 erreicht ist. Der Circuit besteht aus einem NOT-Gate je Eingang und einem balancierten Binärbaum aus AND-Gates über die Literale von a^*. Die im Datensatz gespeicherte Zahl der LBV-Runden ist $\lfloor \log_2 m \rfloor + 1$.

Satz 11.1 (Größe der Target-AND-Formeln)**.** Ein Target-AND-Circuit mit $m \geq 2$ Eingängen hat genau $2m - 1$ Gates. Seine Tseitin-CNF der ersten Messreihe hat $6m - 4$ Klauseln. Die Formel der neuen Version mit Ausgangsbedingung hat $3m - 1$ Variablen, $5m - 2$ Klauseln und $11m - 6$ Literalvorkommen.

Beweis. Es gibt m NOT-Gates. Ein Binärbaum mit m Blättern hat unabhängig von m genau $m - 1$ innere Knoten, jeder ein AND-Gate; das ergibt $m + (m - 1) = 2m - 1$ Gates. In der ersten Reihe erzeugt ein NOT-Gate 2 und ein AND-Gate mit zwei Eingängen 4 Klauseln, zusammen $2m + 4(m - 1) = 6m - 4$. In der neuen Version entfällt die redundante Klausel $(\neg y \vee x_1 \vee x_2)$, ein AND-Gate erzeugt also 3 Klauseln mit zusammen $2 + 2 + 3 = 7$ Literalen; ein NOT-Gate erzeugt 2 Klauseln mit 4 Literalen. Hinzu kommt die Unit-Klausel des Ausgangs. Das ergibt $2m + 3(m - 1) + 1 = 5m - 2$ Klauseln und $4m + 7(m - 1) + 1 = 11m - 6$ Literale. Die Variablen sind die m Eingänge und die $2m - 1$ Gates, also $3m - 1$. □

Die Messdaten bestätigen den Satz: $m = 20$ ergibt 39 Gates und 116 Klauseln in der ersten Reihe sowie 98 Klauseln und 214 Literale in der neuen Version; $m = 100$ ergibt 498 Klauseln und 1 094 Literale, $m = 1000$ ergibt 4 998 und 10 994.

11.3.2 Target-AND-Random

Dieser Typ erweitert den Target-AND-Circuit mit $m = 10$ um $d \in \{1, \ldots, 12\}$ zufällige Zwischenschichten. Jede Schicht enthält $\max(2, \lfloor |L|/2 \rfloor + r)$ Gates mit gleichverteiltem $r \in \{0, 1, 2\}$, wobei $|L|$ die Größe der vorherigen Schicht ist. Jedes Gate hat einen zufälligen Typ aus $\{\text{AND}, \text{OR}, \text{NAND}, \text{NOR}, \text{XOR}\}$ und zwei oder drei zufällig gewählte Eingänge der vorherigen Schicht. Ein abschließendes AND-Gate verknüpft das Target-AND mit dem ersten Gate der letzten Zufallsschicht. Der Zufallsgenerator wird in jedem Circuit mit dem festen Seed 42 initialisiert; die Circuits sind deshalb deterministisch, und der Circuit der Tiefe $d+1$ ergänzt den der Tiefe d (bis auf das abschließende AND-Gate) um eine Schicht. Folglich ist die Streuung der Läufe innerhalb einer Tiefe reines Messrauschen und keine Variation zwischen Zufallscircuits. Die Größe wächst von 27 Gates ($d = 1$) auf 54 Gates ($d = 12$); in der neuen Version entspricht das 174 bis 409 Literalen.

11.3.3 ISCAS'85

Die ISCAS'85-Benchmarks [5] sind kombinatorische Standard-Schaltkreise der Hardware-Verifikation. Das Framework liest die `.bench`-Dateien ein und verwendet die zehn klassischen Circuits c432 bis c7552 sowie zwei kleine Beispielcircuits (c3 mit 4 und c17 mit 6 Gates). Die Größen reichen von 4 bis 3 512 Gates (c7552) und von 14 bis 10 690 Klauseln der ersten Reihe. c1355 ist die Variante von c499, in der die XOR-Gates durch NAND-Gates ersetzt sind; das wird für den Äquivalenz-Miter genutzt. Der Parser der neuen Version sortiert die Gates topologisch und meldet undefinierte Signale und Zyklen als Fehler; die BUFF-Gates, die die erste Reihe ohne Klauseln ließ, werden nun kodiert.

11.3.4 Miter-Instanzen

Ein *Miter* verbindet zwei Circuits A und B mit gemeinsamen Eingängen; sein Ausgang ist das ODER über die XOR-Verknüpfungen entsprechender Ausgänge. Mit der Ausgangsbedingung ist die Formel genau dann erfüllbar, wenn sich A und B auf mindestens einer Eingabe unterscheiden; sind sie funktional äquivalent, ist sie *unerfüllbar*. Es gibt neun Miter-Instanzen: acht *Selbst-Miter*, die einen Circuit mit einer Kopie seiner selbst vergleichen (Target-AND mit $m = 10$ und $m = 20$ sowie c17, c432, c499, c880, c1355 und c1908), und einen *Äquivalenz-Miter* von c499 gegen c1355. Alle neun sind unerfüllbar. Ein Miter enthält die Gates beider Circuits und zusätzlich die XOR- und ODER-Gates; `MiterSelf_c1908` hat deshalb 1 786 Gates, c1908 selbst 880. Die Miter sind die einzigen Instanzen, deren Unerfüllbarkeit aus der Konstruktion folgt; unerfüllbar sind in der neuen Version außerdem neun der zwölf Target-AND-Random-Instanzen (alle Tiefen außer $d = 1, 7, 11$). Sie prüfen, ob der Solver ein `UNSAT` der Reduktionskette (Korollar 2.1) liefert.

11.3.5 Skalierungsinstanzen

Target-AND mit $m \in \{100, 1000, 10\,000, 50\,000\}$ (bis zu 99 999 Gates) soll prüfen, ob ein Skalierungseffekt über das Messrauschen hinauswächst. Ausgewertet sind nur $m = 100$ und $m = 1000$; der Lauf wurde bei $m = 10\,000$ abgebrochen (Abschnitt 11.8.1).

11.4 CNF-Kodierung und ihre Korrektur

Für ein Gate mit Variable y und Eingangsvariablen $x_1, \ldots, x_k$ erzeugt die *erste* Fassung der Kodierung die Klauseln aus Tabelle 11.4.

Gate	Klauseln	Anzahl
NOT	$(\neg y \vee \neg x_1),\ (y \vee x_1)$	2
AND	$(\neg y \vee x_1 \vee \ldots \vee x_k),\ (\neg y \vee x_i)_i,\ (y \vee \neg x_1 \vee \ldots \vee \neg x_k)$	$k+2$
OR	$(y \vee \neg x_1 \vee \ldots \vee \neg x_k),\ (y \vee \neg x_i)_i,\ (\neg y \vee x_1 \vee \ldots \vee x_k)$	$k+2$
NAND, NOR	Klauselmuster von OR beziehungsweise AND	$k+2$
XOR	vier Klauseln über y, x_1, x_2	4

Tabelle 11.4: Kodierung der ersten Messreihe. Eine Prüfung gegen alle Eingabekombinationen zeigt: NOT, AND und OR sind korrekt, NAND, NOR und XOR nicht.

Der Fehler. NAND verwendet im Quelltext dasselbe Muster wie OR, NOR dasselbe wie AND, und die vier XOR-Klauseln beschreiben $y \leftrightarrow \neg(x_1 \oplus x_2)$, also XNOR. Die erzeugten Dateien beschreiben für Gates dieser drei Typen nicht die Funktion des Circuits. Betroffen sind vor allem ISCAS'85 (c17 besteht ausschließlich aus NAND-Gates, c6288 zu etwa 88 % aus NAND-, NOR- und XOR-Gates) und die Zwischenschichten von Target-AND-Random. Die Target-AND-Circuits verwenden nur NOT und AND und sind nicht betroffen. Die Arbeit hat die fehlerhafte Kodierung für die erste Reihe bewusst nicht verändert, damit die dokumentierten Ergebnisse den tatsächlich ausgeführten Experimenten entsprechen.

Keine Ausgangsbedingung. Die erste Kodierung enthält keine Unit-Klausel, die den Ausgang auf wahr festlegt. Jede Eingabebelegung lässt sich daher zu einer erfüllenden Belegung der Gesamtformel fortsetzen; die Formeln der ersten Reihe sind unabhängig vom Circuit erfüllbar. Das hat Folgen für die Aussagekraft: Es gab keine unerfüllbaren Instanzen, und ein korrekt arbeitender Solver müsste immer `SAT` melden.

Die Korrektur in Version 2.0.0. Die neue Version hat alle Gate-Typen gegen ihre Wahrheitstabellen getestet und kodiert zusätzlich BUFF, XNOR und n-äres XOR. Der erste Ausgang des Circuits wird durch eine Unit-Klausel auf wahr gesetzt (`constraint="output"`); damit sind die Formeln nicht mehr trivial erfüllbar. Die neue Kodierung ist die Voraussetzung dafür, dass die Miter überhaupt unerfüllbar sein können.

11.5 Messmethodik

11.5.1 Erste Messreihe

Jeder Lauf schreibt die CNF in eine temporäre Datei und startet den Solver mit `subprocess.run` und einem Zeitlimit von 30 s. Gemessen wird die Wallclock-Zeit zwischen zwei Aufrufen von `time.time()` vor dem Start und nach dem Ende des Prozesses, in Millisekunden. Sie umfasst Prozessstart, Einlesen der Datei, Lösen und Prozessende.

Als erfüllbar gilt eine Instanz, wenn `SAT` in Standard- oder Fehlerausgabe vorkommt und `UNSAT` nicht. Der Speicherverbrauch wird nicht erfasst (das Feld `memory_usage_mb` ist stets 0). Je Lauf werden Circuit-Typ, Name, Zahl der Eingänge, Gates und Klauseln, Erfüllbarkeit, Laufzeit, Zeitstempel, LBV-Runden und Metadaten als JSON und CSV gespeichert. Es gibt 1 bis 12 Wiederholungen je Instanz, ohne Aufwärmphase.

11.5.2 Neue Version 2.0.0

Die neue Version startet jeden Solver als eigenen Prozess (`subprocess.Popen`, Standard- und Fehlerausgabe in temporäre Dateien). Die Laufzeit t ist die Zeit zwischen zwei Aufrufen von `time.perf_counter_ns()` in Millisekunden; sie enthält weiterhin Prozessstart, Einlesen, Lösen und Prozessende. Beim Warten auf das Prozessende liest das Framework über `os.wait4` die Spitzen-Speichernutzung (`ru_maxrss`). Ein Timer-Thread beendet den Prozess nach Ablauf des Zeitlimits (Status `TIMEOUT`). Der Status folgt der Reihenfolge aus Abschnitt 11.2.

Für jede Instanz und jeden Solver gilt der Ablauf in Abbildung 11.2. Zuerst laufen 3 Warm-up-Läufe, deren Ergebnisse verworfen werden. Die Zahl der gemessenen Wiederholungen ist

$$n \;=\; \max\Big(3,\; \min\big(30,\; \lfloor 60\,000 / t_w \rfloor\big)\Big),$$

wobei t_w die Laufzeit des letzten Warm-up-Laufs in Millisekunden ist. Der Zeitbudget-Term begrenzt den Aufwand je Instanz auf etwa 60 s: Schnelle Instanzen werden 30-mal gemessen, langsame mindestens dreimal. Endet schon ein Warm-up-Lauf mit `TIMEOUT`, `ERROR` oder `NO_OUTPUT`, wird die Instanz nur noch einmal gemessen ($n = 1$). Berichtet wird der *Median* der n Läufe; Läufe mit `TIMEOUT` gehen nicht in den Median der übrigen ein. Bei $n = 1$ ist der angegebene Wert die Dauer dieses einen Laufs. Ein Wert bei `TIMEOUT` oder `ERROR` ist deshalb keine Median-Schätzung, sondern ein einzelner Messwert und bei `ERROR` die Zeit bis zum Fehler, keine Lösungszeit.

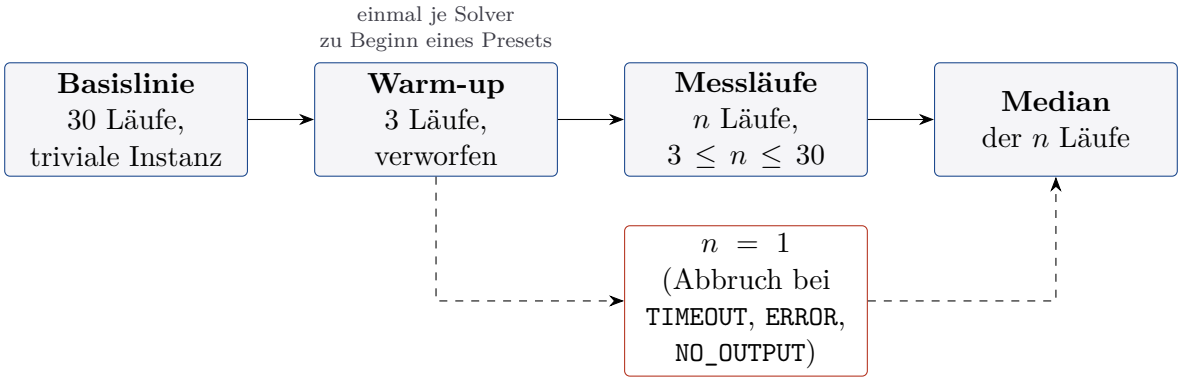


Abbildung 11.2: Messablauf der neuen Version je Instanz und Solver.

Die *Basislinie* wird je Solver zu Beginn eines Presets gemessen: 30 Läufe mit der trivialen Instanz `p cnf 2 2` mit den Klauseln $(x_1 \vee x_2)$ und $(\neg x_1 \vee x_2)$. Ihr Median ist die Prozessstart-Grundlast, die in allen Laufzeiten enthalten ist.

Aspekt	Erste Messreihe	Neue Version 2.0.0
Zeitmessung	`time.time()`, Auflösung praktisch 15,625 ms	`time.perf_counter_ns`, Auflösung 1 ns
Ergebnis	nicht erfasst (`satisfiable` stets `False`)	Status je Lauf, Rückgabecode, Modellprüfung
Kodierung	NAND, NOR, XOR fehlerhaft; BUFF ohne Klauseln	alle Gate-Typen gegen Wahrheitstabellen getestet
Ausgangsbedingung	keine, Formeln stets erfüllbar	Unit-Klausel auf den ersten Ausgang
Unerfüllbare Instanzen	keine	neun Miter
Größte Instanz	3 512 Gates	Skalierung bis $m = 50\,000$ vorgesehen
Wiederholungen	1 bis 12, ohne Warm-up	bis 30, 3 Warm-up-Läufe, Median
Zeitlimit	30 s	60 s
Basislinie	keine	Prozessstart-Grundlast ($n = 30$)
Speicher	nicht gemessen	Spitzen-RSS je Lauf (`wait4`)
Umgebung	nicht dokumentiert	dokumentiert (Abschnitt 11.6)
Vergleichssolver	keiner	Python-CDCL-Referenzsolver

Tabelle 11.5: Unterschiede zwischen erster Messreihe und neuer Version.

11.5.3 Vergleich der beiden Aufbauten

11.5.4 Die beiden Solver

Subgraph-SAT-Solver. Das in Kapitel 10 beschriebene Kommandozeilenprogramm, in der neuen Reihe mit `g++ -O2` gebaut. Seine Laufzeit enthält außer der Reduktion auch Datei-I/O und den Start des externen Subgraph-Prozesses; die Phasenzeiten des Solvers (Reduktion, Subgraph-Aufruf, Extraktion) sind eine andere Messung als die Wallclock-Zeit des Frameworks. Die Schranke $\Theta(n^3)$ bezieht sich auf den Subgraph Algorithmus allein und beschreibt weder automatisch die Kosten der gesamten Implementierung noch belegt sie deren Korrektheit.

Referenzsolver. Ein kleiner CDCL-Solver in reinem Python (Zwei-Watched-Literals-Propagation, 1-UIP-Lernen mit nicht-chronologischem Backjumping, VSIDS, Phase-Saving, Luby-Restarts). Er ist ausdrücklich nicht auf Geschwindigkeit ausgelegt und mit MiniSat oder CaDiCaL nicht konkurrenzfähig. Er spricht dasselbe Aufrufprotokoll wie ein externer Solver und dient (i) als unabhängige Prüfung der Ergebnisse und (ii) als grober Vergleichsmaßstab. Weil er als Python-Programm startet, enthält jede seiner Laufzeiten die Startzeit des Interpreters. *Messwerte dieses Solvers sagen nichts über die Leistung etablierter SAT-Solver aus.*

11.5.5 Experiment-Suites der ersten Reihe

Die vier Suites unterscheiden sich in Umfang und Wiederholungszahl (Tabelle 11.6). Quick und Standard enthalten jeden Circuit einmal, Full und Performance fünfmal (Target-AND und Target-AND-Random) beziehungsweise einmal (ISCAS'85). Full und Performance sind in Umfang und Zusammensetzung identisch. Insgesamt entstehen 288 Läufe: 108

Target-AND, 144 Target-AND-Random und 36 ISCAS'85.

Suite	T-AND	T-AND-R	ISCAS	Läufe	Mittelwert $\bar{t}$ (ms)
Quick	9	12	0	21	24,52
Standard	9	12	12	33	22,46
Full	45	60	12	117	21,68
Performance	45	60	12	117	21,27
Summe	108	144	36	288	21,81

Tabelle 11.6: Die vier Suites der ersten Messreihe. T-AND: Target-AND, T-AND-R: Target-AND-Random.

Die neue Version arbeitet mit dem Full-Preset (Tabelle 11.7).

Parameter	Wert
Wiederholungen (Obergrenze)	30
Warm-up-Läufe	3
Mindestzahl Wiederholungen	3
Zeitlimit je Lauf, Zeitbudget je Instanz	60 s
Instanzen	46 (Target-AND 9, Random 12, ISCAS 12, Miter 9, Skalierung 4)
Solver	Subgraph-SAT-Solver, Referenzsolver

Tabelle 11.7: Konfiguration des Full-Presets der neuen Version.

11.5.6 Statistische Auswertung

Alle Kennzahlen werden aus den CSV-Dateien berechnet: Mittelwert $\bar{t}$, Standardabweichung s (mit $n-1$), Median, Quartile, Minimum und Maximum. Zusammenhänge werden mit der Rangkorrelation nach Spearman (ρ), der Pearson-Korrelation (r) und einer linearen Regression geprüft. Da die Laufzeiten deutlich nicht normalverteilt sind, verwendet die Arbeit für Gruppenvergleiche rangbasierte Tests: Kruskal–Wallis für mehr als zwei Gruppen und Mann–Whitney-U für zwei Gruppen, jeweils zum Niveau $\alpha = 0{,}05$.

11.6 Die Umgebung der neuen Messreihe

Alle Messungen der neuen Version liefen am 11. Oktober 2026 auf einer virtuellen Maschine in der Cloud (Datei `src/results/hardware.md`). Die Umgebung der ersten Reihe wurde nicht dokumentiert.

Vier Eigenschaften dieser Umgebung sind für die Deutung wichtig.

– **Ein einziger Kern.** Solver-Prozess und Messprozess (mit dem Timer-Thread) teilen sich die eine CPU. Das kann sehr kurze Läufe im Bereich weniger Millisekunden leicht verzerren.

Merkmal	Wert
Prozessor	Intel Xeon @ 2,80 GHz, x86_64, **1 Socket, 1 Kern, 1 Thread (eine CPU)**
Cache	L2 1 MiB, L3 33 MiB
Arbeitsspeicher	3,9 GiB, kein Swap
Betriebssystem	Linux, Kernel 6.18.44 (glibc 2.39)
Python	CPython 3.12.3; NumPy 2.4.4, Matplotlib 3.10.8
Compiler	`g++` (C++17, `-O2 -Wall -Wextra`)
Zeitgeber	`time.perf_counter_ns` (monoton, 1 ns)

Tabelle 11.8: Hardware- und Softwareumgebung der neuen Messreihe.

– **Wenig Speicher, kein Swap.** Die Adjazenzmatrix der Reduktion wächst quadratisch mit der Zahl der Literalvorkommen. Bei sehr großen Instanzen kann der Speicher knapp werden; ohne Swap kann eine Überschreitung den Prozess beenden. Ob das die Ursache der `ERROR`-Läufe und des Abbruchs war, wurde nicht geprüft.

– **Build ohne `-march=native`.** Die Binaries wurden mit `g++ -O2` gebaut; die CMake-Konfiguration des Solvers setzt dagegen `-O3 -march=native`. Die Laufzeiten sind mit einem CMake-Build nur eingeschränkt vergleichbar.

– **Virtuelle Maschine.** Taktrate und Cache sind die Werte, die die VM meldet; die zugeteilte Rechenzeit kann schwanken.

11.7 Ergebnisse der ersten Messreihe

Über alle 288 Läufe liegt die Laufzeit zwischen 6,02 und 40,27 ms; Mittelwert 21,81 ms, Standardabweichung 7,25 ms, Median 19,65 ms. Kein Lauf endete mit einem Fehler oder einem Timeout. Die Instanzen überdecken von 4 bis 3 512 Gates (Faktor 878) und von 14 bis 10 690 Klauseln (Faktor 764).

11.7.1 Laufzeit über der Instanzgröße (F1 bis F3)

Target-AND (F1). Gates und Klauseln wachsen nach Satz 11.1 linear von 7 auf 39 Gates. Die mittlere Laufzeit je Eingangszahl liegt zwischen 17,49 ms ($m = 4$) und 24,29 ms ($m = 20$) ohne erkennbaren Trend; sie springt bei $m = 6$ auf 24,23 ms und erreicht bei $m = 10$ ein Nebenminimum von 19,09 ms. Die Standardabweichung innerhalb eines m-Wertes (6,02 bis 8,56 ms) ist größer als der Unterschied der Mittelwerte. Die Rangkorrelation zwischen Gate-Anzahl und Laufzeit beträgt $\rho = 0{,}106$ ($p = 0{,}275$, $n = 108$).

Target-AND-Random (F2). Bei $m = 10$ wächst der Circuit mit der Tiefe von 27 Gates und 91 Klauseln ($d = 1$) auf 54 Gates und 206 Klauseln ($d = 12$). Die mittlere Laufzeit schwankt unsystematisch zwischen 18,79 ms ($d = 8$) und 26,57 ms ($d = 5$) und ist bei $d = 12$ (20,56 ms) kleiner als bei $d = 1$ (23,08 ms). Es gilt $\rho = -0{,}086$ ($p = 0{,}306$, $n = 144$); die Regression ergibt $-0{,}28$ ms je Schicht bei $R^2 = 0{,}018$.

ISCAS'85 (F3). Je Circuit liegen drei Läufe vor (Standard, Full, Performance). Die mittleren Laufzeiten liegen zwischen 15,79 ms (c3540) und 31,85 ms (c1355). Der größte Circuit c7552 (3 512 Gates, 10 690 Klauseln) wird im Mittel in 21,26 ms bearbeitet, nicht langsamer als der ISCAS-Mittelwert von 20,23 ms. Über alle 36 Läufe gilt $\rho = 0{,}104$ ($p = 0{,}547$). c1355 ist der einzige Circuit mit in allen drei Wiederholungen erhöhter Zeit (31,69 bis 32,00 ms); ob das eine Eigenschaft der Instanz oder Zufall ist, lässt sich mit drei Läufen nicht entscheiden.

Gesamtzusammenhang. Über alle 288 Läufe beträgt $\rho = 0{,}024$ ($p = 0{,}686$); die Regression ergibt $-0{,}6\,\mu$s je Gate bei $R^2 = 0{,}002$. In keiner Teilmenge ist ein Zusammenhang signifikant ($p > 0{,}1$), die Bestimmtheitsmaße liegen unter 2,5 %.

11.7.2 Vergleich der Typen und Suites (F4)

Die Typen unterscheiden sich nicht signifikant (Kruskal–Wallis $H = 2{,}63$, $p = 0{,}268$), ebenso wenig ISCAS'85 von den synthetischen Läufen (Mann–Whitney $p = 0{,}140$) und die vier Suites untereinander ($H = 2{,}39$, $p = 0{,}495$). Die gleich aufgebauten Suites Full und Performance weichen im Mittel um +2,8 %, −5,6 % und +0,7 % ab (Target-AND, Random, ISCAS), nicht signifikant ($p = 0{,}513$; 0,483; 0,751). Die Messungen sind in diesem Sinn reproduzierbar, doch das gilt auch für jede Messgröße, die nur Rauschen enthält.

11.7.3 Zeitliche Auflösung

Fast die Hälfte der Läufe liegt in zwei schmalen Klassen: 76 Läufe (26,4 %) in $[15; 16)$ ms und 60 Läufe (20,8 %) in $[31; 32)$ ms, zusammen 136 von 288 (47,2 %). Insgesamt liegen 173 Werte (60,1 %) innerhalb von ± 1 ms um 15,625 ms oder 31,25 ms; bei Gleichverteilung über 6 bis 40 ms wären nur etwa 12 % zu erwarten. 15,625 ms = 1/64 s ist die typische Aktualisierungsrate der Systemuhr unter Windows; die Häufung ist mit einer groben Auflösung von `time.time()` konsistent. Die Arbeit behandelt das als gut gestützte Hypothese, nicht als bewiesen, denn der Datensatz enthält keine Angaben zum Betriebssystem.

Auch die Form der Verteilung stützt die Hypothese: Der kleinste ISCAS-Circuit c3 (4 Gates) und der Circuit c3540 (1 669 Gates) liegen beide bei etwa 15,8 ms, obwohl ein Faktor 417 in der Gate-Anzahl zwischen ihnen liegt. Die Mittelwerte und Standardabweichungen beschreiben eine zweigipflige Verteilung schlecht; $\bar{t} \pm s$ ist hier kein Bereich, in dem die Läufe typischerweise liegen.

11.7.4 Solver-Ausgabe und Erfüllbarkeit

In keinem der 288 Läufe wurde ein Erfüllbarkeitsergebnis erfasst: `satisfiable` ist durchgehend `False`, `solver_output` stets leer, und es wurde kein Fehler protokolliert. Der Prozess startete also und endete ohne erkannte Ausgabe, ohne dass der Rückgabecode ausgewertet worden wäre. Da die Formeln erfüllbar waren (Abschnitt 11.4), hätte ein korrekt arbeitender Solver `SAT` melden müssen. Dieser Widerspruch ist der gewichtigste Mangel der ersten Reihe; er ist mit dem Befund B2 verwandt (Kapitel 15), nach dem der Exit-Code des Solvers nicht eindeutig ist und eine Auswertung allein der Ausgabe oder des Codes in die Irre führt.

11.7.5 Was die erste Messreihe nicht stützt

Eine Unabhängigkeit der Laufzeit von der Größe ist nur dann eine Aussage über den Algorithmus, wenn die Zeit die Rechenzeit ist. Das ist aus vier Gründen nicht gesichert: Der Messwert wird vom Prozessstart dominiert; die Zeitauflösung ist grob; es wurde keine Solver-Ausgabe erfasst; und die Kodierung ist für NAND, NOR und XOR fehlerhaft, die ISCAS-Läufe messen also Formeln, die nicht die ISCAS-Circuits darstellen. Die erste Reihe kann die Schranke $\Theta(n^3)$ daher weder bestätigen noch widerlegen. Die Aussage „konstante Laufzeit“ ist *nicht* ableitbar: Konstanz im Rauschen einer vom Prozessstart bestimmten Messung ist keine Konstanz des Algorithmus. Als Befund bleibt eine obere Schranke: Der Prozess endete für alle Instanzen ohne Timeout in unter 41 ms.

11.8 Ergebnisse der neuen Version

11.8.1 Datenbasis

Der Lauf des Full-Presets wurde nicht beendet. Die Konsolenausgabe endet bei der Zeile für den Subgraph-SAT-Solver auf `TargetAND_m10000`, ohne Ergebnis; ob der Speicher von 3,9 GiB der Grund war, wurde nicht geprüft. Daraus folgt:

- Ausgewertet sind 44 von 46 Instanzen, je für beide Solver; es fehlen `TargetAND_m10000` und `TargetAND_m50000`.
- Die Dateien, die erst am Ende eines Laufs geschrieben werden (Ergebnis-CSV und -JSON, Zusammenfassung, Umgebung, Basislinie, Konfiguration, Diagramme), liegen nicht vor. Datenbasis ist eine Zwischenstand-CSV mit den Spalten `solver`, `instanz`, `gates`, `status`, `n`, `median_ms` und `anmerkung`; ihre 88 Zeilen stimmen mit den 88 Ergebniszeilen der Konsolenausgabe überein.
- Je Instanz und Solver steht *nur der Median* und die Zahl n der Läufe darin. Streumaße, Rückgabecodes, Ausgaben, Speicherverbrauch und das Ergebnis der Modellprüfung je Lauf sind nicht verfügbar.
- In die Mediane gingen 933 Läufe des Subgraph-SAT-Solvers und 1 257 des Referenzsolvers ein (Summe der n ohne Warm-up).

11.8.2 Basislinie

Die Basislinie ($n = 30$) beträgt für den Subgraph-SAT-Solver 8,43 ms ($s = 1{,}80$ ms), für den Referenzsolver 56,18 ms ($s = 14{,}63$ ms), das 6,7-Fache wegen des Interpreterstarts. Beide Werte sind in allen Laufzeiten enthalten. Differenzen unterhalb weniger Millisekunden (Subgraph-SAT-Solver) und unterhalb von etwa 15 ms (Referenzsolver) sind von der Grundlast nicht zu unterscheiden. Deshalb liegen zwei Mediane des Subgraph-SAT-Solvers (c3: 6,98 ms, c17: 7,24 ms) und 30 von 44 des Referenzsolvers unter der jeweiligen Basislinie: Sie ist selbst ein Median mit Streuung und wurde einmal gemessen.

11.8.3 Gesamtüberblick

Der Referenzsolver erreicht bei allen 44 Instanzen ein Ergebnis. Der Subgraph-SAT-Solver erreicht bei 33 ein Ergebnis, bei 5 ein Zeitlimit und bei 6 den Status `ERROR`. Tabelle 11.9 gibt die Verteilung je Familie wieder.

Familie	Instanzen	Subgraph-SAT-Solver	Referenzsolver
Target-AND	9	9× SAT	9× SAT
Target-AND-Random	12	3× SAT, 9× UNSAT	3× SAT, 9× UNSAT
ISCAS'85	12	8× SAT, 4× ERROR	12× SAT
Miter	9	3× UNSAT, 5× Timeout, 1× ERROR	9× UNSAT
Skalierung	2	1× SAT, 1× ERROR	2× SAT
Summe	44	33 Ergebnisse, 5 Timeouts, 6 ERROR	44 Ergebnisse

Tabelle 11.9: Status je Instanzfamilie in der neuen Version (Zwischenstand, 44 Instanzen).

Bei den 33 Instanzen, auf denen beide Solver entscheiden, stimmen die Ergebnisse in allen Fällen überein. Bei den 13 Instanzen mit aus der Konstruktion bekanntem Ergebnis trifft der Subgraph-SAT-Solver die erwartete Antwort, der Referenzsolver tut es bei allen 20 solchen Instanzen. Der Subgraph-SAT-Solver verfehlt zwei bekannte Antworten, `MiterSelf_c1908` und `TargetAND_m1000`, jeweils mit `ERROR`; ein `ERROR` ist keine falsche Antwort, aber auch kein Ergebnis. Für keine ausgewertete Instanz wurde ein ungültiges Modell gemeldet; da die Modellprüfung nur für den ersten Lauf je Instanz und nur bei ausgegebenem Modell erfolgt und die Einzelergebnisse fehlen, ist das nicht nachprüfbar.

11.8.4 Target-AND und Skalierung

m	Gates	Klauseln	Literale	$\tilde{t}_{\mathrm{SG}}$ (ms)	$\tilde{t}_{\mathrm{Ref}}$ (ms)
4	7	18	38	9,52	—
6	11	28	60	8,53	—
8	15	38	82	10,11	—
10	19	48	104	13,01	65,36
16	31	78	170	19,02	41,02
20	39	98	214	17,88	—
100	199	498	1 094	321,23	53,56
1000	1 999	4 998	10 994	ERROR (25 090)	102,17

Tabelle 11.10: Target-AND und Skalierung, Auswahl der Werte der Arbeit; „—“ bedeutet, dass der Wert hier nicht wiedergegeben ist. Klauseln und Literale folgen Satz 11.1.

Für $m = 4$ bis 20 steigt der Median des Subgraph-SAT-Solvers nicht streng monoton, aber erkennbar von 9,52 auf 17,88 ms (Minimum 8,53 ms bei $m = 6$, Maximum 19,02 ms bei $m = 16$). Die Regression über die neun Werte ergibt 0,66 ms je zusätzlichem Eingang bei $R^2 = 0{,}836$, $\rho = 0{,}883$. Die Instanz wächst dabei um den Faktor 5,6 (38 → 214 Literale), der Median nur um 1,9; für $m \leq 8$ liegen die Mediane höchstens 1,7 ms über der Basislinie. Der Referenzsolver zeigt keinen Anstieg (zwischen 41,02 und 65,36 ms um die Basislinie, $\rho = -0{,}417$).

Bei $m = 100$ steigt der Median auf 321,23 ms, das 6,0-Fache des Referenzsolvers (53,56 ms). Von $m = 20$ auf $m = 100$ wächst die Laufzeit um den Faktor 18,0 bei ei-

nem Literalfaktor von 5,1; nach Abzug der Basislinie ergibt sich der Faktor 33,1 und ein Exponent von etwa 2,1, aus nur zwei Punkten. Bei $m = 1000$ scheitert der Subgraph-SAT-Solver nach 25,09 s mit `ERROR`, obwohl die Instanz nach Konstruktion erfüllbar ist und der Referenzsolver sie in 102,17 ms löst.

11.8.5 Target-AND-Random

Der Median des Subgraph-SAT-Solvers steigt mit der Tiefe nahezu monoton von 14,59 ms ($d = 1$) auf 60,04 ms ($d = 12$), um den Faktor 4,1 bei einer Verdopplung der Gates ($27 \to 54$) und einem Literalfaktor von 2,35. Es gibt eine Abweichung ($d = 9$: 37,32 ms unter $d = 8$: 39,13 ms). Es gilt $\rho = 0{,}993$, die Regression ergibt 3,73 ms je Schicht bei $R^2 = 0{,}953$. Über der Literalzahl (174 bis 409) liegen die Punkte eng an einer Geraden mit 0,174 ms je Literal ($R^2 = 0{,}936$); der Achsenabschnitt ($-19{,}3$ ms) ist negativ, die Gerade beschreibt daher nur diesen Messbereich. Erfüllbare ($d = 1, 7, 11$) und unerfüllbare Instanzen liegen auf derselben Geraden; die Selbst-Miter von Target-AND (19,97 und 53,58 ms bei 219 und 439 Literalen) liegen nahe an dem, was die Gerade ergibt (18,7 und 57,0 ms).

Der Referenzsolver schwankt dagegen unsystematisch zwischen 42,28 ($d = 7$) und 53,58 ms ($d = 11$) ($R^2 = 0{,}145$). Die Kurven kreuzen sich zwischen $d = 11$ (47,68 gegen 53,58 ms) und $d = 12$ (60,04 gegen 44,42 ms).

11.8.6 ISCAS'85

Tabelle 11.11 gibt alle zwölf Circuits wieder, Abbildung 11.3 zeigt die Mediane beider Solver auf logarithmischer Skala.

Circuit	Gates	Klauseln	Literale	n_{SG}	$\tilde{t}_{\mathrm{SG}}$ (ms)	$\tilde{t}_{\mathrm{Ref}}$ (ms)
c3	4	12	26	30	6,98	45,19
c17	6	19	43	30	7,24	47,31
c432	160	515	1 259	30	515,75	71,23
c499	202	715	1 947	30	1 094,34	51,71
c880	383	1 113	2 571	30	2 167,29	47,61
c1355	546	1 611	3 739	13	3 945,39	48,84
c1908	880	2 379	5 375	6	9 931,22	52,76
c2670	1 193	3 270	7 422	3	20 427,66	70,09
c3540	1 669	4 609	10 487	1	ERROR (26 358)	72,42
c5315	2 307	6 694	15 466	1	ERROR (29 415)	67,58
c6288	2 416	7 217	16 817	1	ERROR (24 766)	77,80
c7552	3 512	9 657	21 945	1	ERROR (40 737)	89,88

Tabelle 11.11: ISCAS'85 in der neuen Version. Alle Instanzen sind erfüllbar (Referenzsolver: SAT). n_{SG} ist die Zahl der Messläufe des Subgraph-SAT-Solvers; bei ERROR steht die Zeit bis zum Fehler in Klammern.

Die Mediane des Subgraph-SAT-Solvers wachsen von c432 (515,75 ms) bis c2670 (20 427,66 ms) durchgehend monoton über rund eineinhalb Größenordnungen. Bei einem Faktor 7,5 in der Gate-Anzahl (160 bis 1 193) steigt die Laufzeit um den Faktor 39,6. Wegen des Zeitbudgets sinkt die Zahl der Läufe mit der Laufzeit: $n = 30$ bis c880, 13 bei

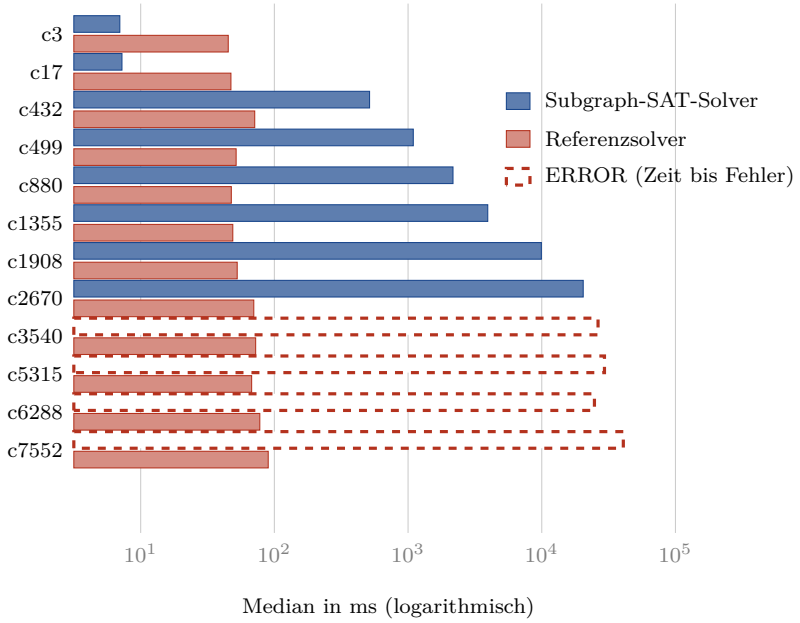


Abbildung 11.3: Mediane der zwölf ISCAS'85-Circuits (Werte aus Tabelle 11.11), logarithmische Achse. Die Balkenlänge ist proportional zu $\log_{10} t$ ab $t = 10^{0,5}$ ms.

c1355, 6 bei c1908 und 3 bei c2670; die größten Mediane stützen sich auf wenige Läufe. Die vier Circuits ab 1 669 Gates enden mit `ERROR` nach 24,77 bis 40,74 s (ein Lauf, da bereits der Warm-up fehlschlug); diese Zeiten sind keine Lösungszeiten. Der Referenzsolver bleibt flach zwischen 45,19 und 89,88 ms (Median 60,17 ms), nahe der Basislinie.

Potenzgesetz. Eine Anpassung $t \propto L^b$ über der Literalzahl L der sechs gelösten Circuits c432 bis c2670 (1 259 bis 7 422 Literale) ergibt

$$b = 2{,}08 \pm 0{,}07 \text{ (Standardfehler)}, \qquad R^2 = 0{,}996, \qquad 95\,\%\text{-Intervall } [1{,}89;\ 2{,}27].$$

Der Exponent liegt nahe 2. Das ist mit der Struktur der Reduktion vereinbar, in der die Adjazenzmatrix des Clique-Graphen durch paarweise Vergleiche der Literalvorkommen entsteht und $O(L^2)$ Schritte benötigt; bewiesen ist dieser Zusammenhang durch die Messung nicht. Der Fit stützt sich auf sechs Punkte aus verschiedenen Circuits und auf Mediane aus 13, 6 und 3 Läufen; er gilt für die Literalzahl der CNF, nicht für die Knotenzahl des Clique-Graphen, und er trennt die Reduktion nicht von der Rechenzeit des externen Subgraph-Prozesses. Er bestätigt oder widerlegt die Schranke $\Theta(n^3)$ des Subgraph Algorithmus nicht (Abschnitt 11.10).

Der Ausfall ab 10 487 Literalen. Alle ISCAS-Circuits bis 7 422 Literale werden gelöst, alle ab 10 487 enden mit `ERROR`. Dieselbe Trennung zeigt der gesamte Datensatz: Alle sechs `ERROR`-Instanzen haben mindestens 10 487 Literale (10 487, 10 994, 11 125, 15 466, 16 817, 21 945), alle Instanzen mit höchstens 7 958 Literalen enden mit einem Ergebnis oder einem Zeitlimit. Auch `TargetAND_m1000`, ein einfacher AND-Baum, fällt aus. Der Ausfall hängt an der Instanzgröße, nicht an der logischen Schwierigkeit. Die Fehlerzeiten liegen mit 24,77 bis 40,74 s deutlich unter dem Zeitlimit von 60 s. Die Ursache wurde nicht untersucht; zu prüfen sind der Speicherbedarf der Reduktion und des externen Subgraph-Prozesses auf der Maschine mit 3,9 GiB ohne Swap, die Größe der JSON-Dateien mit den Kantenlisten und die interne Fehlerbehandlung.

11.8.7 Miter-Instanzen

Instanz	Gates	Literale	$\tilde{t}_{\mathrm{SG}}$ (ms)	Status (SG)	$\tilde{t}_{\mathrm{Ref}}$ (ms)
MiterSelf_c17	15	116	9,86	UNSAT	46,92
MiterSelf_TargetAND_m10	39	219	19,97	UNSAT	47,66
MiterSelf_TargetAND_m20	79	439	53,58	UNSAT	44,95
MiterSelf_c432	328	2 623	60 041	Timeout	279,73
MiterSelf_c499	437	4 374	60 069	Timeout	1 853,19
Miter_c499_c1355	781	6 166	60 180	Timeout	1 723,69
MiterSelf_c880	793	5 532	60 114	Timeout	3 293,14
MiterSelf_c1355	1 125	7 958	60 274	Timeout	3 139,56
MiterSelf_c1908	1 786	11 125	25 560	ERROR	3 374,42

Tabelle 11.12: Die neun unerfüllbaren Miter-Instanzen. Der Referenzsolver löst alle korrekt als UNSAT.

Der Subgraph-SAT-Solver löst nur die drei kleinsten, jeweils korrekt als `UNSAT`; fünf Instanzen laufen in das Zeitlimit, `MiterSelf_c1908` endet mit `ERROR` (zu erwarten bei 11 125 Literalen). Das einzige `UNSAT`, das die Reduktionskette (Korollar 2.1) auf größeren Formeln hätte liefern müssen, bleibt damit unbelegt. Die Mediane des Referenzsolvers sind nicht streng nach Gate-Anzahl geordnet (c880: 3,29 s über c1355: 3,14 s) und stützen sich auf unterschiedlich viele Läufe (7 bis 30).

Bei vergleichbarer Literalzahl ist der Unterschied zwischen erfüllbaren und unerfüllbaren Instanzen groß: `MiterSelf_c432` (2 623 Literale) erreicht das Zeitlimit, während c880 (2 571 Literale) nach 2,17 s gelöst wird, mindestens 27,7-mal schneller. Aus den Daten lässt sich nicht entscheiden, ob das an der Unerfüllbarkeit liegt oder an der Struktur der Miter-Formeln, denn beides ändert sich gleichzeitig; die Miter von c499, c880, c1355 und c1908 enthalten Klauseln mit 26 bis 33 Literalen, die ISCAS-Formeln höchstens 10.

11.8.8 Vergleich der beiden Solver

Bei den 24 Instanzen mit höchstens 392 Literalen ist der Subgraph-SAT-Solver jeweils schneller (Quotient $\tilde{t}_{\mathrm{SG}}/\tilde{t}_{\mathrm{Ref}}$ zwischen 0,15 bei c3 und c17 und 0,94 bei $d = 10$). Ab 409 Literalen ist er in allen Fällen langsamer: 1,35 bei $d = 12$, 1,19 bei `MiterSelf_TargetAND_m20`, 6,0 bei $m = 100$, 7,2 bei c432, 21,2 bei c499, 45,5 bei c880, 80,8 bei c1355, 188,2 bei c1908 und 291,5 bei c2670, dem größten Quotienten einer gelösten Instanz. Die Timeouts entsprechen Quotienten von mindestens 18,3 bis mindestens 214,6. Die Kreuzung bei rund 400 Literalen ist vor allem ein Effekt der Startkosten (8,43 gegen 56,18 ms) und hängt an der Wahl eines Python-Programms als Referenz. Sie besagt nicht, dass der Subgraph-SAT-Solver für kleine Formeln den besseren Algorithmus hat, und sie sagt nichts über MiniSat oder CaDiCaL.

11.9 Vergleich der beiden Reihen

Die erste Reihe hatte vermutet, die konstanten Laufzeiten seien ein Messartefakt aus Prozessstart und Zeitauflösung. Die neue Version stützt das in drei Punkten: Mit Nanosekundenauflösung liegen die Mediane nicht mehr auf den Zeittakten bei 15,625 und 31,25 ms

(die Einzelläufe fehlen im Zwischenstand, die Aussage betrifft nur Mediane); die Basislinie von 8,43 ms liegt in der Größenordnung der kleinsten Messwerte, und erst darüber steigen die Mediane; und bei größeren Instanzen wächst die Laufzeit des Subgraph-SAT-Solvers deutlich. Die in der ersten Reihe beobachtete Unabhängigkeit von der Instanzgröße war demnach keine Eigenschaft des Solvers. Außerdem liefert der Subgraph-SAT-Solver bei kleinen und mittleren Instanzen ein Ergebnis, das mit dem Referenzsolver übereinstimmt; die erste Reihe hatte keines erfasst. Ein absoluter Vergleich der Zeiten ist wegen der anderen Formeln und der undokumentierten Umgebung nicht möglich.

11.10 Was die Messungen für diese Arbeit bedeuten

Dieser Abschnitt trennt, was die Experimente stützen, von dem, was sie nicht stützen, und verbindet sie mit den Befunden der Arbeit.

Was die neuen Daten stützen.

- Die Messung enthält Skalierungsinformation: Der Median wächst bei Target-AND-Random um den Faktor 4,1 zwischen 27 und 54 Gates, bei ISCAS'85 um mehr als drei Größenordnungen zwischen c17 und c2670. Laufzeiten ab etwa 20 ms werden von der Basislinie nicht mehr dominiert.

- Im gemessenen Bereich wächst die Zeit näherungsweise quadratisch in der Literalzahl ($2{,}08 \pm 0{,}07$).

- Soweit Ergebnisse vorliegen, sind sie korrekt: Alle 33 gemeinsam entschiedenen Instanzen stimmen überein, alle bekannten Antworten werden getroffen.

- Bei kleinen Instanzen ist kein Unterschied zwischen SAT und UNSAT erkennbar.

Was sie nicht stützen.

- *Keine Aussage über die Schranke* $\Theta(n^3)$. Der Exponent 2,08 gilt für die Literalzahl und die ganze Pipeline aus Reduktion, Datei-I/O und externem Prozess, nicht für den Subgraph Algorithmus allein, auf dessen Kern sich die Schranke bezieht (Satz 2.2 und Kapitel 10). Das Framework sieht nur die Außenfläche des Solvers; Laufzeiten des Algorithmus, der Reduktion und des Prozesses sind nicht getrennt.

- *Keine Aussage über etablierte SAT-Solver.* Der Referenzsolver ist ein Python-Programm ohne Anspruch auf Geschwindigkeit.

- *Keine Aussage über die Ursache der* `ERROR`*- und Timeout-Läufe.* Rückgabecode, Fehlermeldung und Speicherverbrauch liegen im Zwischenstand nicht vor.

- *Keine Aussage über die Streuung*: Es gibt nur Mediane. Unterschiede von wenigen Millisekunden, etwa die Nicht-Monotonie bei $d = 8$ und $d = 9$, lassen sich nicht vom Rauschen trennen.

- *Keine Skalierung bis* 10^5 *Gates.* Die größte gelöste Instanz hat 1 193 Gates.

Verbindung zu den Befunden. Die Experimente berühren drei Befunde aus Kapitel 15. Erstens B2: Der Exit-Code 20 des Solvers ist mehrdeutig. Das Framework der neuen Version umgeht das, indem es die Zeile `Status:` vor dem Code auswertet; die erste Reihe zeigt, was geschieht, wenn man weder Code noch Ausgabe richtig liest. Zweitens die Fehlerklassen des Solvers (Kapitel 10): Dass der Subgraph-SAT-Solver ab 10 487 Literalen mit `ERROR` endet, ist von außen als Fehler des Solver-Prozesses sichtbar, die Ursache liegt in der Reduktion oder im Adapter, die das Framework nicht sieht. Drittens: Die Arbeit des Frameworks nennt weder das benutzte Subgraph-Programm noch den Modus (`SUBGRAPH_MODE`, Tabelle 7.2) noch eine Solver-Version. Ein Lauf im Modus `heuristic` misst aber einen anderen Algorithmus als einer im Modus `exact`; ohne diese Angaben ist die Messreihe nicht reproduzierbar im Sinn dieser Arbeit.

Eigene Abschätzung (nicht geprüft). Bei $L = 10\,487$ Literalen gibt es höchstens $L^2/2 \approx 5{,}5 \cdot 10^7$ Literalpaare, also eine Adjazenzmatrix mit bis zu etwa $1{,}1 \cdot 10^8$ Einträgen. Wie viel Speicher das je nach Darstellung und Normalisierung auf drei Literale je Klausel belegt, wurde nicht nachgerechnet; die Rechnung macht lediglich plausibel, dass die Grenze mit dem Speicher der Maschine zusammenhängen *kann*, und ersetzt keine Messung des Spitzenspeichers.

Gültigkeitsbedrohungen.

- *Intern:* ein Kern für Solver und Messprozess; die Basislinie des Referenzsolvers hat $s = 14{,}63\,\text{ms}$, Mediane unter etwa 70 ms sind nicht unterscheidbar; die Basislinie wurde nur einmal gemessen.
- *Konstrukt:* gemessen wird die Wallclock-Zeit eines Prozesses einschließlich Start, nicht die solverinterne Zeit; Nettozeiten sind Differenzen von Medianen.
- *Extern:* Target-AND ist ein einfacher AND-Baum, die Random-Instanzen sind klein, ISCAS'85 ist für moderne SAT-Solver klein; die Ausgangsbedingung betrifft nur den ersten Ausgang.
- *Umgebung:* eine virtuelle Maschine mit einem Kern, 3,9 GiB Speicher ohne Swap und `-O2` statt `-O3 -march=native`.
- *Unvollständiger Lauf:* Die 44 ausgewerteten Instanzen sind vollständig, der Datensatz ist ein Zwischenstand.

11.11 Reproduktion und offene Punkte

Die Auswertung der neuen Version erzeugt das Skript `src/paper_analysis_v2.py` aus den Dateien unter `src/results/`: der Zwischenstand-CSV, der Konsolenausgabe mit den Basislinien und der Hardware-Beschreibung. Es berechnet die Struktur der Instanzen mit den Generatoren neu, prüft sie gegen die gemessenen Gate-Zahlen und schreibt alle im Text genannten Kennzahlen nach `summary_v2.json`. Ein vollständiger neuer Lauf startet mit `python -m src.run_experiments --preset full --solver-path <Pfad> --with-reference`. Der Code und die Rohdaten sind im Repository des Frameworks verfügbar (Tabelle 3.1).

Fünf Punkte sind offen. Erstens muss das Full-Preset vollständig wiederholt werden, mit allen Ergebnisdateien und Rückgabecodes, und es ist zu klären, warum der Lauf bei `TargetAND_m10000` abbrach. Zweitens ist die Ursache der `ERROR`-Läufe ab 10 487 Literalen zu untersuchen (Rückgabecode, Fehlermeldung, Spitzenspeicher) und `TargetAND_m10000` sowie `TargetAND_m50000` auf einer Maschine mit mehr Speicher zu messen. Drittens ist der Solver mit CMake (`-O3 -march=native`) zu bauen und seine Version festzuhalten. Viertens ist der Referenzsolver durch MiniSat oder CaDiCaL zu ergänzen. Fünftens braucht es mehr Messpunkte zwischen 1 000 und 10 000 Literalen (etwa Target-AND mit $m = 200$ bis 800), um den Exponenten an einer Instanzfamilie mit einem Parameter zu prüfen, sowie Instanzen mit gleicher Literalzahl und unterschiedlicher Klauselllänge, um den Einfluss der langen Miter-Klauseln von dem der Unerfüllbarkeit zu trennen. Kapitel 18 greift diese Punkte auf.

Kapitel 12

Vergleich der Kopplungsarten

12.1 Vier Gestaltungsprinzipien

– **Korrektheit:** Jede Kopplung muss die Aussage des Ursprungs bewahren; Abweichungen müssen bekannt und geprüft sein.

– **Isolierung:** Ein Projekt soll nur die Schnittstelle kennen, die es braucht.

– **Leistung:** Die Kopplung darf den Rechenkern nicht überdecken.

– **Erweiterbarkeit:** Neue Funktionen im Kern dürfen bestehende Nutzer nicht stören.

12.2 Die Kopplungsarten nebeneinander

Kopplungsart	Beispiele	Aufrufkosten	Stärke	Schwäche
Bibliothek im Prozess (`ctypes`)	**W7**, **W8**	≈ 0,018 ms	schnell	ABI, Compiler, Bauzwang
Python-Import	**W6**	Funktionsaufruf	einfach	Laufzeit der Python-Fassung
Eigene Neuimplementierung	**W9**, **W10**	Funktionsaufruf	keine Bauabhängigkeit	Pflicht zu Differenzialtests
Prozess mit Dateien	**W5**, **W14**, **W15**	≈ 3,4 ms	klare Trennung, austauschbar	Textprotokoll, Zeitlimit
Prozess mit stdin/stdout	**W4**, **W17**	Prozessstart + JSON	einfach zu testen	Kodierung je Vergleich
Begriffliche Übernahme	**W11**, **W18**	keine Vergleiche	größter Gewinn	nur für diese Relation

Tabelle 12.1: Kopplungsarten. Der Prozessaufruf des Adapters kostet nach eigener Messung 3,41 ms, der In-Prozess-Aufruf 0,0185 ms, also etwa das 184-Fache (Kapitel 14).

12.3 Bewertung

Keine Kopplungsart ist überlegen; entscheidend ist, was die Anwendung braucht. Gen-DB vergleicht in der Größenordnung von 10^5 bis 10^6 Paaren je Suche; dort lohnt die Bibliothekskopplung, und die Kosten von 3,4 ms je Prozessaufruf wären untragbar. Der Solver braucht dagegen einen Aufruf je Formel; 3,4 ms sind klein gegenüber der Suche, und die Prozessgrenze erlaubt den Austausch des Orakels und eine Zeitüberwachung. Der größte Gewinn stammt aber aus einer Wiederverwendung der *Idee*: Der Paarindex (**W11**) ersetzt $\approx 20\,000$ Vergleiche je Anfrage durch einige Listenzugriffe.

Auffällig ist, dass die Stärken sich ausschließen: Schnelligkeit (Bibliothek) und Austauschbarkeit (Prozess) lassen sich nicht in derselben Kopplung haben. Der Solver profitiert davon, dass sein Orakel nur zwei Dateien und einen Exit-Code umfasst; Gen-DB profitiert davon, dass der Wrapper nur `int`-Felder austauscht. In beiden Fällen ist die Schnittstelle sehr schmal, und das ist die eigentliche Voraussetzung für die Wiederverwendung.

Kapitel 13

Laden von C/C++-Bibliotheken: zwei Verfahren und ein einheitliches Konzept

Kapitel 12 hat Kopplungsarten verglichen und *Bibliothek im Prozess* als schnellste gefunden (**W7**, **W8**). Dieses Kapitel betrachtet den Preis dieser Kopplung: Eine C/C++-Bibliothek muss von Python *gefunden*, *gebaut*, *geladen* und *geprüft* werden, und das geschieht auf jedem Betriebssystem anders. Zwei Projekte der Kette lösen dieselbe Aufgabe auf verschiedene Weise: Gen-DB (Kapitel 9) und `github.com/naphez86/subgraph-sat-solver-econ`, das den SAT-Solver als Bibliothek in Python nutzt. Das Kapitel beschreibt beide Verfahren als praktische Anwendung (Abschnitte 13.2 bis 13.4) und stellt danach das Konzept eines *einheitlichen Ladeverhaltens* vor (Abschnitt 13.5), das die Unterschiede an einer Stelle vereinheitlichen soll. Das Konzept ist ein Diskussionsentwurf und in keinem der beiden Projekte umgesetzt.

13.1 Ausgangslage: ein Konzept, drei Namen

Dasselbe Konzept, eine zur Laufzeit ladbare Bibliothek, heißt je nach Betriebssystem `.dll`, `.so` oder `.dylib`. Dahinter stehen drei Binärformate, drei Lader und drei Regeln dafür, welche Funktionen nach außen sichtbar sind (Tabelle 13.1). Wer Software für mehrere Systeme ausliefert, muss alles dreimal bauen, testen und dokumentieren.

13.2 Verfahren 1: Gen-DB baut den Wrapper beim ersten Gebrauch

Gen-DB bekommt von `github.com/naphez86/csubgraph` eine *statische* Bibliothek (`libsubgraphlib.a`). Python kann ein Archiv nicht laden. Gen-DB (`csubgraph_native.py`) macht deshalb aus Archiv und einem kleinen C-Wrapper (`csubgraph_shim.cpp`) *zur Laufzeit* eine gemeinsame Bibliothek und lädt diese mit `ctypes`. Der Weg von der Einstellung bis zum Aufruf hat zehn Schritte; Abbildung 13.1 zeigt den Fluss.

1. **Einstellung.** Die Umgebungsvariable `CSUBGRAPH_LIB_PATH` (aus der `.env`, über Pydantic als `Config.csubgraph_lib_path`) zeigt auf die `.a`-Datei oder auf einen Ordner. Ist sie leer, wird Python benutzt.

Aspekt	Windows	Linux	macOS
Dateiendung	`.dll`	`.so` (oft `.so.1.2.3`)	`.dylib` (für Plug-ins `.so`)
Namenspräfix	keines (MSVC) oder `lib` (MinGW)	`lib`	`lib`
Binärformat	PE/COFF	ELF	Mach-O
Lade-Aufruf	`LoadLibrary`	`dlopen`	`dlopen`
Symbolsuche	`GetProcAddress`	`dlsym`	`dlsym`
Sichtbarkeit (Standard)	nichts exportiert ohne das Attribut `dllexport`	alles sichtbar, außer als `hidden` markiert	alles sichtbar, außer als `hidden` markiert
Suche nach Abhängigkeiten	DLL-Suchpfad, `PATH`, `add_dll_directory`	`RPATH`, `RUNPATH`, `LD_LIBRARY_PATH`	`@rpath`, `@loader_path`, `DYLD_*`
Begleitdatei zum Linken	Importbibliothek `.lib`	die `.so` selbst	die `.dylib` selbst
Versionierung	Manifest, Dateiversion	SONAME, Symbolversionen	`install_name`, Kompatibilitätsversion
Initialisierung	`DllMain`	`constructor`, `.init_array`	`constructor`, `__mod_init_func`
Fehlermeldung	`GetLastError` (Zahl)	`dlerror` (Text)	`dlerror` (Text)
Entladen	`FreeLibrary`	`dlclose`	`dlclose`

Tabelle 13.1: Was beim Laden einer gemeinsamen Bibliothek auf den drei Betriebssystemen verschieden ist.

2. **Auffinden** (`find_static_library`). Eine Datei gilt unmittelbar; bei einem Ordner werden `.`, `build`, `build/Release` und `Release` der Reihe nach nach `libsubgraphlib.a` durchsucht.

3. **Inhalt prüfen** (nur Multi-Omics). Eine Textsuche im Archiv prüft, ob die Klasse `MultiOmics` enthalten ist (`_has_multiomics`). Fehlt sie, bleibt der Einzelvergleich nutzbar, und nur Multi-Omics fällt auf Python zurück.

4. **Header finden.** Die Header liegen neben dem Archiv oder eine Ebene darüber (auch `include/`); sonst nimmt Gen-DB die mitgelieferte Kopie unter `backend/native/`.

5. **Cache-Schlüssel.** Ein SHA-256-Wert über Archiv, Wrapper-Quelle, Header und ABI-Version bestimmt den Dateinamen im Cache-Ordner (`GENDB_NATIVE_CACHE` oder ein Ordner im Temp-Verzeichnis). Existiert die Datei, wird nichts gebaut.

6. **Compiler finden.** In dieser Reihenfolge: `CXX`, `g++`, `c++`, `clang++`. Fehlt jeder, ist die Bibliothek nicht nutzbar.

7. **Bauen.** Ein Aufruf mit Zeitlimit von 300 s (Listing 13.1); das Ergebnis entsteht in einer Temporärdatei.

8. **Atomar umbenennen** (`os.replace`). Parallel startende Worker laden so nie eine halbe Datei; war ein anderer Prozess schneller und hat die Datei unter Windows

schon geladen, wird die eigene Temporärdatei verworfen.

9. **Laden und Version prüfen.** `ctypes.CDLL`, dann `csub_abi_version()`; weicht der Wert von der erwarteten Version ab, ist die Bibliothek nicht nutzbar.

10. **Fehlerbehandlung.** Jeder Fehler in den Schritten 2 bis 9 wird als `NativeLibraryError` gemeldet; der Executor schreibt eine Warnung ins Log und benutzt das Python-Paket (Abschnitt 9.3).

Listing 13.1: Aufruf, mit dem Gen-DB den Wrapper baut (aus `build_shared_library`, gekuerzt; Linux)

```
g++ -std=c++17 -O2 -fPIC -shared -I <header_dir> csubgraph_shim.cpp \
    -Wl,--whole-archive libsubgraphlib.a -Wl,--no-whole-archive \
    [-lgcov] -o <cache>/csubgraph_shim_<hash>.so
# macOS: statt whole-archive: -Wl,-force_load,libsubgraphlib.a
# Windows: ohne -fPIC; mit -static-libgcc -static-libstdc++ -static
```

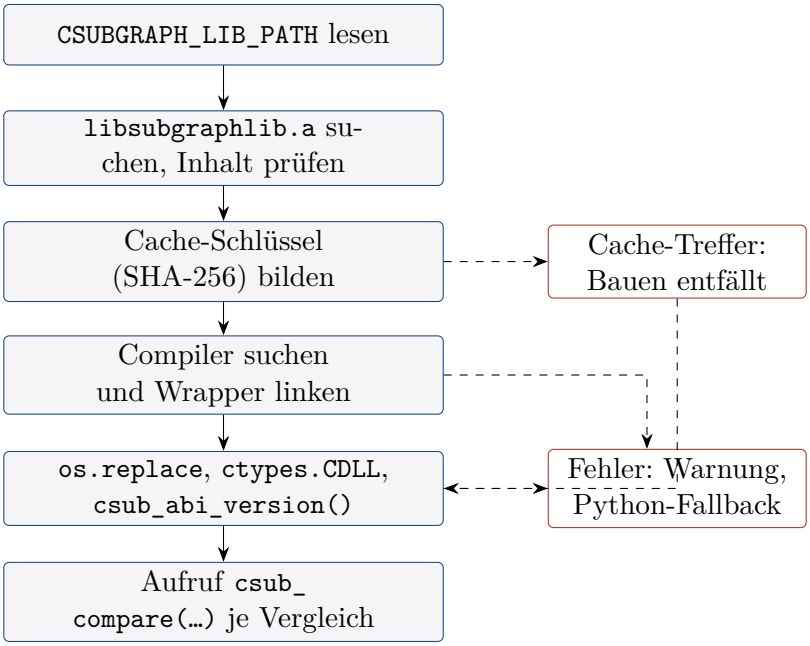


Abbildung 13.1: Ladevorgang in Gen-DB. Durchgezogen: Normalfall beim ersten Gebrauch; gestrichelt: Cache-Treffer und Fehlerweg.

Zwei Eigenschaften sind für den Vergleich wichtig. Erstens wird die Auswahl zwischen C++ und Python je Prozess *einmal* getroffen (`_get_native`, `_native_resolved`), und der Executor löst sie im Hauptprozess vor dem Start der Worker auf, damit nicht alle Worker gleichzeitig den Bau anstoßen (Schritt 6 in Abbildung 9.1). Zweitens enthält der Wrapper selbst Plattformcode: Das Makro `CSUB_EXPORT` ist unter Windows `extern "C" __declspec(dllexport)` und sonst `extern "C" __attribute__((visibility("default")))`. Der Vertrag ist schmal: `int`-Felder, ein Fehlerpuffer von 512 Byte und je eine Versionsfunktion für Einzelvergleich und Multi-Omics.

13.3 Verfahren 2: subgraph-sat-solver-econ lädt eine fertig gebaute Bibliothek

github.com/naphez86/subgraph-sat-solver-econ (Python) nutzt den C++-Solver als Bibliothek. Der Aufbau ist ein anderer: Hier ist die *gemeinsame* Bibliothek selbst das Bauergebnis von CMake, und Python lädt nur noch (`src/native.py`). Der Solver (`Cnf.cpp`, `Reduction.cpp`, `SubgraphSATSolver.cpp`) wird nicht kopiert, sondern als statische Bibliothek `satlib` aus einem Checkout eingebunden; der Shim `wsat_shim` wird dagegen als `SHARED` gebaut.

1. **Solver-Quellen finden** (`find_solver_dir`): Argument, Umgebungsvariable `SUBGRAPH_SAT_SOLVER_DIR`, sonst `third_party/subgraph-sat-solver`; maßgeblich ist das Vorhandensein von `SubgraphSATSolver.cpp`.

2. **Bauen mit CMake** (`build_shim`): `cmake -S src/native -B build -DSUBGRAPH_SAT_SOLVER_DIR=… -DCMAKE_BUILD_TYPE=Release` und `cmake --build`. Das Bauen ist ein ausdrücklicher Schritt, den der Anwender auslöst; die Bibliothek wird nicht beim Laden gebaut. Die `CMakeLists.txt` setzt `CMAKE_POSITION_INDEPENDENT_CODE ON` und blendet Symbole standardmäßig aus (`CMAKE_CXX_VISIBILITY_PRESET hidden`); sichtbar sind nur die mit `WSAT_API` markierten Funktionen. Es gibt bewusst kein `-march=native`, damit die Bibliothek übertragbar bleibt.

3. **Dateinamen je Plattform** (`library_filename`): `wsat_shim.dll` (Windows, MSVC), ersatzweise `libwsat_shim.dll` (MinGW), `libwsat_shim.dylib` (macOS), `libwsat_shim.so` (sonst).

4. **Suchreihenfolge beim Laden** (`NativeLibrary.load`): übergebener Pfad, Umgebungsvariable `WSAT_SHIM_PATH`, `build/<Dateiname>`. Der erste vorhandene Pfad gewinnt; sonst `NativeUnavailable`.

5. **Abhängigkeiten unter Windows.** Alle Verzeichnisse aus `PATH` werden mit `os.add_dll_directory` in die DLL-Suche aufgenommen, damit abhängige DLLs (zum Beispiel die MinGW-Laufzeit) gefunden werden. Unter Linux ist das nicht nötig.

6. **Laden und Version prüfen.** `ctypes.CDLL`, dann `wsat_abi_version()` gegen `ABI_VERSION`; bei Abweichung `NativeUnavailable`.

7. **Vertrag.** `wsat_solve` nimmt dreizehn Argumente: Klauseln als flaches `int`-Feld (jede Klausel mit 0 abgeschlossen), Variablenzahl, Pfad des Subgraph-Programms, Arbeitsverzeichnis, Zeitlimit, zwei Schalter, Modellpuffer, eine Statistikstruktur mit Phasenzeiten und einen Fehlerpuffer. Rückgabe: `SAT`, `UNSAT`, `UNKNOWN` oder `ERROR`. Jede SAT-Antwort wird in Python zusätzlich gegen die Formel geprüft.

Das Verfahren hat *keinen* stillen Rückfall: Wird nichts gefunden oder ist die ABI-Version falsch, löst Python eine Ausnahme aus und es läuft keine andere Implementierung. Das ist für Experimente sinnvoll, bei denen ein stiller Wechsel die Messung verfälschen würde (vgl. Kapitel 11). Auch hier enthält der Wrapper Plattformcode: `WSAT_API` ist unter Windows `__declspec(dllexport)` und sonst `__attribute__((visibility("default")))`. Eine zusätzliche Prozessgrenze liegt *hinter* der Bibliothek: Der Solver startet das

Subgraph-Programm als eigenen Prozess (`execl`); in github.com/naphez86/subgraph-sat-solver-econ ist das ein von `make_launcher` erzeugtes Startskript für das Python-Orakel (**W14**).

13.4 Gegenüberstellung der beiden Verfahren

Aspekt	Gen-DB	subgraph-sat-solver-econ
Ausgangsform der C++-Bibliothek	statische `.a` aus CMake	statische `satlib`, in CMake zum `SHARED`-Ziel gelinkt
Wer erzeugt die gemeinsame Bibliothek?	Python, beim ersten Gebrauch (`g++` direkt)	CMake, ausdrücklicher Bauschritt (`build_shim`)
Build-Werkzeug zur Laufzeit	nur Compiler (`CXX`, `g++`, `c++`, `clang++`)	`cmake` und Compiler
Dateiname	`csubgraph_shim_<hash>.so/.dll/.dylib` im Cache	`wsat_shim.dll`, `libwsat_shim.dll/.so/.dylib` in `build/`
Einstellung	`CSUBGRAPH_LIB_PATH` (Archiv oder Ordner, `.env`)	`WSAT_SHIM_PATH`, `SUBGRAPH_SAT_SOLVER_DIR`
Wiederverwendung des Ergebnisses	Cache nach Hash aus Archiv, Quelle, Header, ABI	Bauordner; Neubau auf Anforderung
Gleichzeitiger Start mehrerer Prozesse	Temporärdatei, `os.replace`, Vorab-Laden im Hauptprozess	nicht vorgesehen (ein Prozess)
Sichtbarkeit der Symbole	`CSUB_EXPORT` (nur Wrapper)	`WSAT_API` und `hidden` als Standard
Abhängigkeiten unter Windows	statisches Linken der MinGW-Laufzeit	`add_dll_directory` für alle `PATH`-Einträge
Versionsprüfung	`csub_abi_version`, `csub_omics_abi_version`	`wsat_abi_version`
Verhalten bei Fehlern	Warnung, Fallback auf Python	Ausnahme `NativeUnavailable`, kein Fallback
Eignung	Dienst mit vielen Vergleichen, soll auch ohne Compiler laufen	Experimente, bei denen die Implementierung feststehen muss

Tabelle 13.2: Die zwei Verfahren zum Laden einer C/C++-Bibliothek.

Beide Verfahren sind für ihren Zweck folgerichtig. Gen-DB soll auf Rechnern ohne passenden Compiler weiterlaufen und baut deshalb erst bei Bedarf, mit Rückfall. github.com/naphez86/subgraph-sat-solver-econ soll eine feste, bekannte Bibliothek messen und vermeidet jede stille Ersetzung. Der Unterschied liegt *nicht* in der Fachaufgabe, sondern darin, wer baut, wann gebaut wird, wie der Name lautet und was bei Fehlern geschieht.

Aufschlussreich ist, wie viel des Codes mit der Fachaufgabe nichts zu tun hat. In den beiden Projekten ist folgender Code reiner Plattformcode:

– `library_filename()` mit vier Dateinamen je nach `sys.platform` und der Sonderfall für MinGW in `build_shim()`;

– der Windows-Block mit `os.add_dll_directory` für jeden `PATH`-Eintrag;

– die Makros `CSUB_EXPORT` und `WSAT_API` mit ihren Windows- und Nicht-Windows-Zweigen;

– die plattformabhängigen Linker-Schalter in Gen-DB (`-fPIC` nicht unter Windows, `--whole-archive` oder `-force_load`, statische MinGW-Laufzeit);

– die plattformabhängige Dateiendung (`.dll`, `.dylib`, `.so`) im Cache-Namen;

– der Abschnitt der README von `github.com/naphez86/subgraph-sat-solver-econ` mit PowerShell-Syntax und vier möglichen Dateinamen.

Ein Befund aus Kapitel 15 gehört in dieselbe Klasse: B1, das fehlende `-fPIC` in der `CMakeLists.txt` von `csubgraph`, führte unter Linux zu einem Linkfehler im Gen-DB-Wrapper (Lauf 8 in Tabelle 14.1). Das ist ein Fehler im Plattformcode, nicht in der Fachlogik, und er tritt erst auf dem zweiten System auf. Das Konzept des folgenden Abschnitts zielt darauf, genau diese Klasse von Code und Fehlern an eine Stelle zu verlegen.

13.5 Das Konzept: eine einheitliche gemeinsame Bibliothek mit standardisiertem Ladeverfahren

Dieser Abschnitt gibt das Konzept des einheitlichen Ladeverhaltens ausführlich wieder. Es liegt als Diskussionsentwurf vor (Markdown-Datei `unified-shared-library-proposal.md`, englisch; Gegenstand dieses Abschnitts ist eine deutsche, etwas ausführlichere Fassung desselben Inhalts). Anlass sind die beiden Projekte aus den Abschnitten 13.2 und 13.3. Der Entwurf schlägt eine *einheitliche gemeinsame Bibliothek* mit der Dateiendung `.sl` (englisch *shared library*) vor: eine Namenskonvention, eine Beschreibung des Containers, eine Regel für den Export von Funktionen und, vor allem, ein *standardisiertes Ladeverfahren*.

13.5.1 Die Kernforderung

> **Das Ladeverfahren muss durch Abstraktion und Standardisierung betriebssystemunabhängig werden.**

Ein Programm soll auf Windows, Linux und macOS mit derselben Schnittstelle, derselben Suchreihenfolge, demselben Fehlermodell und demselben Lebenszyklus sagen können: „lade die Bibliothek `wsat_shim`, finde das Symbol `wsat_solve` und rufe es auf“. Alles Plattformspezifische liegt hinter einer festgelegten Schnittstelle und ist Aufgabe der Lader-Implementierung, nicht jedes einzelnen Projekts. Der Abschnitt erklärt, warum sich das lohnt, welche Teile sich vereinheitlichen lassen und welche nicht, und wie man in Stufen dorthin kommt.

13.5.2 Problemanalyse

Aufwand ohne Mehrwert. Jedes Projekt, das eine native Bibliothek anbietet, muss je Plattform mindestens folgende Fragen lösen:

1. *Benennung:* Welcher Dateiname wird gesucht?
2. *Export:* Wie wird eine Funktion sichtbar? Das verlangt Makros, `.def`-Dateien oder Sichtbarkeits-Schalter.
3. *Suchpfade:* Wo liegen Abhängigkeiten, und wie wird das eingestellt?
4. *Laufzeitbibliothek:* Wird die C++-Laufzeit statisch oder dynamisch gebunden, und wie kommt sie zum Anwender?
5. *Bau:* Welcher Compiler, welche Schalter?
6. *Test:* Läuft alles auf jedem System mit jeder Werkzeugkette?
7. *Ladecode:* Welcher Aufruf, in welcher Reihenfolge, mit welcher Fehlerbehandlung?

Das ist in jedem Projekt dieselbe Arbeit. Sie wird tausendfach wiederholt, und jede Wiederholung führt eigene Fehler ein.

Vermeidbare Fehlerquellen.

- Eine Funktion ist unter Linux sichtbar, unter Windows aber nicht exportiert. Der Fehler zeigt sich erst auf dem zweiten System.
- Eine DLL lässt sich unter Windows nicht laden, weil eine abhängige DLL nicht im Suchpfad liegt. Die Fehlermeldung nennt oft nur die angeforderte Datei, nicht die fehlende Abhängigkeit.
- MinGW- und MSVC-Bibliotheken werden gemischt, und die C++-Laufzeiten passen nicht zusammen.
- Dateien mit gleichem Namen haben verschiedene Bedeutung: Unter macOS ist `.so` ein Plug-in, `.dylib` eine Bibliothek.
- Die Suchpfad-Semantik unterscheidet sich in Feinheiten (aktuelles Verzeichnis, `PATH`, `LD_LIBRARY_PATH`). Das führt zu „läuft bei mir“-Situationen und zu Sicherheitsproblemen.

Verständnis und Einstieg. Für Einsteiger und Fachfremde, etwa Wissenschaftler, die einen Solver aus Python nutzen wollen, ist nicht ersichtlich, warum es drei Namen für dasselbe Konzept gibt. Der Lernaufwand betrifft das Betriebssystem statt der eigentlichen Aufgabe. In `github.com/naphez86/subgraph-sat-solver-econ` und in Gen-DB musste dafür ein eigener README-Abschnitt geschrieben werden.

Abhängigkeit vom Betriebssystem. Code, der eine Bibliothek lädt, kann nicht portabel sein, solange er den Dateinamen und den Lade-Aufruf kennen muss. Jede Abstraktion (`ctypes.CDLL`, `libloading`, `dlopen`-Hüllen) ist ein Hilfsmittel, das genau diese Lücke füllt. Dass es solche Hüllen in jeder Sprache gibt, belegt den Bedarf; jede legt aber ihre eigene Suchreihenfolge, ihre eigenen Fehlermeldungen und ihre eigenen Randfälle fest. *Die Abstraktion existiert, aber sie ist nicht standardisiert.*

13.5.3 Ziele und Nichtziele

Ziele.

1. Eine Namenskonvention auf allen Systemen: `<Name>.sl`.
2. **Ein standardisiertes, betriebssystemunabhängiges Ladeverfahren**, festgelegt durch eine Spezifikation und nicht durch das Verhalten des darunterliegenden Systemladers.
3. Ein Exportmodell: ausdrücklich, auf Anforderung, auf allen Plattformen gleich.
4. Ein Metadatenblock mit ABI-Version, Name, Version und Abhängigkeiten.
5. Eine Suchpfad-Regel mit festgelegter Rangfolge.
6. Ein Fehlermodell mit maschinenlesbaren Codes und lesbaren Meldungen, die auch fehlende Abhängigkeiten benennen.
7. Gleichzeitiges Bestehen mit vorhandenen Bibliotheken (kein Austausch auf einen Schlag).

Nichtziele.

1. Nicht: identischer Maschinencode, der auf jedem Betriebssystem und jeder CPU läuft (Abschnitt 13.5.7).
2. Nicht: PE, ELF oder Mach-O kurzfristig zu ersetzen.
3. Nicht: C++-ABIs zu standardisieren. Die stabile Grenze ist die **C-ABI**.

13.5.4 Die Kernanforderung im Einzelnen: ein betriebssystemunabhängiges Ladeverfahren

Eine einheitliche Dateiendung hilft, aber die eigentliche Vereinheitlichung muss im *Verfahren* geschehen, mit dem eine Bibliothek gefunden, geprüft, geladen, initialisiert, abgefragt und freigegeben wird.

Prinzip: Verhalten festlegen, nicht den Mechanismus. Die Spezifikation legt fest, *was in welcher Reihenfolge geschehen muss*, nicht, welcher Systemaufruf es ausführt. Eine konforme Implementierung darf unter Windows `LoadLibraryEx`, unter Linux und macOS `dlopen` oder einen eigenen Abbilder-Lader im Benutzerraum für ELF, PE und Mach-O verwenden. Der Aufrufer kann den Unterschied nicht erkennen. Das ist derselbe Gedanke wie bei einer POSIX-artigen Portabilitätsschicht, dem Modulsystem einer Sprachlaufzeit oder dem WebAssembly-Komponentenmodell: eine dünne, streng festgelegte Schnittstelle über uneinheitlichen Systemen.

Schichtenarchitektur. Abbildung 13.2 zeigt die Schichten. Alles Betriebssystemspezifische liegt im *Plattformadapter*. Alles darüber ist auf jedem System gleich und lässt sich mit einer einzigen Konformitätssuite einmal testen.

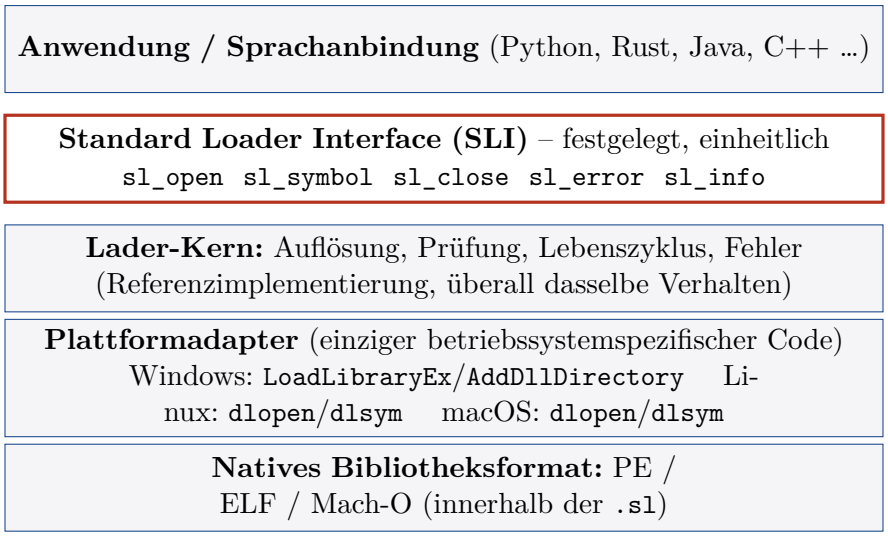


Abbildung 13.2: Schichtenarchitektur des standardisierten Laders. Rot umrandet: die festgelegte Schnittstelle, auf die sich Anwendungen verlassen.

Das standardisierte Ladeverfahren. Ein konformer Lader führt auf jeder Plattform die folgenden neun Schritte in dieser Reihenfolge aus (Abbildung 13.3).

1. **Namensauflösung.** Eingabe ist ein logischer Name (`wsat_shim`) oder ein expliziter Pfad. Der Lader hängt bei Bedarf `.sl` an. Plattform-Präfixe (`lib`) und -Suffixe (`.dll`, `.so.1`) sind nie Teil des logischen Namens.

2. **Auffinden.** Der Lader durchläuft die Suchreihenfolge aus Abschnitt 13.5.4 und hält beim ersten Treffer an. Das Ergebnis des Durchlaufs (alle geprüften Kandidaten) bleibt für die Diagnose erhalten.

3. **Container-Prüfung.** Der Lader liest den `.sl`-Kopf und prüft Kennung, Container-Version, Zielplattform, CPU-Architektur und ABI-Version. Eine Abweichung ergibt einen eigenen Fehler (Abschnitt 13.5.4), *bevor* Code abgebildet wird.

4. **Auflösung der Abhängigkeiten.** Die Abhängigkeitsliste der Metadaten wird rekursiv mit derselben Suchreihenfolge aufgelöst, wobei das Verzeichnis der anfragenden Bibliothek ein fester Suchursprung ist. Zyklen werden erkannt und gemeldet. Alle Abhängigkeiten werden aufgelöst, *bevor* die Bibliothek selbst abgebildet wird, damit eine fehlende Abhängigkeit zu einem Fehler führt, der sie benennt.

5. **Abbilden.** Der Plattformadapter bildet die native Nutzlast mit der Systemfunktion oder dem eigenen Abbilder ein.

6. **Initialisierung.** In festgelegter Reihenfolge: zuerst die Abhängigkeiten, dann die Bibliothek. Der optionale Einstieg `sl_init` wird je Prozess und Handlegruppe genau einmal aufgerufen. Plattformhaken (`DllMain`, `constructor`) werden so gekapselt, dass Reihenfolge und Wiedereintritts-Regeln überall gleich beobachtbar sind.

7. **Bindung der Exporttabelle.** Für `sl_symbol` sind nur die in den Metadaten als exportiert markierten Symbole sichtbar. Die Suche nach einem nicht exportierten Symbol schlägt auf allen Plattformen fehl, auch dort, wo das native Format es sichtbar gemacht hätte.

8. **Rückgabe des Handles.** Der Aufrufer erhält ein undurchsichtiges Handle. Handles sind referenzgezählt; wer dieselbe Bibliothek zweimal öffnet, erhält dieselbe zugrunde liegende Instanz.

9. **Freigabe.** `sl_close` verringert den Zähler. Bei Null läuft `sl_fini` (falls definiert), die Bibliothek wird entladen und Ressourcen werden freigegeben, in umgekehrter Abhängigkeitsreihenfolge.

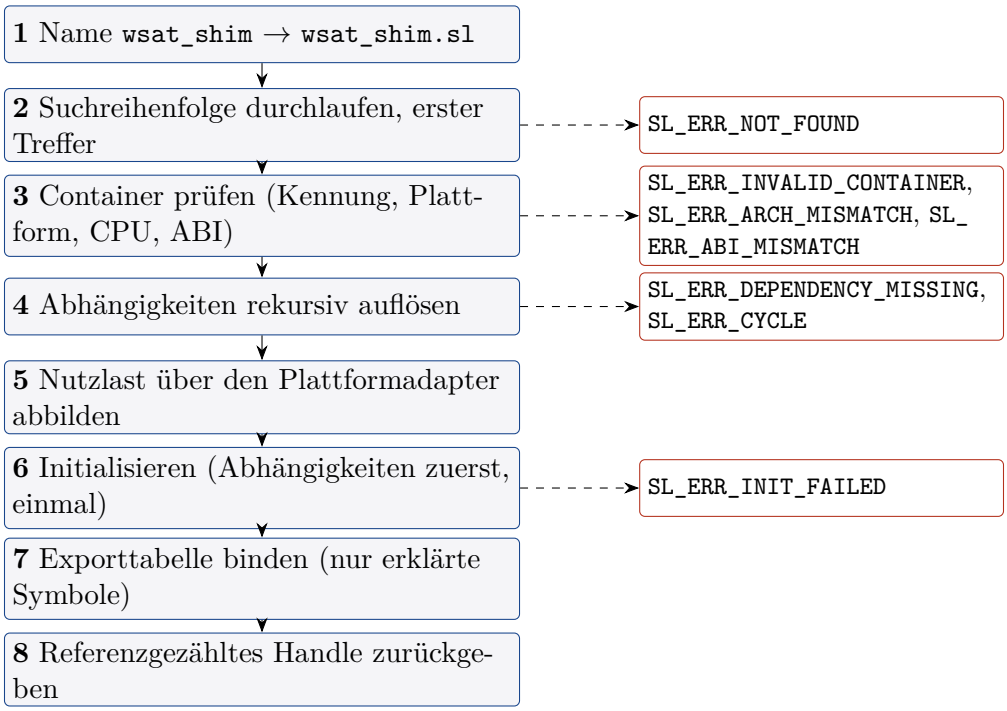


Abbildung 13.3: Ablauf von `sl_open`. Links der Normalfall, rechts (gestrichelt) die Fehlercodes, die an der jeweiligen Stelle auftreten können. Schritt 9, die Freigabe mit `sl_close`, ist nicht eingezeichnet.

Reihenfolge von Initialisierung und Freigabe. Aus den Schritten 4, 6 und 9 folgt eine Eigenschaft, die man beweisen kann.

Satz 13.1 (Reihenfolge von Initialisierung und Freigabe)**.** Sei der Abhängigkeitsgraph der beim Öffnen beteiligten Bibliotheken zyklenfrei. Dann ist die Initialisierungsreihenfolge eine topologische Ordnung des Graphen (Abhängigkeiten zuerst), und die Freigabereihenfolge ist deren Umkehrung. Insbesondere sind beim Aufruf von `sl_init` einer Bibliothek alle ihre Abhängigkeiten initialisiert, und beim Aufruf von `sl_fini` sind alle Bibliotheken, die von ihr abhängen, bereits freigegeben, ihre eigenen Abhängigkeiten aber noch abgebildet.

Beweis. Nach Schritt 4 sind alle Abhängigkeiten einer Bibliothek vor ihr aufgelöst, und nach Schritt 6 werden sie vor ihr initialisiert; das ist die Definition einer topologischen Ordnung, die bei Zyklenfreiheit existiert (bei einem Zyklus bricht Schritt 4 mit `SL_ERR_CYCLE` ab). Schritt 9 gibt in umgekehrter Reihenfolge frei. Ist B von A abhängig, steht in der Initialisierung B vor A, in der Freigabe also A vor B: Wer B nutzt, ist freigegeben, bevor B es wird. □

Suchreihenfolge. Auf jeder Plattform gilt dieselbe Rangfolge; die Suche hält beim ersten Treffer an.

1. der **explizite Pfad**, den der Aufrufer übergibt;
2. die **Umgebungsvariable** `SL_PATH`; das Listentrennzeichen wird in der Spezifikation fest vorgegeben (nicht `;` auf einem System und `:` auf dem anderen), zum Beispiel `:` mit festgelegter Maskierung, oder alternativ über eine Listen-Schnittstelle;
3. das **Verzeichnis des anfragenden Moduls** (der Bibliothek oder des Programms, das `sl_open` aufruft), einschließlich eines festgelegten Unterverzeichnisses wie `./sl/`;
4. das **Anwendungsverzeichnis**;
5. die **systemweiten Verzeichnisse**, die der Adapter je Plattform festlegt, und *nur* diese.

Auf allen Plattformen ausdrücklich ausgeschlossen:

- Das **aktuelle Arbeitsverzeichnis** wird nie implizit durchsucht (verhindert DLL-Hijacking).
- Plattformspezifische Variablen (`PATH`, `LD_LIBRARY_PATH`, `DYLD_LIBRARY_PATH`) werden für die Auflösung von `.sl` *nicht* herangezogen. Das beseitigt eine ganze Klasse von Überraschungen und macht das Verhalten reproduzierbar.
- Abhängigkeiten werden in derselben Reihenfolge gesucht wie die Bibliothek selbst, ohne plattformspezifische Ausnahmen. Der Bedarf an `os.add_dll_directory` entfällt.

Der zweite Punkt lässt sich als Eigenschaft fassen.

Definition 13.1 (Suchliste und Auflösung)**.** Für einen Aufruf `sl_open`(n, p) mit logischem Namen n und optionalem explizitem Pfad p ist die *Suchliste* $S = (s_1, \dots, s_k)$ die Folge der Verzeichnisse: p (falls gegeben), die Einträge von `SL_PATH`, das Verzeichnis des anfragenden Moduls (mit `./sl/`), das Anwendungsverzeichnis und die systemweiten Verzeichnisse des Adapters. Die *Auflösung* ist der erste Eintrag s_i, in dem eine Datei `n.sl` liegt.

Satz 13.2 (Reproduzierbarkeit der Auflösung)**.** Zwei Prozessumgebungen, die in p, `SL_PATH`, Modulverzeichnis, Anwendungsverzeichnis, systemweiten Verzeichnissen und dem Dateibestand dieser Verzeichnisse übereinstimmen, liefern dieselbe Auflösung, unabhängig von `PATH`, `LD_LIBRARY_PATH`, `DYLD_LIBRARY_PATH` und dem Arbeitsverzeichnis.

Beweis. Nach Definition 13.1 ist S eine Funktion nur der genannten Größen, und die Auflösung ist eine Funktion von S und dem Dateibestand. Die ausgeschlossenen Variablen und das Arbeitsverzeichnis gehen in keine der beiden Funktionen ein. □

Der Satz ist bewusst schwach: Er sagt, dass die genannten Einflüsse *ausgeschlossen* sind, nicht, dass der Lader fehlerfrei ist. Bei den heutigen Systemladern gilt eine solche Aussage nicht, weil dort `PATH`, `LD_LIBRARY_PATH` und das Arbeitsverzeichnis je nach Plattform in die Suche eingehen.

Die standardisierte Programmierschnittstelle. Eine kleine, sprachneutrale C-Schnittstelle bildet die normative Schnittstelle; Sprachanbindungen sind dünne Hüllen darüber (Listing 13.2).

Listing 13.2: Normative C-Schnittstelle des Standard Loader Interface

```c
typedef struct sl_handle sl_handle;

typedef enum {
    SL_OK = 0,
    SL_ERR_NOT_FOUND, /* no candidate in the search order */
    SL_ERR_DEPENDENCY_MISSING, /* a declared dependency could not be found */
    SL_ERR_ARCH_MISMATCH, /* wrong CPU architecture or platform target */
    SL_ERR_ABI_MISMATCH, /* incompatible ABI version */
    SL_ERR_INVALID_CONTAINER, /* corrupt or unknown container */
    SL_ERR_SYMBOL_NOT_FOUND, /* symbol absent or not exported */
    SL_ERR_INIT_FAILED, /* sl_init returned failure */
    SL_ERR_CYCLE, /* cyclic dependency */
    SL_ERR_ACCESS_DENIED, /* rejected by security policy */
    SL_ERR_INTERNAL /* platform adapter failure */
} sl_status;

typedef struct {
    sl_status code;
    const char *message; /* human readable, stable wording */
    const char *failed_name; /* the library or dependency that failed */
    const char *searched; /* newline-separated list of paths tried */
    int native_error; /* original OS error code, for diagnostics */
} sl_error;

sl_status sl_open (const char *name_or_path, unsigned flags,
                   sl_handle **out, sl_error *err);
sl_status sl_symbol(sl_handle *h, const char *symbol,
                   void **out, sl_error *err);
sl_status sl_info (sl_handle *h, /* name, version, abi, deps, path */ ...);
sl_status sl_close (sl_handle *h, sl_error *err);
```

Die Sprachanbindungen sehen dann überall gleich aus. In Python:

Listing 13.3: Referenzanbindung in Python (Entwurf)

```python
import sl # reference binding
lib = sl.open("wsat_shim") # identical on Windows, Linux, macOS
solve = lib.symbol("wsat_solve", argtypes=[...], restype=...)
```

Dem steht der heutige Code gegenüber, der eine Plattformunterscheidung, einen Pfadlauf und plattformspezifische Vorbereitung braucht, bevor das erste Symbol benutzt werden kann (Listing 13.6 in Abschnitt 13.5.11).

Fehlermodell. Die Fehlerbehandlung ist heute einer der am wenigsten portablen Teile des Ladens: `GetLastError()` liefert eine Zahl, `dlerror()` einen freien Text, und beide lassen die eigentliche Ursache oft aus. Der Standard verlangt:

– einen festen Satz von **Fehlercodes** (Listing 13.2), die überall dasselbe bedeuten;

– ein Feld `failed_name`, das immer den tatsächlichen Verursacher nennt, also die fehlende Abhängigkeit und nicht nur die angeforderte Bibliothek;

– ein Feld `searched` mit den geprüften Pfaden in der Reihenfolge der Suche;

– den ursprünglichen Betriebssystemfehler, zur Fehlersuche erhalten in `native_error`;

– Meldungen mit stabilem Wortlaut, damit sie dokumentiert und gesucht werden können.

Beispiel der beabsichtigten Diagnose:

Listing 13.4: Beabsichtigte Diagnose bei einer fehlenden Abhaengigkeit

```
SL_ERR_DEPENDENCY_MISSING: cannot load 'wsat_shim'
  missing dependency: 'cryptosolver' (required >= 1.2)
  searched:
    /opt/app/sl/cryptosolver.sl
    /opt/app/cryptosolver.sl
    /usr/local/lib/sl/cryptosolver.sl
```

Zum Vergleich die Meldungen heute: In github.com/naphez86/subgraph-sat-solver-econ lautet die Fehlermeldung „Bibliothek nicht ladbar: …“ und nennt als mögliche Ursachen falsche Architektur, fehlende abhängige DLL oder `.so` und die MinGW-Laufzeit. Welche davon zutrifft, muss der Anwender selbst herausfinden. Gen-DB meldet nur „Wrapper-Bibliothek nicht ladbar“ mit dem Text des Systems und fällt auf Python zurück; die Ursache steht nur im Log.

Lebenszyklus und Threads. Die Spezifikation legt Verhaltensweisen fest, die heute in Feinheiten abweichen:

– `sl_open` und `sl_close` sind thread-sicher. Gleichzeitiges Öffnen derselben Bibliothek erzeugt *eine* Instanz. (Gen-DB löst dasselbe Problem heute mit einer Temporärdatei und `os.replace`; Abschnitt 13.2.)

– Die Initialisierung läuft höchstens einmal und nie, während ein Lader-interner Lock gehalten wird, den Benutzercode wieder betreten könnte.

– Die Initialisierung darf `sl_open` nicht für eine Bibliothek aufrufen, die gerade initialisiert wird (Meldung `SL_ERR_CYCLE`). Das vermeidet die unter Windows bekannte Deadlock-Klasse des Lader-Locks.

– Das Entladen, während Symbole noch benutzt werden, liegt in der Verantwortung des Aufrufers; der Lader garantiert aber auf jeder Plattform dieselbe Referenzzählung.

Standardisierter Symbolexport.

– **Standardmäßig versteckt.** Nur ausdrücklich markierte Symbole werden exportiert, über ein einheitliches Makro oder Attribut (`SL_EXPORT`, später möglicherweise `[[sl::export]]`).

– Das Build-Werkzeug übersetzt das in `__declspec(dllexport)`, `__attribute__((visibility("default")))` oder eine Exportliste, sodass Projekte das nie selbst schreiben.

– Die Liste der exportierten Symbole steht in den `.sl`-Metadaten und wird vom Lader durchgesetzt. Ein Symbol, das nicht in der Liste steht, ist unerreichbar, was auch immer das native Format sichtbar machen würde.

– Die exportierte Schnittstelle ist eine **C-ABI** (`extern "C"`, Ganzzahltypen fester Breite). C++-Typen überschreiten die Grenze nicht.

Beide Projekte halten diese Regel bereits ein, jedes mit eigenem Mittel: Die Schnittstellen `csub_compare` und `wsat_solve` sind `extern "C"` mit `int`- und Zeigertypen (Abschnitte 13.2 und 13.3); nur der Mechanismus (Makro, `hidden`-Voreinstellung) ist verschieden.

Konformitätstests. Ein Standard ist nur so gut wie seine Testsuite. Das Vorhaben sollte eine **Konformitätssuite** mitliefern, die jede Lader-Implementierung, jeder Plattformadapter und jede Sprachanbindung bestehen muss. Sie deckt Suchreihenfolge, Auflösung von Abhängigkeiten, Fehlercodes, Lebenszyklus und die Durchsetzung des Exports ab. So wird gleiches Verhalten geprüft, statt es anzunehmen.

13.5.5 Das Format `.sl`

Benennung.

– Dateiendung `.sl` auf allen Systemen.

– Namensregel `<Name>.sl`, ohne Plattform-Präfix und ohne Versionsendung im Dateinamen. Die Version steht in den Metadaten.

– Optional können Plattform und Architektur in Paketnamen oder in den Metadaten kodiert werden, nicht im Dateinamen (zum Beispiel `wsat_shim-x86_64.sl` nur, wenn für die Verteilung ausdrücklich gewünscht).

Metadatenkopf. Jede `.sl` trägt einen standardisierten Kopf (Tabelle 13.3). Heute baut jedes Projekt einen solchen Kopf selbst; in beiden Beispielprojekten existiert er als Funktion `*_abi_version()` (`csub_abi_version`, `csub_omics_abi_version`, `wsat_abi_version`).

Listing 13.5: Beispiel eines Metadatenkopfes (Skizze)

```
[sl]
container_version = 1
name = "wsat_shim"
version = "1.4.0"
abi_version = 3

[[target]]
os = "windows"
arch = "x86_64"
payload = "pe"

[[target]]
```

Feld	Bedeutung
`magic` / `container_version`	kennzeichnet das Format und seine Fassung
`name`	logischer Bibliotheksname
`version`	Version der Bibliothek (semantische Versionierung)
`abi_version`	Version der exportierten C-Schnittstelle
`targets`	Liste der enthaltenen Nutzlasten je Plattform und Architektur
`dependencies`	logische Namen mit Versionsbereichen
`exports`	Liste der exportierten Symbole (optional mit Signaturhinweisen)
`runtime`	erklärte Laufzeitanforderung, eingebettet oder extern
`init` / `fini`	optionale Einstiegspunkte des Lebenszyklus

Tabelle 13.3: Metadatenkopf einer `.sl`-Datei.

```
os = "linux"
arch = "x86_64"
payload = "elf"

[[target]]
os = "macos"
arch = "arm64"
payload = "macho"

[runtime]
cpp = "embedded" # or "external" with a declared dependency

[[dependency]]
name = "cryptosolver"
version = ">=1.2,<2"

[exports]
symbols = ["wsat_abi_version", "wsat_solve", "wsat_free"]

[lifecycle]
init = "wsat_init"
fini = "wsat_fini"
```

Zwei Auslegungen des Containers. Beide sind möglich und werden unter den offenen Fragen (Abschnitt 13.5.13) diskutiert:

- *Nur Namenskonvention:* `name.sl` ist eine umbenannte gewöhnliche DLL/SO/dylib, die Metadaten stehen in einem Seitenabschnitt oder in einer Begleitdatei.
- *Echter Container:* `name.sl` verpackt eine oder mehrere native Nutzlasten (nach Art einer Fat Binary) zusammen mit dem Metadatenblock.

In beiden Fällen ist das **Ladeverfahren aus Abschnitt 13.5.4 identisch**.

13.5.6 Nutzen

Bereich	Heute	Mit `.sl` und Standardlader
Dateinamen im Code	Fallunterscheidung je Plattform	ein Name
Ladeverfahren	**je Betriebssystem andere Schnittstelle, Suchreihenfolge und Lebenszyklus**	**ein festgelegtes Verfahren**
Export-Makros	je Projekt, plattformabhängig	ein Standardattribut
Suchpfad-Logik	`add_dll_directory`, `RPATH`, `@rpath`	eine Regel
Fehlermeldungen	verschieden, teils irreführend	einheitlich, benennen die fehlende Abhängigkeit
Dokumentation	je Plattform	einmal geschrieben
Einstiegshürde	hoch	niedrig
Testmatrix	3 Formate × Werkzeugketten	ein Verhalten, viele Ziele
Sprachanbindungen (Python, Rust, Java, …)	jede mit eigener Plattformlogik	einmal je Sprache
Sicherheitslage	hängt von den Voreinstellungen des Systems ab	eine gehärtete Richtlinie

Tabelle 13.4: Nutzen eines Standardladers gegenüber der heutigen Lage.

13.5.7 Einwände und ehrliche Einschätzung

Ein tragfähiger Vorschlag muss die Gegenargumente kennen. Einige sind sachlich richtig und bestimmen, wie weit sich vereinheitlichen lässt.

„Gleiche CPU-Architektur heißt gleiche Bibliothek.“ Dieser Schluss gilt nur teilweise. Die CPU-Architektur legt den *Befehlssatz* fest, aber nicht alles, wovon eine Bibliothek abhängt:

- die *Aufrufkonvention* (Windows x64 und System V x86-64 übergeben Register verschieden),
- *Systemaufrufe*, *Speicher- und Threadmodell*, *Laufzeitbibliothek* (MSVCRT/UCRT, glibc, libSystem),
- *Datentypen* (`long` ist unter Windows 32 Bit breit, unter Linux 64 Bit),
- das *Containerformat* (PE, ELF, Mach-O) und die *Symbolauflösung*.

Eine echte Vereinheitlichung des Maschinencodes ist durch bloßes Umbenennen deshalb nicht zu erreichen. Das Ziel des Vorschlags ist eine **einheitliche Schnittstelle, ein einheitlicher Container und ein einheitliches Ladeverfahren**, nicht die Behauptung, derselbe Maschinencode laufe überall unverändert.

„Das Problem ist durch Abstraktionen schon gelöst.“ Teilweise. CMake erzeugt je Plattform die richtige Datei, und `ctypes` und ähnliche Werkzeuge laden sie. Das verschiebt die Arbeit aber nur in Werkzeuge, und jedes verhält sich etwas anders. Jedes Projekt muss weiter über Exportregeln, Pfade und Laufzeit nachdenken, wie die beiden Beispielprojekte zeigen. Die vorhandenen Abstraktionen sind *Bequemlichkeiten der Implementierung, keine Standards*: Nichts garantiert, dass `ctypes`, `libloading` und eine Java-Anbindung dieselbe Bibliothek gleich auflösen. Der Vorschlag macht aus der Abstraktion eine Spezifikation.

Altbestand und Verträglichkeit. Milliarden vorhandener Bibliotheken nutzen die heutigen Formate. Ein neues Format muss daneben bestehen, nicht ersetzen. Ein realistischer Weg ist ein Container oder Lader, der intern die vorhandenen Formate nutzt (Abschnitt 13.5.8). Der Lader sollte außerdem gewöhnliche `.dll`-, `.so`- und `.dylib`-Dateien über dieselbe Schnittstelle öffnen können („Altmodus“), damit die Einführung schrittweise möglich ist.

Wer entscheidet? Dateiformate und Lader gehören den Betriebssystemherstellern und den jeweiligen Ökosystemen. Ein Standard setzt sich nur durch, wenn er auf allen drei Seiten umgesetzt oder als Schicht über den Systemen bereitgestellt wird. Deshalb hängen Stufe 1 und 2 des Fahrplans bewusst nicht von der Mitwirkung der Hersteller ab.

„Ein weiterer Lader ist eine weitere Angriffsfläche.“ Das ist berechtigt. Eine neue Ladeschicht muss kleiner und strenger sein als das, was sie umhüllt (Abschnitt 13.5.9).

13.5.8 Umsetzungspfade

Es gibt drei abgestufte Möglichkeiten.

Stufe 1: Konvention und Werkzeuge (sofort machbar). Ein Build-Werkzeug (zum Beispiel ein CMake-Modul) erzeugt auf jedem System eine Datei `name.sl`, intern eine gewöhnliche DLL, `.so` oder `.dylib`, samt Metadaten. Ein kleines Laufzeitpaket je Sprache stellt `sl_open` und `sl_symbol` bereit und vereinheitlicht Suchpfade, Exportprüfung und Fehlermeldungen *nach dem standardisierten Verfahren aus Abschnitt 13.5.4*. Das senkt den Aufwand bereits erheblich, ohne ein Betriebssystem zu ändern.

Stufe 2: eigener Lader (mittlerer Aufwand). Eine Bibliothek im Benutzerraum lädt ein einheitliches Containerformat, das die passende native Nutzlast je Plattform enthält, ähnlich einer Fat Binary. Das ist durch Apples Universal Binaries und durch Bündelungsformate wie APE (*Actually Portable Executable*) erprobt. In dieser Stufe kann der Lader das Abbilden selbst übernehmen, wodurch das Verhalten von Eigenheiten des Betriebssystems unabhängig wird.

Stufe 3: Unterstützung durch das Betriebssystem (langfristig). Die Hersteller erkennen `.sl` nativ. Das ist politisch und technisch der anspruchsvollste Schritt und hängt von einer Standardisierung ab.

Verwandte Ansätze, die zu prüfen sind.

- WebAssembly mit WASI und dem Komponentenmodell: ein Format, mehrere Laufzeiten, zum Preis von Leistung und Schnittstellenaufwand.
- Portable C-ABI-Schichten in Sprachen wie Rust (`libloading`) als Quelle für Erfahrungen beim Entwurf der Schnittstelle.
- Vorhandene Plug-in-Standards (zum Beispiel Pythons stabile ABI, JNI/Panama, Node-API), die Teile der Grenze bereits standardisiert haben.

13.5.9 Sicherheit

Ein einheitlicher Lader ist nur annehmbar, wenn er *sicherer* ist als der Ist-Zustand.

- **Keine implizite Suche im aktuellen Verzeichnis.** Das verhindert DLL-Hijacking.
- **Keine Abhängigkeit von veränderlicher globaler Umgebung** (`PATH`, `LD_LIBRARY_PATH`) bei der Auflösung. Nur `SL_PATH`; sie lässt sich für privilegierte Prozesse abschalten (gehärteter Modus).
- **Strenge Suchursprünge.** Suchverzeichnisse sind, soweit die Plattform es erlaubt, auf Besitzer und Schreibrechte zu prüfen.
- **Optionale Signatur- und Hashprüfung** der `.sl`-Container, erklärt in den Metadaten.
- **Durchsetzung des Exports** begrenzt die erreichbare Oberfläche einer Bibliothek auf die erklärte Schnittstelle.
- **Im Zweifel abbrechen.** Jede Mehrdeutigkeit (zwei Kandidaten mit widersprüchlichen Metadaten, eine ABI-Abweichung) führt zu einem Fehler, nicht zu einer stillen Wahl.

13.5.10 Risiken

- **Zersplitterung.** Ein weiterer Standard neben drei vorhandenen kann das Problem zunächst vergrößern (der „xkcd-927-Effekt“). Gegenmaßnahme: ein Altmodus, der die neue Schicht zur Obermenge statt zum Konkurrenten macht.
- **Sicherheit.** Ein einheitlicher Lader muss Suchpfade streng regeln, sonst wird das Nachladen manipulierter Bibliotheken möglich. DLL-Hijacking ist unter Windows schon heute ein bekanntes Problem.
- **Leistung.** Zusätzliche Schichten dürfen Laden und Aufrufe nicht merklich verlangsamen. Für die hier betrachteten Fälle (wenige Aufrufe mit großen Datenmengen) ist das unkritisch. Die Symbolsuche liefert einfache Funktionszeiger, sodass nach der Bindung kein Aufwand je Aufruf entsteht.
- **Pflege.** Jemand muss Spezifikation, Werkzeuge, Konformitätssuite und Referenzimplementierung pflegen.

– **Auseinanderlaufen der Plattformen.** Betriebssysteme ändern ihr Ladeverhalten. Der Plattformadapter fängt das ab, braucht aber fortlaufende Tests in der Continuous Integration auf allen drei Systemen.

13.5.11 Anwendung auf die beiden Beispielprojekte

Mit Stufe 1 ließe sich in beiden Projekten Folgendes entfernen oder vereinfachen:

– `library_filename()` und die MinGW-Sonderfälle in `build_shim()`;

– der Windows-Block mit `add_dll_directory`;

– das Makro `CSUB_EXPORT` (und `WSAT_API`) und die plattformabhängigen Exportzeilen;

– die plattformspezifischen Teile der README-Anleitung (PowerShell-Zeilenfortsetzung, vier Dateinamen);

– die Sonderbehandlung der statischen Laufzeit für MinGW, sofern die Laufzeit im Container erklärt ist.

Vorher und nachher am Beispiel der Dateinamenswahl:

Listing 13.6: Vorher: Dateiname je Plattform (aus `native.py`, subgraph-sat-solver-econ)

```
def library_filename() -> str:
    if sys.platform == "win32":
        return "wsat_shim.dll"
    if sys.platform == "darwin":
        return "libwsat_shim.dylib"
    return "libwsat_shim.so"
```

Listing 13.7: Nachher: dasselbe mit dem Standardlader (Entwurf)

```
import sl
lib = sl.open("wsat_shim") # same code on every system
```

Die beiden Verfahren aus den Abschnitten 13.2 und 13.3 würden dabei *nicht* gleich: Ob erst bei Bedarf gebaut wird und ob ein Rückfall auf Python besteht, ist eine Entscheidung des Projekts und bleibt es. Vereinheitlicht wird nur, *wie* die Bibliothek gefunden, geprüft und geladen wird. Das Fallback-Verhalten von Gen-DB ließe sich auf `sl_open` aufsetzen, indem ein Fehlercode (zum Beispiel `SL_ERR_NOT_FOUND` oder `SL_ERR_ABI_MISMATCH`) den Wechsel auslöst, statt dass jede Fehlerart einzeln aus dem Systemtext abgeleitet wird. Auch der Fehler B1 (fehlendes `-fPIC`) würde von einem Build-Modul abgedeckt, das Positionsunabhängigkeit und Sichtbarkeit je Plattform selbst setzt.

13.5.12 Empfohlener Fahrplan

1. **Das Ziel schärfen.** Benennung, Export, Suchpfad, Metadaten und vor allem das **Ladeverfahren und seine Schnittstelle** vereinheitlichen. Den Maschinencode nicht vereinheitlichen.

2. **Zuerst die Spezifikation schreiben.** Suchreihenfolge, Lebenszyklus, Fehlercodes, Metadaten und Exportattribut, als versioniertes Dokument, offen für Beiträge von außen.

3. **Den Referenzlader bauen (Stufe 1).** Ein kleiner C-Kern mit Plattformadaptern für Windows, Linux und macOS sowie ein CMake-Modul. Mit der Python-Anbindung beginnen, da beide Projekte `ctypes` nutzen.

4. **Die Konformitätssuite parallel schreiben.** In der Continuous Integration auf allen drei Systemen und mit den Werkzeugketten MSVC und MinGW ausführen.

5. **Die beiden Beispielprojekte migrieren** und messen, wie viel Plattformcode entfällt und wie viele Fehlerarten verschwinden.

6. **Erst nach nachgewiesenem Nutzen** Stufe 2 (Container, Fat Binaries) und Stufe 3 (Unterstützung durch Betriebssysteme) verfolgen.

13.5.13 Offene Fragen

- Soll `.sl` ein eigener Container sein oder nur eine Namenskonvention über vorhandenen Formaten?
- Wie wird die C++-Laufzeit behandelt (eingebettet, mitgeliefert, als vorhanden angenommen)?
- Wie werden mehrere Architekturen (x86-64, ARM64) in einer Datei dargestellt?
- Welche Versionierungs- und Kompatibilitätsregeln gelten (ABI-Versionierung, semantische Versionsbereiche für Abhängigkeiten)?
- Wie genau lauten Trennzeichen und Maskierungsregel für `SL_PATH`, damit sie auf jedem System identisch sind?
- Soll der Lader verzögertes Binden und verzögertes Laden unterstützen, und mit welchen garantierten Eigenschaften?
- Wie wird das Entladen behandelt bei Bibliotheken, die Threads starten oder Rückrufe registrieren?
- Wer pflegt Spezifikation, Referenzimplementierung und Konformitätssuite?

Aus der Sicht dieser Arbeit kommt eine Frage hinzu. Die Suchreihenfolge hält beim *ersten* Treffer an (Definition 13.1), der Abschnitt zur Sicherheit verlangt aber einen Fehler bei *zwei Kandidaten mit widersprüchlichen Metadaten*. Beides ist vereinbar, wenn der Lader im Normalbetrieb dem ersten Treffer folgt und nur im gehärteten Modus auch die späteren Suchursprünge auf widersprüchliche Kandidaten prüft; der Entwurf legt das nicht fest.

13.5.14 Stand

Das Konzept ist ein Diskussionsentwurf. In `github.com/naphez86/subgraph-sat-solver-econ` ist es nach Angabe der README ausdrücklich *nicht umgesetzt*; dort gelten die Dateien `.dll`, `.so` und `.dylib`. Auch in Gen-DB wird heute der in Abschnitt 13.2 beschriebene Weg benutzt. Ausblick und nächste Schritte stehen in Kapitel 18.

Teil IV

Prüfung

Kapitel 14

Kompatibilitätsprüfung

Die Prüfung beantwortet eine praktische Frage: Funktionieren die Verbindungen zwischen den Fassungen und Projekten, wenn man sie aus den aktuellen Quellen baut? Sie macht keine Aussage über Laufzeit in anderen Umgebungen.

14.1 Aufbau

Alle Programme wurden unter Linux mit GCC 13 aus den Quellen gebaut, ohne CMake (kein Netzzugriff auf die `FetchContent`-Quellen): Adapter und Bibliothek mit `g++ -std=c++17 -O2` aus `SubgraphAlgorithm.cpp`, `MultiOmics.cpp`, `ExactSubgraph.cpp` und `SolverCli.cpp`, der Solver aus `Cnf.cpp`, `Reduction.cpp`, `SubgraphSATSolver.cpp` und `main.cpp` (Abschnitt 16.2). Die Python-Referenz und die Neuimplementierungen von Gen-DB wurden direkt importiert. Die Zufallsinstanzen wurden mit festen Seeds erzeugt; die Erzeugungsskripte sind nicht Teil dieser Arbeit.

Die Prüfläufe 1 bis 8 in Tabelle 14.1 stammen aus einer vorangegangenen Prüfung mit derselben Methode. Die Zeilen mit Stern wurden bei der Erstellung dieser Fassung neu ausgeführt.

14.2 Ergebnisse

Gen-DBs eigene Windows-Messdateien wurden nicht neu gemessen, aber gelesen: Python 1180,3 s gegen C++ 185,6 s bei 931 899 Kandidaten, also das 6,36-Fache (Tabelle 9.3).

14.3 Interpretation

Die Läufe 2 bis 4 bestätigen, dass die Neuimplementierungen und der exakte Test dem jeweiligen Original entsprechen. Lauf 1 zeigt, dass die Referenz in einem Randfall von der Portierung abweicht und die Portierung die richtige ist. Die Läufe 5 und 10 zeigen, dass der Signaturmodus nicht als Orakel taugt, und Lauf 8 zeigt einen Bauzwang, den nur ein Linux-Versuch findet. Die Aussagen gelten für die geprüften Instanzen und nicht für alle Eingaben; insbesondere beweist die Prüfung nichts über die Laufzeit des exakten Modus bei großen Formeln.

Nr.	Prüfgegenstand	Umfang	Ergebnis
1	Python-Referenz gegen C++ (**W1**)	20 000 Zufallspaare	127 Abweichungen, alle bei zwei identischen 1×1-Graphen (Python `keep_both`, C++ `IDENTICAL`); sonst keine
2	C++-`MultiOmics` gegen `multiomics_python` (**W2** gegen **W9**)	12 000 Vergleiche	keine Abweichung
3	`ExactSubgraph` gegen Brute-Force (**W3**)	20 000 Paare	keine Abweichung, keine ungültige Einbettung
4	Solver, Modus `exact` (**W14**)	300 Zufalls-CNFs	keine falsche Antwort, keine ungültige Belegung
5	Solver, Modus `heuristic` (**W15**)	dieselben 300 Formeln	130 falsche Antworten (erfüllbar als unerfüllbar), 5 Fehlerstatus
6	Aufrufkosten Adapter (Prozess)	Mittelwert	3,41 ms
7	Aufrufkosten In-Prozess	Mittelwert	0,0185 ms (etwa 184-fach geringer)
8	Linken des Gen-DB-Wrappers unter Linux	ein Bauversuch	scheitert ohne `-fPIC`; mit `-fPIC` erfolgreich
9*	Paarcharakterisierung, Satz 2.2, gegen Rotation und Tabelle	200 000 Zufallspaare kurzer Folgen	keine Abweichung
10*	Beispiel aus Abschnitt 10.4: Solver und Adapter	ein Lauf je Modus	`exact`: $\{x_1{=}T, x_2{=}F, x_3{=}T\}$, Mapping $[0, 3, 4]$; `heuristic`: `false`

Tabelle 14.1: Prüfläufe und Ergebnisse. * = in dieser Fassung neu ausgeführt.

Kapitel 15

Befunde und Verbesserungen

Die folgende Liste fasst alle Befunde aus Quelltextlesung und Prüfung zusammen, geordnet nach Priorität.

Tabelle 15.1: Befunde mit Verbesserungsvorschlägen. Priorität: H hoch, M mittel, N niedrig.

Nr.	Befund	Auswirkung	Verbesserung
B1 (H)	`CMakeLists.txt` von `csubgraph` setzt kein `-fPIC`	Linken des Gen-DB-Wrappers scheitert unter Linux (Lauf 8)	`set(CMAKE_POSITION_INDEPENDENT_CODE ON)`
B2 (H)	Exit-Code 20 des Solvers steht für `UNSAT`, `UNKNOWN` und Fehler	Frameworks, die nur den Code lesen, zählen einen Adapter-Zeitablauf als `UNSAT`	eigener Code für Fehler, Text weiter auswerten
B3 (H)	Solver-Zeitlimit wird nicht an den Adapter weitergereicht	zwei unabhängige Limits, Standard 60 s im Adapter	`setenv` vor `execl`
B4 (M)	`pyproject.toml` von Gen-DB hängt auf `@main`	Änderung am Zweig ändert Verhalten ungeprüft	feste Version (Tag oder Commit)
B5 (M)	Ergebniscodes an vier Stellen kodiert (`Cli.cpp`, zwei Wrapper, `csubgraph_native.py`)	Änderung an einer Stelle bricht die anderen	gemeinsamer Header, Test auf Konsistenz
B6 (M)	`SubgraphAlgorithm` prüft die 63-Knoten-Grenze nicht	Verschiebung um ≥ 64 Bit ist undefiniert	Grenze in `calculateSignatures` prüfen, Ausnahme
B7 (M)	Referenz liefert für identische 1×1-Graphen `keep_both`	Abweichung zu C++ in 127 von 20 000 Fällen	Gleichheitsprüfung in Python ergänzen
B8 (M)	Solver erkennt Antworten über Textsuche mit fester Schreibweise	jede Abweichung (z. B. ohne Leerzeichen) ergibt `UNSAT`	JSON-Parser oder tolerantere Suche
B9 (M)	Modus `heuristic`: 130 falsche Antworten, 5 Fehlerstatus; Ursache der Fehler unbekannt	kein Orakel im Sinne von Def. 2.9	Fehler untersuchen; Modus nur als Diagnose kennzeichnen

Nr.	Befund	Auswirkung	Verbesserung
B10 (N)	Drei Konto-Namen in den Quellen: `naphez86` (`pyproject.toml` Gen-DB, Solver-TeX), `naphets86` (Gen-DB-Quellen, Solver-README), `hjstephan86` (Solver-, Subgraph-, csubgraph-README)	tote Verweise	ein Konto festlegen, Verweise angleichen
B11 (N)	Header des Solvers nennt „polynomielle Zeit“	falsch für den exakten Modus	Kommentar korrigieren
B12 (N)	`CMakeLists.txt` des Solvers fordert `nlohmann/json`, nutzt es aber nicht	unnötiger Download	Anforderung entfernen
B13 (N)	Hilfsfunktion `computeLCS` berechnet einen längsten gemeinsamen *Teilstring*	Missverständnis	umbenennen (`longestCommonSubstring`)
B14 (N)	Zeitlimit von `ExactSubgraph` deckt Vorverarbeitung nicht ab	Limit kann überschritten werden	`tick()` auch in Kern und Färbung

15.1 Mängel der früheren Fassung dieser Arbeit

Die Neufassung behebt folgende Mängel der vorherigen: fehlendes Literaturverzeichnis (`\cite` lief ins Leere), unaufgelöste Verweise (`sec:gendb_native`, `sec:protokoll`, `sec:solver_aufruf`, `sec:solver_komplexitaet`), Zeichenfehler („Gräße“, „kännen“, „wärtlich“, „abgeläst“), die falsche Bezeichnung des Solvers (`SubgraphSATSolver` statt `subgraph-sat-solver` als Programmname), die Einordnung der Kapitel 5 bis 10 als Abschnitte von Kapitel 4, die fehlenden Beweise zur Belegungsextraktion und das fehlende durchgerechnete Beispiel (beides jetzt in Abschnitt 10.3 und 10.4). Außerdem war die Aussage falsch, Gen-DB enthalte keine eigene Implementierung der Relation (Tabelle 3.1).

Kapitel 16

Bau- und Aufrufanweisungen

16.1 Bibliothek und Programme mit CMake

```
cd csubgraph
mkdir -p build && cd build
cmake -DCMAKE_POSITION_INDEPENDENT_CODE=ON .. # -fPIC, noetig fuer Gen-DB unter
    ↪ Linux
cmake --build .
```

Es entstehen `libsubgraphlib.a`, `subgraph-cli`, `subgraph` und `subgraph-tests`. CMake lädt dabei `nlohmann/json` und GoogleTest nach (Netzzugriff nötig).

16.2 Ohne CMake

Adapter und Solver lassen sich ohne CMake bauen, wie in Kapitel 14:

```
C=csubgraph; S=subgraph-sat-solver
g++ -std=c++17 -O2 -I$C $C/SubgraphAlgorithm.cpp $C/MultiOmics.cpp \
    $C/ExactSubgraph.cpp $C/SolverCli.cpp -o subgraph
g++ -std=c++17 -O2 -I$S $S/Cnf.cpp $S/Reduction.cpp \
    $S/SubgraphSATSolver.cpp $S/main.cpp -o solver
```

16.3 Adapter und Solver aufrufen

```
export SUBGRAPH_MODE=exact # Standard; "heuristic" nur zur Diagnose
export SUBGRAPH_TIMEOUT_SEC=60
./subgraph input.json output.json # Adapter einzeln
./solver --subgraph-binary ./subgraph f1.cnf # Solver mit Adapter
```

Die Eingabedatei des Adapters enthält die zwei Graphen `G` und `H` mit den Schlüsseln `nodes` und `edges`. Die Ausgabe ist `{"is_subgraph": true/false}` und bei `true` im exakten Modus zusätzlich `"mapping"`.

16.4 Gen-DB mit der Bibliothek

```
export CSUBGRAPH_LIB_PATH=/pfad/zu/csubgraph/build # oder in die .env
export CXX=g++ # Compiler fuer den Wrapper
python src/experiment_search_backends.py # Experiment Python gegen C++
```

Ohne die Variable läuft Gen-DB mit dem Python-Paket. Gen-DB wählt je Prozess einmal und gibt im Log eine Warnung aus, wenn die Bibliothek gesetzt, aber nicht nutzbar ist. Mit `get_backend_name()` lässt sich prüfen, welche Fassung läuft.

Teil V

Schluss

Kapitel 17

Zusammenfassung

Der Subgraph Algorithmus wurde in Python entwickelt und seither auf achtzehn Weisen wiederverwendet (Katalog, Kapitel 8). Diese Arbeit hat diese Wiederverwendungen geordnet, die tragenden Aussagen bewiesen oder geprüft und die Kette der Projekte von der Python-Referenz bis zu den Experimenten auf Schwachstellen untersucht. Dieses Kapitel fasst zusammen, was dabei herausgekommen ist; was daraus folgen sollte, steht im anschließenden Ausblick (Kapitel 18).

17.1 Die Wiederverwendungen

Die achtzehn Wiederverwendungen **W1** bis **W18** lassen sich in sieben Formen einteilen. Sie unterscheiden sich darin, ob der *Algorithmus* selbst, seine *Relation*, die *Bibliothek* oder nur die *Infrastruktur* übernommen wird (Kapitel 3 und 8). Aufschlussreich ist, was *nicht* wiederverwendet wird: Der exakte Teilgraphtest, den der Subgraph-SAT-Solver braucht, verwendet vom Signaturalgorithmus gar nichts (**W3**, **W14**); der Algorithmus ist dort ein austauschbares Orakel. Die Kette reicht von der Python-Referenz (Kapitel 4) über die C++-Portierung `csubgraph` mit ihren Erweiterungen `MultiOmics` und `ExactSubgraph` (Kapitel 5 und 6) und deren Außenflächen (JSON-Programm, Datei-Adapter, C-Schnittstelle) bis zu den drei Anwendungen Gen-DB, Subgraph-SAT-Solver und Experimente-Framework.

17.2 Die vier inhaltlichen Ergebnisse

1. Die *Paarcharakterisierung* (Satz 2.2) zeigt, dass die Relation des Algorithmus ein Paarschnitt ist. Sie vereint **W2**, **W9**, **W10** und **W11** und führt zum invertierten Paarindex, der Vergleiche ganz vermeidet (Satz 9.1). In Gen-DB bringt der Index nach den eigenen Messdateien das rund 80- bis 115-Fache gegenüber Einzelvergleichen in Python, bei gleichen Trefferlisten.

2. Die *Reduktionskette* des Subgraph-SAT-Solvers von CNF über 3-SAT und Clique auf Subgraph ist korrekt, und die Belegungsextraktion liefert aus jeder Einbettung eine erfüllende Belegung (Satz 10.1). Ein durchgerechnetes Beispiel und ein echter Lauf bestätigen das an einer kleinen Formel.

3. Die *Prüfung* (Kapitel 14) findet in den geprüften Instanzen keine Abweichung zwischen Python-Referenz und C++-Fassung, mit einer erklärbaren Ausnahme bei

identischen 1×1-Graphen. Der Signaturmodus des Solvers ist als Orakel ungeeignet (130 von 300 Antworten falsch).

4. Die *Kopplungsarten* haben klare Einsatzfelder (Kapitel 12): Bibliothek im Prozess für Massenvergleiche (Gen-DB), Prozess mit Dateien für austauschbare Orakel (Solver), Übernahme der Idee, wo sie sich auf einen Index abbilden lässt.

17.3 Das Zusammenspiel der Sprachen: das Sequenzdiagramm

Wie die Bausteine bei einer einzigen Suche zusammenspielen, zeigt das UML-Sequenzdiagramm in Abschnitt 9.8. Es folgt dem Use Case *Subgraph-Suche* in Gen-DB von der Eingabe im Browser bis zur Anzeige des Ergebnisses und hat neun Lebenslinien in drei Sprach- und Prozessbereichen. Fünf Einsichten sind festzuhalten. Die Anfrage überquert drei Grenzen (Netz, Prozess, Sprache), und nur die Sprachgrenze Python→C++ ist billig, weil sie innerhalb des Worker-Prozesses als Funktionsaufruf verläuft (rund 0,018 ms). Der Algorithmus ist austauschbar, die Suche nicht: Nur ein einziger Rahmen des Diagramms enthält Algorithmus-Code. Die Auswahl zwischen C++ und Python fällt einmal je Prozess, nicht je Vergleich. Fehler laufen als Werte, nicht als Ausnahmen. Und die Reihenfolge der Kandidaten bleibt erhalten. Die Messdateien erklären, warum der Gesamtgewinn der C++-Fassung (6,36) kleiner ist als der Gewinn je Einzelvergleich (≈ 37): Der Rechenkern ist danach nicht mehr der Engpass, der serielle Rest der Suche macht 66,7 % der Zeit aus. Das ist eine Aussage über die Kopplung, nicht über den Algorithmus.

17.4 Das Laden der Bibliothek und das einheitliche Konzept

Gen-DB und `github.com/naphez86/subgraph-sat-solver-econ` laden ihre C/C++-Bibliothek auf grundverschiedene Weise (Kapitel 13). Gen-DB baut den Wrapper beim ersten Gebrauch aus einem statischen Archiv mit dem Compiler, legt ihn unter einem Hash-Namen in einen Cache, lädt ihn über `ctypes` und fällt bei jedem Fehler mit einer Warnung auf Python zurück. Das Econ-Projekt lädt eine fertig mit CMake gebaute Bibliothek, prüft ihre ABI-Version, trägt unter Windows die `PATH`-Ordner ein und bricht ohne Rückfall mit einer Ausnahme ab. Beide Verfahren sind für ihren Zweck folgerichtig; der Unterschied liegt in der Fachaufgabe (Dienst gegen Experiment), nicht im Können. Beide enthalten aber viel Code, der mit der Fachaufgabe nichts zu tun hat: Dateinamen je Betriebssystem, Exportmakros, Suchpfade, Linker-Schalter, Anleitungstext. Der Befund B1 (fehlendes `-fPIC`) gehört in dieselbe Klasse.

Daraus entwickelt die Arbeit das Konzept einer *einheitlichen gemeinsamen Bibliothek* mit der Endung `.sl`, ausführlich nach dem Diskussionsentwurf. Seine Kernforderung lautet: Das Ladeverfahren muss durch Abstraktion und Standardisierung betriebssystemunabhängig werden. Festgelegt wird das *Verhalten*, nicht der Mechanismus. Dazu gehören die Schichtenarchitektur mit dem Plattformadapter als einziger betriebssystemspezifischer Stelle, ein Ladeverfahren aus neun festen Schritten, eine feste Suchreihenfolge, die Programmierschnittstelle (`sl_open`, `sl_symbol`, `sl_close`, `sl_error`, `sl_info`) mit einem einheitlichen Fehlermodell, Regeln für Lebenszyklus und Threads, ein standardisierter Symbolexport, ein Metadatenkopf und eine Konformitätssuite. Die Arbeit diskutiert Nutzen,

Einwände (CPU-Architektur ist nicht gleich Bibliothek, Altbestand, Zuständigkeit, Angriffsfläche), drei Umsetzungsstufen, Sicherheit und Risiken und zeigt am Beispiel der beiden Projekte, welcher Plattformcode entfiele. Ein Beweis zeigt, dass aus den Schritten 4, 6 und 9 eine feste Reihenfolge von Initialisierung und Freigabe folgt. Der Entwurf ist *nicht* umgesetzt; er ist ein Konzept mit Fahrplan und offenen Fragen.

17.5 Die Experimente

Das Experimente-Framework (Kapitel 11) startet den Subgraph-SAT-Solver als Prozess und misst Laufzeiten auf Boolean Circuits. Die Arbeit hat die zugehörige wissenschaftliche Arbeit des Frameworks ausgewertet; neu gemessen wurde nichts. Zwei Messreihen liegen vor.

Die *erste Reihe* (288 Läufe, vier Suites) zeigt über Instanzgrößen von 4 bis 3 512 Gates keine Abhängigkeit der Laufzeit von der Größe ($\rho = 0{,}024$). Das ist aber kein Ergebnis über den Algorithmus: Die Werte häufen sich bei 15,625 und 31,25 ms (grobe Uhrauflösung), in keinem Lauf wurde ein Ergebnis erfasst, die Kodierung von NAND, NOR und XOR war fehlerhaft, und die Formeln waren unabhängig vom Circuit erfüllbar. Als Befund bleibt nur eine obere Schranke der Prozesslaufzeit.

Die *neue Version 2.0.0* behebt diese Mängel (korrigierte Kodierung, Ausgangsbedingung, Nanosekundenuhr, Basislinie, Warm-up, Median, Statusauswertung mit Vorrang der `Status:`-Zeile, neun unerfüllbare Miter-Instanzen, ein Python-Referenzsolver). Sie liegt als Zwischenstand mit 44 von 46 Instanzen vor. Der Subgraph-SAT-Solver löst alle erfüllbaren Instanzen bis 1 193 Gates; seine Laufzeit wächst deutlich (ISCAS'85: Exponent $2{,}08 \pm 0{,}07$ über der Literalzahl, von 515,75 ms bei c432 auf 20 427,66 ms bei c2670; Target-AND-Random: Faktor 4,1 zwischen 27 und 54 Gates). In allen 33 gemeinsam entschiedenen Fällen stimmen beide Solver überein. Ab 10 487 Literalen endet der Solver mit `ERROR`, und von den neun unerfüllbaren Miter-Instanzen löst er nur die drei kleinsten. Die Grenzen sind ebenso klar: Der Exponent gilt für die ganze Pipeline, nicht für die Schranke $\Theta(n^3)$ des Algorithmus; der Referenzsolver ist kein Maßstab für MiniSat oder CaDiCaL; es liegen nur Mediane vor; die Ursache der Ausfälle ist ungeklärt; Modus und Version des Solvers sind nicht dokumentiert.

17.6 Prüfung und Befunde

Die Prüfung unter Linux (GCC 13) hat die Verträglichkeit der Fassungen untersucht und vierzehn Befunde mit Verbesserungsvorschlägen ergeben (Kapitel 15): drei hohe (B1 bis B3: fehlendes `-fPIC`, mehrdeutiger Exit-Code 20, nicht weitergereichtes Zeitlimit), sechs mittlere und fünf niedrige. Zwei Befunde berühren die Experimente unmittelbar. B2 ist genau die Verwechslung, die die erste Messreihe in ihrer Auswertung nicht vermied und die die neue Version durch die Priorität der Textzeile umgeht. Und die Ursache der fünf Fehlerstatus im Modus `heuristic` (B9) bleibt wie die der `ERROR`-Läufe ab 10 487 Literalen offen.

17.7 Grenzen der Arbeit

– Der exakte Modus ist im ungünstigsten Fall exponentiell; die Schranke $O(n^3)$ gilt nur für den Signaturalgorithmus.

– Die Gen-DB-Messungen stammen aus einer Windows-Umgebung und wurden nicht neu erhoben.

– Der Quelltext des Experimente-Frameworks wurde nicht geprüft; alle Zahlen des Experimente-Kapitels stammen aus seiner eigenen Arbeit.

– Die neue Messreihe ist ein Zwischenstand (ein Kern, 3,9 GiB, `-O2`); es fehlen Einzelläufe, Rückgabecodes und die beiden größten Skalierungsinstanzen.

– Das Konzept des einheitlichen Ladeverhaltens ist ein Entwurf ohne Referenzimplementierung und ohne Messung seines Nutzens.

– Die Ursache der Fehlerstatus im Modus `heuristic` ist ungeklärt.

Kapitel 18

Ausblick

Aus den Befunden, den Experimenten und dem Konzept des einheitlichen Ladeverhaltens ergeben sich viele weitere Schritte. Dieses Kapitel ordnet sie nach Arbeitsfeldern und gibt zu jedem das Ziel, den Ausgangspunkt in dieser Arbeit, die nötigen Arbeitsschritte, das Kriterium, an dem sich der Erfolg messen lässt, und, wo es eine gibt, die erwartbare Schwierigkeit. Abschnitt 18.8 schlägt am Ende eine Reihenfolge vor. Vorhersagen über Messergebnisse sind als Erwartungen gekennzeichnet; belegt sind sie erst nach der Messung.

18.1 Sofort umsetzbar: die Befunde beheben

18.1.1 Die drei hohen Befunde

Ziel: die gefährlichsten Verwechslungen beseitigen. **Ausgangspunkt:** B1 bis B3 (Tabelle 15.1). **Schritte:** (1) In der `CMakeLists.txt` von `csubgraph` `CMAKE_POSITION_INDEPENDENT_CODE` setzen, damit der Gen-DB-Wrapper unter Linux linkt (B1). (2) Dem Solver einen eigenen Exit-Code für Fehler geben, sodass 20 nur noch `UNSAT` bedeutet (B2); der Text bleibt maßgeblich, bis alle Aufrufer umgestellt sind. (3) Das Zeitlimit des Solvers vor dem Aufruf des Adapters in die Umgebung setzen, damit nicht zwei unabhängige Limits gelten (B3). **Erfolgskriterium:** Der Linux-Lauf der Prüfung (Lauf 8) gelingt ohne Handeingriff, und ein künstlich ausgelöster Zeitablauf wird vom Framework nicht mehr als `UNSAT` gezählt. Zusammen sind es wenige Zeilen.

18.1.2 Die mittleren und niedrigen Befunde

B4 bis B14 sind einzeln klein, aber nicht folgenlos. Die Gruppierung nach Ursache hilft bei der Reihenfolge.

- *Reproduzierbarkeit:* B4 (Abhängigkeit auf `@main` durch einen festen Tag oder Commit ersetzen) und B10 (drei Konto-Namen in den Quellen vereinheitlichen, damit Verweise nicht ins Leere laufen).
- *Gemeinsame Verträge:* B5 (die Ergebniscodes 0 bis 5 stehen an vier Stellen; ein gemeinsamer Header in `csubgraph` und ein Konsistenztest in Gen-DB beseitigen das Risiko) und B8 (die Textsuche im Solver durch einen JSON-Parser ersetzen).
- *Korrektheit an den Rändern:* B6 (Prüfung der 63-Knoten-Grenze in `calculateSignatures` mit Ausnahme statt undefinierter Verschiebung), B7

(Gleichheitsprüfung in der Python-Referenz, damit sie bei identischen 1×1-Graphen wie C++ entscheidet) und B14 (`tick()` auch in Kern und Färbung von `ExactSubgraph`, damit das Zeitlimit die Vorverarbeitung deckt).

– *Hygiene:* B11 (Kommentar „polynomielle Zeit“ im exakten Modus korrigieren), B12 (ungenutzte `nlohmann/json`-Anforderung entfernen) und B13 (`computeLCS` in `longestCommonSubstring` umbenennen).

Für B6 und B7 lohnt ein Regressionstest, der die Randfälle der Prüfung (Graphen mit 63 und 64 Knoten, identische 1×1-Graphen) dauerhaft in die Testsuite aufnimmt, sodass die Kompatibilitätsprüfung aus Kapitel 14 nicht jedes Mal von Hand wiederholt werden muss.

18.1.3 Der Modus `heuristic`

Ziel: entscheiden, ob der Signaturmodus Diagnosewerkzeug oder Orakel sein soll. **Ausgangspunkt:** B9, 130 falsche Antworten von 300 und fünf Fehlerstatus. **Schritte:** (1) Die fünf Fehlerstatus einzeln nachstellen und die Ursache klären (Grenze der Signaturbreite, Eingabeform, Zeitlimit?). (2) Die falschen Antworten nach Art einteilen: Der Signaturalgorithmus berechnet eine Relation, die für den Teilgraphtest notwendig, aber nicht hinreichend ist; erwartet werden deshalb falsch-positive Antworten, falsch-negative wären ein Fehler. (3) Entweder den Modus ausdrücklich als Vorfilter dokumentieren, dessen Ergebnis ein exakter Test bestätigen muss, oder ihn entfernen. **Kriterium:** Jede falsche Antwort ist erklärt oder behoben; der Modus kann nicht mehr unbemerkt als Orakel im Sinne von Definition 2.9 verwendet werden.

18.2 Der Subgraph Algorithmus und seine Verwandten

18.2.1 Der Paarindex für weitere Relationen

Die Wiederverwendung **W11** brachte den größten Gewinn. Offen ist, ob sich eine ähnliche Charakterisierung für andere Relationen finden lässt. **Zu klären:** (1) Die Paarcharakterisierung (Satz 2.2) gilt für die Relation des Algorithmus; welche Abwandlungen (Gewichte, gerichtete Kanten, Knoten- und Kantenlabels zugleich) erhalten die Paarform? (2) Für die klassische Teilgraph-Relation ist das Problem NP-vollständig; ein Index kann dort höchstens als *Vorfilter* dienen, der keine Treffer verliert, aber Kandidaten verwirft. Die Filterwirkung ließe sich an den Gen-DB-Daten messen: Anteil der verworfenen Kandidaten, Anteil der Treffer, die der Filter verliert (muss 0 sein). (3) Wie verhalten sich Speicher und Aufbauzeit des Index bei wachsender Datenbank? **Erwartung:** Der Gewinn bleibt bei der Relation des Algorithmus am größten und ist bei der klassischen Relation auf den Vorfilter beschränkt.

18.2.2 Laufzeit und Schranken

Die Schranke $\Theta(n^3)$ gilt für den Signaturalgorithmus, nicht für die ganze Pipeline. Eine saubere Messung würde die Kosten in ihre Anteile zerlegen. **Schritte:** (1) Im Solver je

Lauf die Phasenzeiten (Reduktion, Aufruf des Subgraph-Programms, Extraktion) ausgeben, soweit sie der Ergebnistyp bereits trägt, und im Experimente-Framework mitschreiben. (2) Den Exponenten getrennt für die Reduktion (erwartet nahe 2 in der Literalzahl, weil die Adjazenzmatrix durch Paarvergleiche entsteht), für den Algorithmus (erwartet höchstens 3 in der Knotenzahl des Clique-Graphen) und für die Datei-Ein- und Ausgabe bestimmen. (3) Beim Fit die Knotenzahl des Clique-Graphen statt der Literalzahl verwenden, die nach der Normalisierung auf höchstens drei Literale je Klausel etwas kleiner ist. **Kriterium:** Die Summe der Anteile stimmt mit der Gesamtzeit überein, und die gemessenen Exponenten lassen sich der theoretischen Schranke zuordnen.

18.2.3 Worst-Case-Instanzen

Die bisherigen Instanzen sind einfache AND-Bäume, kleine Zufallscircuits und ISCAS'85. Für den Algorithmus fehlen Eingaben, die gezielt viele gleiche Signaturen oder lange gemeinsame Teilstrings erzeugen. Eine Konstruktion solcher Familien würde zeigen, ob die Schranke erreicht wird oder die Laufzeit praktisch darunter bleibt. Für den exakten Test (exponentiell im ungünstigsten Fall) sind schwere Instanzen aus der Literatur der Cliquen- und Isomorphieprobleme zu verwenden; erwartet wird, dass die Laufzeit dort viele Größenordnungen vom Signaturalgorithmus abweicht und das Zeitlimit von `ExactSubgraph` (B14) wirksam wird.

18.2.4 Die 63-Knoten-Grenze

Die Signatur nutzt ein Maschinenwort. Dass `SubgraphAlgorithm` die Grenze nicht prüft (B6), ist der sichtbare Teil eines größeren Punktes: Größere Graphen brauchen eine Signaturdarstellung aus mehreren Wörtern. **Schritte:** Die Darstellung auf ein Feld von Wörtern verallgemeinern, die Reihenfolge der Bits so festlegen, dass die Rotation und der längste gemeinsame Teilstring unverändert funktionieren, und die Python-Referenz als Orakel verwenden. **Kriterium:** Gleichheit von Python und C++ auf Graphen mit mehr als 63 Knoten in der Differenzialprüfung. Offen ist, wie stark die Verallgemeinerung die Laufzeit pro Vergleich erhöht; die Messung in Gen-DB (rund 0,074 ms je Einzelvergleich) liefert die Vergleichsbasis.

18.3 Gen-DB und die Suche

18.3.1 Neumessung unter Linux

Die Gen-DB-Messungen stammen aus einer Windows-Umgebung mit vier logischen CPUs. **Schritte:** (1) Denselben Messlauf (`search_backends`, `search_universal`) unter Linux mit gleichen Daten und gleicher Anfragefolge wiederholen. (2) Dabei Betriebssystem, Compiler, Kernzahl und Python-Version aufschreiben. (3) Die Ergebnisse (Verhältnis 6,36 im Gesamtlauf, ≈ 37 je Einzelvergleich, 80- bis 115-fach für den Index) vergleichen. **Erwartung:** Die Verhältnisse bleiben in der Größenordnung, die absoluten Zeiten ändern sich. Weichen die Verhältnisse deutlich ab, ist die Ursache (Prozessstart, Pickle, Betriebssystem-Scheduler) zu klären, denn das Sequenzdiagramm legt nahe, dass der serielle Anteil die Gesamtzeit bestimmt.

18.3.2 Der serielle Anteil der Suche

Nach den Messdateien macht der serielle Anteil mit der C++-Fassung 66,7 % der Zeit aus (Datenbankabfrage, Prozesskommunikation, Trefferliste). Das ist der nächste Engpass, nicht mehr der Algorithmus. Mögliche Maßnahmen, jeweils mit eigener Messung: die Kandidatenabfrage mit einem Index auf Knoten- und Kantenzahl beschleunigen, die Matrizen als gemeinsamen Speicher statt per `pickle` an die Worker geben, die Chunkgröße (256) und die Zahl der Worker an Kernzahl und Datenmenge anpassen, und die Treffer schrittweise ausliefern, statt die ganze Liste zu sammeln. Das Sequenzdiagramm zeigt, an welchen Schritten (Nachrichten 3 bis 7 und 15 bis 17) das ansetzt.

18.3.3 Weitere Eingabearten

Die Multi-Omics-Suche und die universelle Suche folgen demselben Ablauf. Offen ist, wie sich weitere Eingabearten einfügen (etwa Netzwerke mit Kantenlabels oder gerichtete Netzwerke), ohne dass die Fachlogik verzweigt. Das Sequenzdiagramm legt nahe, den Algorithmus-Rahmen als einzige Stelle zu behandeln, die von der Eingabeart abhängt, und die Treffer- und Antwortlogik gemeinsam zu halten.

18.3.4 Ein zweites Sequenzdiagramm

Das vorhandene Diagramm beschreibt den Erfolgsfall. Sinnvoll sind zwei Ergänzungen: ein Diagramm des *Fehlerpfads* (Bibliothek nicht ladbar, Fallback auf Python, Warnung im Log) und eines für den *Subgraph-SAT-Solver* mit den Lebenslinien Aufrufer, Reduktion, Adapter, Subgraph-Programm und Belegungsextraktion. Beide würden die Prozess- und Dateigrenzen sichtbar machen, die im Fließtext nur beschrieben sind.

18.4 Der Subgraph-SAT-Solver

18.4.1 Ursache der Ausfälle ab 10 487 Literalen

Alle sechs `ERROR`-Läufe haben mindestens 10 487 Literale; alle Instanzen mit höchstens 7 958 Literalen enden mit Ergebnis oder Zeitlimit. Das legt eine Größengrenze nahe, belegt aber keine Ursache. **Schritte:** (1) Rückgabecode und Fehlermeldung je Lauf sichern. (2) Den Spitzenspeicher von Reduktion und Subgraph-Prozess messen (das Framework erfasst `ru_maxrss` bereits). (3) Dieselben Instanzen auf einer Maschine mit deutlich mehr Speicher wiederholen. **Hypothesen:** Speichermangel (die eigene Abschätzung in Abschnitt 11.10 nennt bis zu etwa $5{,}5 \cdot 10^7$ Literalpaare bei 10 487 Literalen), ein fest eingebautes Größenlimit im Solver oder in einer Datei-Schnittstelle, oder ein Fehler beim Schreiben sehr großer JSON-Dateien. Welche zutrifft, entscheidet die Messung; die Fehlerzeiten von 24,77 bis 40,74 s liegen deutlich unter dem Zeitlimit, schließen also einen Zeitablauf aus.

18.4.2 Speicherärmere Reduktion

Wenn der Speicher die Grenze setzt, ist die Reduktion der Ansatzpunkt: Die Adjazenzmatrix des Clique-Graphen wächst quadratisch mit den Literalen. Mögliche Auswege sind eine dünn besetzte Darstellung, die Auslagerung in Blöcke, oder eine Vorverarbeitung, die die Formel verkleinert (Einheitsklauseln propagieren, subsumierte Klauseln entfernen), bevor reduziert wird. Zu messen sind Speicherbedarf und Laufzeit gegen die Literalzahl vor und

nach der Änderung; die Korrektheit der Kette (Satz 10.1) muss die Änderung erhalten und lässt sich mit dem Referenzsolver auf denselben Dateien prüfen.

18.4.3 Unerfüllbare Instanzen

Der Solver löst nur die drei kleinsten der neun Miter-Instanzen. Offen ist, ob das an der Unerfüllbarkeit oder an den langen Klauseln der Miter-Formeln liegt (Klauseln mit 26 bis 33 Literalen gegenüber höchstens 10 in den ISCAS-Formeln); beides ändert sich in den Daten gleichzeitig. **Schritte:** Paare von Instanzen mit gleicher Literalzahl und gleicher Klauselänge, aber unterschiedlicher Erfüllbarkeit erzeugen, etwa Miter mit und ohne eingebauten Unterschied; außerdem die Tseitin-Kodierung der Miter-Gates mit kürzeren Klauseln bauen (Hilfsvariablen für die ODER-Verknüpfung der Ausgänge). **Kriterium:** Die Laufzeit hängt bei gleicher Literalzahl und Klauselänge nicht (oder erkennbar) von der Erfüllbarkeit ab.

18.4.4 Vergleich mit etablierten Solvern

Der Python-Referenzsolver ist ausdrücklich kein Maßstab. **Schritte:** MiniSat und CaDiCaL über dasselbe Aufrufprotokoll (Exit-Codes 10 und 20, `s`-Zeile) anbinden und auf denselben DIMACS-Dateien messen; die Basislinie je Solver bestimmen. **Erwartung:** Etablierte Solver lösen alle Instanzen der bisherigen Experimente in Millisekunden, sodass sich die Frage verschiebt: Wo, wenn überhaupt, ist der Subgraph-Weg konkurrenzfähig? Plausible Kandidaten sind Instanzen mit viel gleicher Struktur, bei denen der Paarindex Vergleiche spart. Wichtig ist, dass das Ergebnis ehrlich berichtet wird, auch wenn es negativ ist: Die Reduktionskette ist ein Beweis der Korrektheit und der Wiederverwendung eines Orakels, kein Geschwindigkeitsversprechen.

18.4.5 Alternative Orakel

Der exakte Teilgraphtest in der Kette verwendet vom Signaturalgorithmus nichts (**W3**, **W14**). Eine Folgeuntersuchung könnte weitere Teilgraph-Orakel austauschbar anbinden und über die Dateischnittstelle vergleichen, etwa eine eigene Implementierung nach dem VF2-Verfahren oder einen vorhandenen Isomorphie-Solver. Das Protokoll (Datei in, Exit-Code und Text aus) ist dafür vorgesehen; zu klären ist, wie das Orakel Modus, Zeitlimit und Version meldet (Abschnitt 18.5.1).

18.5 Das Experimente-Framework

18.5.1 Metadaten in jeder Messreihe

Die Arbeit des Frameworks nennt weder das benutzte Subgraph-Programm noch den Modus (`SUBGRAPH_MODE`, `exact` oder `heuristic`), die Zeitlimits des Adapters noch die Versionsstände von Solver, Adapter und Bibliothek. Ein Lauf im Modus `heuristic` misst aber einen anderen Algorithmus als einer im Modus `exact`. **Schritte:** Das Framework schreibt je Messreihe eine Metadatei mit Modus, Zeitlimits (Adapter und Solver), Commit-Hashes aller beteiligten Repositories, Compiler und Compilerschaltern, Betriebssystem, Kernzahl und Speicher. Die Datei wird vor dem ersten Lauf geschrieben, damit sie auch bei einem abgebrochenen Lauf vorliegt (der Lauf der neuen Version brach bei `TargetAND_m10000` ab,

und die am Ende geschriebenen Dateien fehlten). **Kriterium:** Aus den Ergebnisdateien allein lässt sich jede Messung einem Build und einer Konfiguration zuordnen.

18.5.2 Vollständige Wiederholung des Full-Presets

Schritte: Das Full-Preset auf einer Maschine mit mehr Speicher und mehr als einem Kern wiederholen; alle Ergebnisdateien sichern, auch die Einzelläufe; Rückgabecodes und Fehlermeldungen je Lauf speichern; die Skalierungsinstanzen $m = 10\,000$ und $m = 50\,000$ messen; den Solver mit CMake (`-O3 -march=native`) statt `g++ -O2` bauen und beide Builds vergleichen. Dass die Einzelläufe fehlten, verhinderte jede Aussage über die Streuung; mit ihnen lassen sich Quartile, Konfidenzintervalle und ein Test auf die Nicht-Monotonie bei $d = 8$ und $d = 9$ nachholen.

18.5.3 Mehr Messpunkte und bessere Fits

Der Exponent $2{,}08 \pm 0{,}07$ stützt sich auf sechs Circuits, die zugleich Größe, Struktur und Gate-Mischung ändern. Besser wäre eine Familie mit einem Parameter: Target-AND mit $m = 200$ bis 800 schließt die Lücke zwischen 1 094 und 10 994 Literalen. Weiter helfen mehr Wiederholungen bei den großen Instanzen (heute 13, 6 und 3 Läufe wegen des Zeitbudgets) und ein Vergleich verschiedener Fit-Formen (reines Potenzgesetz, Potenzgesetz mit Achsenabschnitt), um zu sehen, ob der Exponent von der Wahl abhängt.

18.5.4 Weitere Instanzfamilien

Offen sind unter anderem: strukturell andere Benchmarks (die Hardware-Verifikation hat weitere Familien als ISCAS'85), Zufalls-3-SAT nahe dem Phasenübergang (Verhältnis von Klauseln zu Variablen um 4,27), Probleme aus der Praxis der Wiederverwendung (Gen-DB-Netzwerke als Cliquen- und Teilgraphprobleme) und Instanzen mit bekannter, aber schwer zu findender Lösung. Hinzu kommen Strukturmaße der ISCAS-Circuits (Tiefe, Fan-out, Clustering) und ihr Einfluss auf die Laufzeit. Jede Familie braucht dieselbe Disziplin wie die neue Version: Status je Lauf, Modellprüfung, Basislinie.

18.5.5 Die Messreihen reproduzierbar halten

Die Auswertung der neuen Version (Skript `paper_analysis_v2.py`) erzeugt alle Zahlen des Berichts aus den Rohdateien. Dieses Vorgehen sollte für diese Arbeit übernommen werden: Jede Zahl des Experimente-Kapitels sollte per Skript aus einer maschinenlesbaren Datei in die Tabelle gelangen, statt von Hand abgeschrieben zu werden, und ein Test sollte Abweichungen zwischen Text und Daten melden.

18.6 Das einheitliche Laden von Bibliotheken

Das Konzept (Abschnitt 13.5) ist ein Entwurf. Sein eigener Fahrplan und die Offene-Fragen-Liste sind der Ausgangspunkt; dieser Abschnitt ergänzt, wie sich das Vorhaben in dieser Arbeit überprüfen ließe.

18.6.1 Der Fahrplan in Schritten

1. **Das Ziel schärfen:** Benennung, Export, Suchpfad, Metadaten und vor allem das Ladeverfahren und seine Schnittstelle vereinheitlichen, nicht den Maschinencode.
2. **Die Spezifikation schreiben:** Suchreihenfolge, Lebenszyklus, Fehlercodes, Metadaten und Exportattribut als versioniertes Dokument.
3. **Den Referenzlader bauen (Stufe 1):** ein kleiner C-Kern mit Plattformadaptern für Windows, Linux und macOS sowie ein CMake-Modul; mit der Python-Anbindung beginnen, da beide Projekte `ctypes` nutzen.
4. **Die Konformitätssuite parallel schreiben,** in der Continuous Integration auf allen drei Systemen und mit den Werkzeugketten MSVC und MinGW.
5. **Die beiden Beispielprojekte migrieren** und messen.
6. **Erst nach nachgewiesenem Nutzen** Stufe 2 (Container, Fat Binaries) und Stufe 3 (Betriebssystemunterstützung) verfolgen.

18.6.2 Den Nutzen messen

Der größte Mangel des Entwurfs ist, dass sein Nutzen behauptet, aber nicht gemessen ist. Die Migration der beiden Projekte liefert die Gelegenheit. **Messgrößen:** Zeilen Plattformcode vor und nach der Migration (die Liste in Abschnitt 13.5.11 nennt die betroffenen Stellen: `library_filename()`, der `add_dll_directory`-Block, die Exportmakros, die Linker-Schalter, die plattformabhängigen Teile der README); Zahl der Fehlerarten, die der Lader mit einem Code statt eines Systemtextes meldet; Zeit bis zum ersten erfolgreichen Lauf eines neuen Nutzers auf jedem System; Zahl der Plattform-Sonderfälle in der Continuous Integration. **Kriterium:** Beide Projekte laufen auf Windows, Linux und macOS mit demselben Ladecode, und die Messgrößen sinken.

18.6.3 Die offenen Fragen

Der Entwurf nennt acht offene Fragen (Container oder Namenskonvention, C++-Laufzeit, mehrere Architekturen, Versionierung, Trennzeichen von `SL_PATH`, verzögertes Laden, Entladen bei Threads, Zuständigkeit). Aus dieser Arbeit kommt eine neunte hinzu: Die Suchreihenfolge hält beim *ersten* Treffer an, der Abschnitt zur Sicherheit verlangt aber einen Fehler bei zwei Kandidaten mit widersprüchlichen Metadaten. Ein möglicher Ausweg ist ein Normalbetrieb, der dem ersten Treffer folgt, und ein gehärteter Modus, der auch spätere Suchursprünge prüft. Diese Entscheidung ist in der Spezifikation festzuhalten und in der Konformitätssuite zu testen. Für die Frage der C++-Laufzeit liefert Gen-DB einen Hinweis: Dort wird die MinGW-Laufzeit unter Windows statisch gelinkt, im Econ-Projekt dagegen werden alle `PATH`-Ordner eingetragen; ein Lader müsste beide Fälle abdecken oder die Laufzeit im Container erklären.

18.6.4 Zwischenlösung: ein gemeinsames Hilfspaket

Bevor ein Lader existiert, lässt sich ein Teil des Nutzens schon mit einem kleinen gemeinsamen Python-Paket erreichen, das beide Projekte importieren: Dateiname je Plattform, Suchreihenfolge, `add_dll_directory`, ABI-Prüfung und einheitliche Fehlertexte. Es ist

keine Standardisierung, aber es entfernt die doppelte Pflege und liefert Erfahrung für die Spezifikation. Zu entscheiden ist, ob das Paket aus dem Econ-Projekt (strenges Verhalten ohne Rückfall) oder aus Gen-DB (Hash-Cache und Rückfall) abgeleitet wird; empfehlenswert ist ein gemeinsamer Kern mit einem Schalter für den Rückfall, sodass die zwei Verfahren als zwei Konfigurationen derselben Bibliothek erscheinen.

18.6.5 Ausweitung auf weitere Sprachen

Beide Projekte nutzen `ctypes`. Der Entwurf nennt weitere Sprachanbindungen (Rust, Java, C++). Eine nützliche erste Erweiterung wäre eine zweite Sprache neben Python, um zu prüfen, ob die Schnittstelle sprachneutral ist: Wenn die Anbindung in Rust nur die Funktionen `sl_open`, `sl_symbol`, `sl_close`, `sl_error` und `sl_info` aufrufen muss und ihr Verhalten durch dieselbe Konformitätssuite geprüft wird, trägt die Abstraktion.

18.7 Verbesserungen an dieser Arbeit

- **Die Quelltextprüfung des Experimente-Frameworks nachholen.** Heute stammen alle Zahlen aus seiner eigenen Arbeit. Ein Lauf des Frameworks im Rahmen dieser Prüfung, mit denselben Aussagearten (gelesen, geprüft) wie bei den anderen Projekten, würde die Lücke schließen.

- **Die Experimente unter der Prüfumgebung wiederholen,** damit die Messwerte in Kapitel 11 nicht nur zitiert, sondern *geprüft* im Sinn von Abschnitt 1.2 sind.

- **Die Katalog-Tabelle und das Abhängigkeitsbild aus den Quellen erzeugen,** damit sie bei einer neuen Wiederverwendung nicht von Hand nachgezogen werden müssen.

- **Die Befunde als Tickets führen** und in dieser Arbeit nach dem Beheben als erledigt kennzeichnen.

18.8 Eine Reihenfolge

Die Arbeitsfelder hängen voneinander ab. Eine sinnvolle Reihenfolge nach Nutzen und Aufwand:

1. **Sofort (Stunden bis Tage):** die drei hohen Befunde (B1 bis B3), die Metadatei je Messreihe und das Sichern von Rückgabecode und Einzelläufen.

2. **Kurzfristig (Tage bis Wochen):** die mittleren und niedrigen Befunde, die vollständige Wiederholung des Full-Presets mit mehr Speicher, die Neumessung von Gen-DB unter Linux und die Klärung des Modus `heuristic`.

3. **Mittelfristig (Wochen bis Monate):** die Ursache der Ausfälle ab 10 487 Literalen und die speicherärmere Reduktion, die Zerlegung der Laufzeit in Phasen, der Vergleich mit MiniSat und CaDiCaL, die Worst-Case-Instanzen und die Erweiterung der Signatur über 63 Knoten.

4. **Langfristig:** der Referenzlader und die Konformitätssuite, die Migration beider Projekte, der Paarindex für weitere Relationen und, nach nachgewiesenem Nutzen, die Stufen 2 und 3 des Konzepts.

Zwei Abhängigkeiten sind zu beachten. Erstens setzt jede Aussage über die Skalierung des Solvers die Metadatei und die Einzelläufe voraus; ohne sie bleiben Zahlen unvergleichbar. Zweitens sollten B1 und B5 vor der Migration auf einen gemeinsamen Lader behoben sein, weil beide sonst im Lader als vermeintliche Ladefehler erscheinen. Die Messungen und Zusagen dieses Kapitels sind Erwartungen, die erst durch die jeweilige Messung zu belegen sind.

Literaturverzeichnis

[1] S. Epp: *Der Subgraph Algorithmus*, 2026. https://github.com/naphez86/subgraph.

[2] R. M. Karp: Reducibility among Combinatorial Problems. In: R. E. Miller, J. W. Thatcher (Hrsg.): *Complexity of Computer Computations*, Plenum Press, 1972, S. 85–103.

[3] M. R. Garey, D. S. Johnson: *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.

[4] G. S. Tseitin: On the Complexity of Derivation in Propositional Calculus. In: *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968.

[5] F. Brglez, H. Fujiwara: A Neutral Netlist of 10 Combinational Benchmark Circuits and a Target Translator in Fortran. *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 1985.

[6] S. Epp: *Learning M-DNF in Boolean Circuits*. Masterarbeit, Universität Paderborn, 2013.

[7] S. A. Cook: The Complexity of Theorem-Proving Procedures. *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, 1971.

[8] Google: *GoogleTest*, Version 1.14.0, https://github.com/google/googletest.

[9] N. Lohmann: *JSON for Modern C++*, Version 3.11.2, https://github.com/nlohmann/json.